

Guia de reciclaje de componentes web

Documento 1 - Funcionalidades 1 a 10 en el orden de la tabla de contenido original

Este documento corrige y reorganiza la guía para seguir exactamente el orden de la tabla de contenido del recurso unificado: Navbar fija glass, Sidebar ocultable, Submenus laterales, Router de secciones, Dropdown superior, Tema claro/oscuro, Scroll progress, Back to top, Reveal on scroll y Contador animado.

La idea no es solo ver el código, sino aprender a reciclar cada funcionalidad en cualquier proyecto. Por eso cada componente incluye: para que sirve, que copiar, que cambiar, código HTML, CSS, JavaScript modular, JavaScript normal, pasos de implementación, errores comunes y checklist.

Convenciones para usar esta guía

Convencion	Significado	Ejemplo
ID	Identificador unico. No debe repetirse en la pagina.	id="sidebar"
Clase	Nombre reutilizable para aplicar CSS o seleccionar elementos.	class="glass-navbar"
data-*	Atributo personalizado para guardar datos que JS puede leer.	data-section="inicio"
is-open / is-visible	Clases de estado. JS las agrega o quita para cambiar la interfaz.	.sidebar.is-open
Modular	Código con export/import. Recomendado para proyectos organizados.	export class SidebarToggle {}
Normal	Código sin módulos. Util cuando quieres un solo archivo script-normal.js.	function initSidebarToggle() {}

Orden corregido de funcionalidades 1 a 10

#	Funcionalidad	Para que sirve	Archivo principal que toca
1	Navbar fija glass	Barra superior fija con blur y transparencia.	HTML + CSS + JS
2	Sidebar ocultable	Menu lateral que se abre y cierra.	HTML + CSS + JS
3	Submenus laterales	Opciones internas desplegables.	HTML + CSS + JS
4	Router de secciones	Muestra una seccion diferente en el body.	HTML + JS
5	Dropdown superior	Menu desplegable en navbar.	HTML + CSS + JS
6	Tema claro/oscuro	Cambia variables CSS y guarda preferencia.	CSS + JS
7	Scroll progress	Barra que muestra avance de scroll.	HTML + CSS + JS
8	Back to top	Boton para volver arriba.	HTML + CSS + JS

#	Funcionalidad	Para que sirve	Archivo principal que toca
9	Reveal on scroll	Animacion al aparecer en pantalla.	CSS + JS
10	Contador animado	Cuenta de 0 hasta un valor objetivo.	HTML + JS

Estructura recomendada de archivos

Para reciclar cualquier componente en proyectos nuevos, mantén una estructura simple. Puedes usar modulos o version normal, pero no mezcles las dos al mismo tiempo en la misma pagina para evitar duplicar eventos.

Estructura sugerida

```
mi-proyecto/
├─ index.html
├─ styles.css
├─ funciones.js      // clases y funciones exportadas
├─ script.js        // importa y arranca componentes
└─ script-normal.js // alternativa sin import/export
```

Regla practica: si usas import/export, en HTML carga `<script type="module" src="script.js"></script>`. Si no usas modulos, carga solo `<script src="script-normal.js"></script>`.

1. Navbar fija glass

La navbar fija glass es una barra superior que permanece visible mientras el usuario hace scroll. Usa transparencia, blur y sombra para verse cristalina sin tapar totalmente el fondo. Es muy usada en landing pages, dashboards, portafolios, ecommerce y aplicaciones web modernas.

Donde se usa en paginas reales: cabeceras de paginas, menus principales, paneles administrativos, sitios de marca, portafolios y single page applications.

Dependencias internas: puede trabajar sola, pero normalmente se conecta con el Router de secciones mediante botones con data-section.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*.	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<header class="glass-navbar" id="mainNavbar">
  <a href="#inicio" class="glass-navbar__brand">Mi Proyecto</a>

  <nav class="glass-navbar__links" aria-label="Menu principal">
    <button type="button" data-section="inicio">Inicio</button>
    <button type="button" data-section="servicios">Servicios</button>
    <button type="button" data-section="contacto">Contacto</button>
  </nav>

  <button type="button" class="glass-navbar__menu" id="openSidebar">
    Menu
  </button>
</header>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

/* Variable util para que el contenido no quede debajo de la navbar */
:root {
  --navbar-height: 72px;
  --glass-bg: rgba(255, 255, 255, 0.14);
  --glass-border: rgba(255, 255, 255, 0.28);
  --primary: #2563eb;
}

body {
  padding-top: var(--navbar-height);
}

.glass-navbar {
  position: fixed;
  top: 0;
  left: 0;
  z-index: 1000;
  width: 100%;
  min-height: var(--navbar-height);
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  padding: 14px 24px;
  background: var(--glass-bg);
  border-bottom: 1px solid var(--glass-border);
  backdrop-filter: blur(18px);
  -webkit-backdrop-filter: blur(18px);
  box-shadow: 0 12px 30px rgba(15, 23, 42, 0.18);
}

.glass-navbar__brand {
  color: #ffffff;
  font-weight: 800;
  text-decoration: none;
}

.glass-navbar__links {
  display: flex;
  gap: 10px;
}

.glass-navbar button {
  border: 0;
  border-radius: 999px;
  padding: 10px 14px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.12);
  cursor: pointer;
}

.glass-navbar button:hover,
.glass-navbar button.is-active {
  background: var(--primary);
}

@media (max-width: 720px) {
  .glass-navbar__links {
    display: none;
  }
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```
export class GlassNavbar {
  constructor(navbarId = "mainNavbar") {
    this.navbar = document.getElementById(navbarId);
    this.buttons = this.navbar?.querySelectorAll("[data-section]") || [];
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => {
        this.setActive(button);
      });
    });
  }

  setActive(activeButton) {
    this.buttons.forEach((button) => button.classList.remove("is-active"));
    activeButton.classList.add("is-active");
  }
}
```

script.js - como llamarlo

```
// script.js
import { GlassNavbar } from './funciones.js';

new GlassNavbar();
```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initGlassNavbar(navbarId = "mainNavbar") {
  const navbar = document.getElementById(navbarId);
  if (!navbar) return;

  const buttons = navbar.querySelectorAll("[data-section]");

  buttons.forEach((button) => {
    button.addEventListener("click", () => {
      buttons.forEach((item) => item.classList.remove("is-active"));
      button.classList.add("is-active");
    });
  });
}

initGlassNavbar();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
#mainNavbar	HTML y JS	Cambia el id solo si tambien cambias el argumento new GlassNavbar('nuevold').
.glass-navbar	CSS y HTML	Puedes renombrarla, pero cambia todas sus reglas CSS.
data-section	Botones	Debe coincidir con las secciones si usas router.
--navbar-height	:root	Ajusta la altura si tu navbar es mas alta o mas baja.

Elemento	Donde esta	Como adaptarlo
--glass-bg	:root	Sube o baja la transparencia; 0.10 es mas transparente, 0.30 mas solido.

Paso a paso para implementarlo en otro proyecto

1. Copia el bloque <header> justo despues de abrir <body>.
2. Copia las variables CSS y las reglas de .glass-navbar en tu styles.css.
3. Verifica que el body tenga padding-top para que el contenido no quede escondido debajo de la navbar.
4. Si vas a controlar secciones, pon data-section en cada boton y crea secciones con esos mismos nombres.
5. Importa GlassNavbar en script.js y crea new GlassNavbar().
6. Prueba scroll: la barra debe quedarse fija y el blur debe verse sobre el fondo.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
La navbar tapa el contenido	Falta padding-top en body	Usa body { padding-top: var(--navbar-height); }
No se ve el blur	El fondo detras es plano o el navegador no soporta backdrop-filter	Usa un fondo con degradados y agrega -webkit-backdrop-filter.
Los botones no se activan	El JS no encuentra #mainNavbar	Revisa que el id del HTML coincida con el constructor.

Checklist de prueba

- La navbar queda fija al hacer scroll.
- Los botones cambian a estado activo al hacer clic.
- En celular se ocultan los links si asi lo definiste.
- El contenido no queda debajo de la barra superior.

Adaptaciones profesionales recomendadas

- Agrega aria-label al nav para mejorar accesibilidad.
- Usa una clase .is-scrolled para oscurecer la navbar cuando el usuario baja.
- Conecta el boton Menu con el Sidebar ocultable de la funcionalidad 2.

2. Sidebar ocultable

El sidebar ocultable es un menu lateral que entra y sale de la pantalla cuando el usuario lo pide. Es muy comun en dashboards, paneles administrativos, aplicaciones tipo CRM, proyectos con muchas secciones y paginas responsive.

Donde se usa en paginas reales: dashboards, aplicaciones internas, ecommerce con filtros, sistemas de aprendizaje, paneles con muchas opciones.

Dependencias internas: puede abrirse desde un boton de la navbar; se complementa con overlay y submenus laterales.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*.	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<button type="button" id="openSidebar" class="btn-menu">Abrir menu</button>

<aside class="sidebar" id="sidebar" aria-hidden="true">
  <div class="sidebar__header">
    <h2>Panel</h2>
    <button type="button" id="closeSidebar">Cerrar</button>
  </div>

  <nav class="sidebar__nav">
    <button type="button" data-section="inicio">Inicio</button>
    <button type="button" data-section="perfil">Perfil</button>
    <button type="button" data-section="configuracion">Configuracion</button>
  </nav>
</aside>

<div class="sidebar-overlay" id="sidebarOverlay"></div>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

body.sidebar-open {
  overflow: hidden;
}

.sidebar {
  position: fixed;
  top: 0;
  left: 0;
  z-index: 1200;
  width: min(320px, 86vw);
  height: 100vh;
  padding: 20px;
  color: #ffffff;
  background: rgba(15, 23, 42, 0.88);
  border-right: 1px solid rgba(255, 255, 255, 0.18);
  backdrop-filter: blur(20px);
  -webkit-backdrop-filter: blur(20px);
  transform: translateX(-105%);
  transition: transform 0.25s ease;
}

.sidebar.is-open {
  transform: translateX(0);
}

.sidebar__header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  margin-bottom: 20px;
}

.sidebar__nav {
  display: grid;
  gap: 8px;
}

.sidebar__nav button,
.sidebar__header button,
.btn-menu {
  border: 0;
  border-radius: 12px;
  padding: 11px 14px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.12);
  cursor: pointer;
}

.sidebar-overlay {
  position: fixed;
  inset: 0;
  z-index: 1100;
  background: rgba(0, 0, 0, 0.45);
  opacity: 0;
  pointer-events: none;
  transition: opacity 0.25s ease;
}

.sidebar-overlay.is-visible {
  opacity: 1;
  pointer-events: auto;
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada


```

export class SidebarToggle {
  constructor(config = {}) {
    this.sidebar = document.getElementById(config.sidebarId || "sidebar");
    this.openButton = document.getElementById(config.openButtonId || "openSidebar");
    this.closeButton = document.getElementById(config.closeButtonId ||
      "closeSidebar");
    this.overlay = document.getElementById(config.overlayId || "sidebarOverlay");
    this.init();
  }

  init() {
    this.openButton?.addEventListener("click", () => this.open());
    this.closeButton?.addEventListener("click", () => this.close());
    this.overlay?.addEventListener("click", () => this.close());
    document.addEventListener("keydown", (event) => {
      if (event.key === "Escape") this.close();
    });
  }

  open() {
    this.sidebar?.classList.add("is-open");
    this.overlay?.classList.add("is-visible");
    document.body.classList.add("sidebar-open");
    this.sidebar?.setAttribute("aria-hidden", "false");
  }

  close() {
    this.sidebar?.classList.remove("is-open");
    this.overlay?.classList.remove("is-visible");
    document.body.classList.remove("sidebar-open");
    this.sidebar?.setAttribute("aria-hidden", "true");
  }
}

```

script.js - como llamarlo

```

// script.js
import { SidebarToggle } from './funciones.js';

new SidebarToggle();

```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initSidebarToggle() {
  const sidebar = document.getElementById("sidebar");
  const openButton = document.getElementById("openSidebar");
  const closeButton = document.getElementById("closeSidebar");
  const overlay = document.getElementById("sidebarOverlay");

  function open() {
    sidebar.classList.add("is-open");
    overlay.classList.add("is-visible");
    document.body.classList.add("sidebar-open");
    sidebar.setAttribute("aria-hidden", "false");
  }

  function close() {
    sidebar.classList.remove("is-open");
    overlay.classList.remove("is-visible");
    document.body.classList.remove("sidebar-open");
    sidebar.setAttribute("aria-hidden", "true");
  }

  openButton?.addEventListener("click", open);
  closeButton?.addEventListener("click", close);
  overlay?.addEventListener("click", close);
  document.addEventListener("keydown", (event) => {
    if (event.key === "Escape") close();
  });
}

initSidebarToggle();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
#sidebar	HTML/JS	ID del panel lateral. Cambialo en HTML y en config.sidebarId.
#openSidebar	HTML/JS	Boton que abre. Puede estar en navbar o en cualquier parte.
#closeSidebar	HTML/JS	Boton interno que cierra el menu.
#sidebarOverlay	HTML/JS/CSS	Capa oscura para cerrar al hacer clic fuera.
width	CSS .sidebar	Modifica el ancho del panel, por ejemplo 280px o 360px.

Paso a paso para implementarlo en otro proyecto

1. Copia el aside, el boton de abrir y el overlay.
2. Copia CSS de .sidebar, .sidebar-overlay y body.sidebar-open.
3. Copia la clase SidebarToggle en funciones.js.
4. En script.js importa SidebarToggle y crea new SidebarToggle().
5. Prueba abrir, cerrar, cerrar con overlay y cerrar con tecla Escape.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
El sidebar aparece siempre visible	Falta transform translateX(-105%)	Verifica la clase .sidebar y que .is-open solo se agregue con JS.
No cierra con overlay	El overlay no existe o tiene otro id	Revisa #sidebarOverlay y el constructor.
La pagina sigue haciendo scroll	Falta body.sidebar-open	Agrega overflow: hidden cuando el sidebar esta abierto.

Checklist de prueba

- Abre desde el boton principal.
- Cierra con boton Cerrar.
- Cierra con clic en overlay.
- Cierra con Escape.
- En movil no se sale de la pantalla.

Adaptaciones profesionales recomendadas

- Agrega focus trap si el sidebar contiene formularios.
- Usa aria-expanded en el boton de abrir para accesibilidad avanzada.
- Guarda el estado abierto/cerrado si deseas recordar el layout de un dashboard.

3. Submenus laterales

Los submenus laterales permiten agrupar opciones dentro del sidebar. El usuario puede abrir una categoria y ver sus opciones internas. Es clave para organizar proyectos grandes sin saturar la pantalla.

Donde se usa en paginas reales: paneles administrativos, menus de configuracion, rutas de aprendizaje, categorias de tienda, menus de usuario.

Dependencias internas: normalmente va dentro del Sidebar ocultable, pero tambien sirve en menus verticales fijos.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<aside class="sidebar" id="sidebar">
  <div class="side-group">
    <button type="button" class="side-group__toggle">
      Productos
    </button>
    <div class="side-group__content">
      <button type="button" data-section="cervezas">Cervezas</button>
      <button type="button" data-section="combos">Combos</button>
      <button type="button" data-section="promos">Promociones</button>
    </div>
  </div>

  <div class="side-group">
    <button type="button" class="side-group__toggle">
      Administracion
    </button>
    <div class="side-group__content">
      <button type="button" data-section="usuarios">Usuarios</button>
      <button type="button" data-section="reportes">Reportes</button>
    </div>
  </div>
</aside>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

.side-group {
  display: grid;
  gap: 6px;
  margin-bottom: 10px;
}

.side-group__toggle {
  width: 100%;
  display: flex;
  align-items: center;
  justify-content: space-between;
  border: 0;
  border-radius: 12px;
  padding: 12px 14px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.14);
  cursor: pointer;
}

.side-group__toggle span {
  transition: transform 0.2s ease;
}

.side-group.is-open .side-group__toggle span {
  transform: rotate(45deg);
}

.side-group__content {
  display: grid;
  gap: 6px;
  max-height: 0;
  overflow: hidden;
  transition: max-height 0.25s ease;
}

.side-group.is-open .side-group__content {
  max-height: 260px;
}

.side-group__content button {
  margin-left: 12px;
  border: 0;
  border-radius: 10px;
  padding: 10px 12px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.08);
  text-align: left;
  cursor: pointer;
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```
export class SidebarSubmenus {
  constructor(groupSelector = ".side-group") {
    this.groups = document.querySelectorAll(groupSelector);
    this.init();
  }

  init() {
    this.groups.forEach((group) => {
      const toggle = group.querySelector(".side-group__toggle");
      toggle?.addEventListener("click", () => {
        group.classList.toggle("is-open");
      });
    });
  }
}
```

script.js - como llamarlo

```
// script.js
import { SidebarSubmenus } from './funciones.js';

new SidebarSubmenus();
```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initSidebarSubmenus(groupSelector = ".side-group") {
  const groups = document.querySelectorAll(groupSelector);

  groups.forEach((group) => {
    const toggle = group.querySelector(".side-group__toggle");
    toggle?.addEventListener("click", () => {
      group.classList.toggle("is-open");
    });
  });
}

initSidebarSubmenus();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
.side-group	HTML/CSS/JS	Cada grupo nuevo debe usar esta clase para que JS lo reconozca.
.side-group__toggle	HTML/JS	Boton que abre y cierra el grupo.
.side-group__content	HTML/CSS	Contenedor de opciones internas.
data-section	Botones internos	Conecta cada opcion con una seccion del Router.
max-height	CSS	Aumentalo si el submenu tiene muchas opciones.

Paso a paso para implementarlo en otro proyecto

1. Inserta uno o varios .side-group dentro del sidebar.
2. Copia las reglas CSS de side-group y sus hijos.

3. Copia SidebarSubmenus a funciones.js.
4. Importa y ejecuta new SidebarSubmenus() en script.js.
5. Duplica un grupo completo para crear nuevas categorias.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
El submenu no abre	Falta clase .side-group__toggle	El boton que abre debe tener exactamente esa clase o debes cambiar el selector.
Se cortan opciones	max-height muy bajo	Sube max-height a 400px o usa calculo dinamico con scrollHeight.
Abren todos a la vez	Se esta usando una clase global mal aplicada	Asegurate de aplicar toggle solo al group actual.

Checklist de prueba

- Cada grupo abre y cierra de forma independiente.
- Las opciones internas se ven alineadas y con margen.
- La animacion no corta textos.
- Los data-section internos funcionan si estan conectados al router.

Adaptaciones profesionales recomendadas

- Agrega modo acordeon: cerrar los otros grupos cuando uno se abre.
- Agrega iconos SVG en lugar del signo +.
- Usa aria-expanded para indicar si el grupo esta abierto.

4. Router de secciones

El router de secciones permite cambiar lo que se muestra en el body sin recargar la pagina. Es una base sencilla para crear una SPA pequeña: al hacer clic en un boton, se ocultan las demas secciones y se muestra solo la solicitada.

Donde se usa en paginas reales: portafolios, demos de componentes, dashboards simples, proyectos academicos, landing pages interactivas.

Dependencias internas: funciona con cualquier boton que tenga data-section y con secciones que tengan data-view-section e id coincidente.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*.	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<nav class="section-menu">
  <button type="button" data-section="inicio">Inicio</button>
  <button type="button" data-section="servicios">Servicios</button>
  <button type="button" data-section="contacto">Contacto</button>
</nav>

<main>
  <section id="inicio" data-view-section class="view-section is-visible">
    <h2>Inicio</h2>
    <p>Contenido de inicio.</p>
  </section>

  <section id="servicios" data-view-section class="view-section">
    <h2>Servicios</h2>
    <p>Contenido de servicios.</p>
  </section>

  <section id="contacto" data-view-section class="view-section">
    <h2>Contacto</h2>
    <p>Contenido de contacto.</p>
  </section>
</main>
```

Codigo CSS minimo

CSS que debes llevar a styles.css


```

.view-section {
  display: none;
  animation: fadeIn 0.25s ease;
}

.view-section.is-visible {
  display: block;
}

[data-section].is-active {
  background: #2563eb;
  color: #ffffff;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(8px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```

export class SectionRouter {
  constructor(config = {}) {
    this.buttonSelector = config.buttonSelector || "[data-section]";
    this.sectionSelector = config.sectionSelector || "[data-view-section]";
    this.buttons = document.querySelectorAll(this.buttonSelector);
    this.sections = document.querySelectorAll(this.sectionSelector);
    this.defaultSection = config.defaultSection || "inicio";
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => {
        this.show(button.dataset.section);
      });
    });

    this.show(this.defaultSection);
  }

  show(sectionId) {
    this.sections.forEach((section) => {
      section.classList.toggle("is-visible", section.id === sectionId);
    });

    this.buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.section === sectionId);
    });
  }
}

```

script.js - como llamarlo

```

// script.js
import { SectionRouter } from './funciones.js';

new SectionRouter();

```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initSectionRouter(defaultSection = "inicio") {
  const buttons = document.querySelectorAll("[data-section]");
  const sections = document.querySelectorAll("[data-view-section]");

  function show(sectionId) {
    sections.forEach((section) => {
      section.classList.toggle("is-visible", section.id === sectionId);
    });

    buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.section === sectionId);
    });
  }

  buttons.forEach((button) => {
    button.addEventListener("click", () => show(button.dataset.section));
  });

  show(defaultSection);
}

initSectionRouter();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
data-section	Botones	Su valor debe ser igual al id de la seccion que quieres mostrar.
data-view-section	Secciones	Marca las secciones que el router puede controlar.
.is-visible	CSS/JS	Clase que muestra la seccion activa.
.is-active	CSS/JS	Clase visual para el boton activo.
defaultSection	JS	Seccion que aparece al cargar la pagina.

Paso a paso para implementarlo en otro proyecto

1. Crea un boton con data-section='nombre'.
2. Crea una seccion con id='nombre' y data-view-section.
3. Copia CSS de .view-section y .is-visible.
4. Copia SectionRouter a funciones.js.
5. En script.js ejecuta new SectionRouter({ defaultSection: 'inicio' }).
6. Prueba que cada boton muestre solo su seccion.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
El boton no muestra nada	data-section no coincide con el id	Iguala exactamente data-section='servicios' con id='servicios'.
Se ven todas las secciones	Falta CSS display none	Agrega .view-section { display: none; }.
La seccion inicial sale vacia	defaultSection no existe	Usa un id real como defaultSection.

Checklist de prueba

- Cada boton muestra una seccion distinta.
- Solo una seccion queda visible a la vez.
- El boton activo cambia de estilo.
- No hay IDs repetidos en HTML.

Adaptaciones profesionales recomendadas

- Agrega location.hash para que la URL cambie con cada seccion.
- Cierra el sidebar automaticamente al cambiar de seccion en movil.
- Carga contenido dinamico desde arrays si quieres evitar mucho HTML repetido.

5. Dropdown superior

El dropdown superior es un menu desplegable que aparece al hacer clic en un boton. Sirve para opciones de usuario, configuraciones rapidas, idioma, perfil, notificaciones o acciones secundarias.

Donde se usa en paginas reales: navbar de usuario, perfiles, acciones de tarjetas, menus de configuracion, dashboards y ecommerce.

Dependencias internas: puede ir dentro de la navbar fija glass o en cualquier contenedor. Usa una clase abierta para mostrar/ocultar.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<div class="dropdown" id="userDropdown">
  <button type="button" class="dropdown__button">
    Carlos
    <span>v</span>
  </button>

  <div class="dropdown__menu">
    <button type="button" data-section="perfil">Mi perfil</button>
    <button type="button" data-section="configuracion">Configuracion</button>
    <button type="button" id="logoutButton">Cerrar sesion</button>
  </div>
</div>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

.dropdown {
  position: relative;
  display: inline-block;
}

.dropdown__button {
  border: 0;
  border-radius: 999px;
  padding: 10px 14px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.14);
  cursor: pointer;
}

.dropdown__menu {
  position: absolute;
  top: calc(100% + 10px);
  right: 0;
  min-width: 190px;
  padding: 8px;
  background: rgba(15, 23, 42, 0.88);
  border: 1px solid rgba(255, 255, 255, 0.20);
  border-radius: 16px;
  backdrop-filter: blur(18px);
  -webkit-backdrop-filter: blur(18px);
  box-shadow: 0 18px 45px rgba(0, 0, 0, 0.28);
  opacity: 0;
  transform: translateY(-8px);
  pointer-events: none;
  transition: opacity 0.2s ease, transform 0.2s ease;
}

.dropdown.is-open .dropdown__menu {
  opacity: 1;
  transform: translateY(0);
  pointer-events: auto;
}

.dropdown__menu button {
  width: 100%;
  border: 0;
  border-radius: 10px;
  padding: 10px 12px;
  color: #ffffff;
  background: transparent;
  text-align: left;
  cursor: pointer;
}

.dropdown__menu button:hover {
  background: rgba(255, 255, 255, 0.12);
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```
export class TopDropdown {
  constructor(dropdownId = "userDropdown") {
    this.dropdown = document.getElementById(dropdownId);
    this.button = this.dropdown?.querySelector(".dropdown__button");
    this.init();
  }

  init() {
    this.button?.addEventListener("click", (event) => {
      event.stopPropagation();
      this.dropdown.classList.toggle("is-open");
    });

    document.addEventListener("click", () => this.close());
    document.addEventListener("keydown", (event) => {
      if (event.key === "Escape") this.close();
    });
  }

  close() {
    this.dropdown?.classList.remove("is-open");
  }
}
```

script.js - como llamarlo

```
// script.js
import { TopDropdown } from './funciones.js';

new TopDropdown();
```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initTopDropdown(dropdownId = "userDropdown") {
  const dropdown = document.getElementById(dropdownId);
  const button = dropdown?.querySelector(".dropdown__button");

  button?.addEventListener("click", (event) => {
    event.stopPropagation();
    dropdown.classList.toggle("is-open");
  });

  document.addEventListener("click", () => {
    dropdown?.classList.remove("is-open");
  });

  document.addEventListener("keydown", (event) => {
    if (event.key === "Escape") dropdown?.classList.remove("is-open");
  });
}

initTopDropdown();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
#userDropdown	HTML/JS	ID del dropdown. Si cambias uno, cambia el otro.
.dropdown__button	HTML/JS/CSS	Boton que abre/cierra el menu.

Elemento	Donde esta	Como adaptarlo
.dropdown__menu	HTML/CSS	Contenedor de opciones.
right: 0	CSS	Cambia a left: 0 si quieres que abra hacia la izquierda visual.
data-section	Opciones	Puedes usarlo para abrir secciones del router.

Paso a paso para implementarlo en otro proyecto

1. Copia el contenedor .dropdown en la navbar o en una card.
2. Copia el CSS del dropdown.
3. Copia TopDropdown en funciones.js.
4. Importa y ejecuta new TopDropdown('userDropdown').
5. Agrega o elimina botones dentro de .dropdown__menu segun tus opciones.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
Se abre y se cierra al instante	No se uso event.stopPropagation	Mantén event.stopPropagation en el click del boton.
Queda debajo de otros elementos	z-index bajo o padre con overflow hidden	Sube z-index o evita overflow hidden en el contenedor padre.
No cierra al hacer clic fuera	Falta listener en document	Agrega document.addEventListener('click', close).

Checklist de prueba

- Abre al hacer clic en el boton.
- Cierra al hacer clic fuera.
- Cierra con Escape.
- Las opciones internas son clicables.

Adaptaciones profesionales recomendadas

- Convierte cada opcion en enlace <a> si dirige a otra pagina.
- Agrega iconos y separadores para opciones importantes.
- Cierra el dropdown despues de hacer clic en cualquier opcion interna.

6. Tema claro/oscuro

El cambio de tema permite alternar entre apariencia oscura y clara. En proyectos profesionales se hace con variables CSS para no reescribir todo el diseño. El tema elegido puede guardarse en localStorage para que el usuario conserve su preferencia.

Donde se usa en paginas reales: dashboards, blogs, portafolios, aplicaciones web, ecommerce y paneles de configuracion.

Dependencias internas: usa variables CSS, un atributo data-theme en <html> o <body>, un boton y localStorage.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-.*.	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<button type="button" id="themeToggle" class="theme-toggle">
  Cambiar tema
</button>
```

Codigo CSS minimo

CSS que debes llevar a styles.css


```

:root {
  --page-bg: #0f172a;
  --surface: rgba(255, 255, 255, 0.12);
  --text: #ffffff;
  --muted: rgba(255, 255, 255, 0.72);
  --primary: #2563eb;
}

[data-theme="light"] {
  --page-bg: #f8fafc;
  --surface: rgba(15, 23, 42, 0.06);
  --text: #0f172a;
  --muted: #475569;
  --primary: #1d4ed8;
}

body {
  background: var(--page-bg);
  color: var(--text);
  transition: background 0.25s ease, color 0.25s ease;
}

.card,
.glass-navbar,
.sidebar {
  background: var(--surface);
}

.theme-toggle {
  border: 0;
  border-radius: 999px;
  padding: 10px 14px;
  color: #ffffff;
  background: var(--primary);
  cursor: pointer;
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```

export class ThemeToggle {
  constructor(config = {}) {
    this.button = document.getElementById(config.buttonId || "themeToggle");
    this.storageKey = config.storageKey || "kit-theme";
    this.root = document.documentElement;
    this.init();
  }

  init() {
    const savedTheme = localStorage.getItem(this.storageKey) || "dark";
    this.apply(savedTheme);

    this.button?.addEventListener("click", () => {
      const current = this.root.dataset.theme === "light" ? "light" : "dark";
      const next = current === "light" ? "dark" : "light";
      this.apply(next);
    });
  }

  apply(theme) {
    this.root.dataset.theme = theme;
    localStorage.setItem(this.storageKey, theme);
    if (this.button) {
      this.button.textContent = theme === "light" ? "Tema oscuro" : "Tema claro";
    }
  }
}

```

script.js - como llamarlo

```
// script.js
import { ThemeToggle } from './funciones.js';

new ThemeToggle();
```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initThemeToggle() {
  const button = document.getElementById("themeToggle");
  const storageKey = "kit-theme";
  const root = document.documentElement;

  function apply(theme) {
    root.dataset.theme = theme;
    localStorage.setItem(storageKey, theme);
    if (button) {
      button.textContent = theme === "light" ? "Tema oscuro" : "Tema claro";
    }
  }

  const savedTheme = localStorage.getItem(storageKey) || "dark";
  apply(savedTheme);

  button?.addEventListener("click", () => {
    const current = root.dataset.theme === "light" ? "light" : "dark";
    apply(current === "light" ? "dark" : "light");
  });
}

initThemeToggle();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
#themeToggle	HTML/JS	Boton que cambia tema. Cambia el id si actualizas el constructor.
--page-bg	CSS	Color o fondo principal de la pagina.
--surface	CSS	Color de tarjetas/nav/sidebar.
storageKey	JS	Clave de localStorage. Cambiala si mezclas varios proyectos.
data-theme	HTML/JS/CSS	Atributo que activa las variables del tema claro u oscuro.

Paso a paso para implementarlo en otro proyecto

1. Define tus colores base en :root.
2. Define los colores alternativos en [data-theme='light'] o [data-theme='dark'].
3. Usa variables en tus componentes: color: var(--text), background: var(--surface).
4. Copia ThemeToggle a funciones.js.

5. Importa y ejecuta `new ThemeToggle()`.
6. Recarga la pagina y verifica que el tema elegido se conserve.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
Al cambiar tema no cambia nada	Los componentes no usan variables CSS	Reemplaza colores fijos por <code>var(--nombre)</code> .
No recuerda el tema	<code>localStorage</code> bloqueado o clave distinta	Verifica <code>storageKey</code> y el navegador.
Texto ilegible	Mismo color para fondo y texto	Ajusta <code>--text</code> y <code>--surface</code> en ambos temas.

Checklist de prueba

- El boton alterna entre tema claro y oscuro.
- Al recargar se mantiene la preferencia.
- Texto, cards y botones siguen siendo legibles.
- No quedan colores fijos que rompan el tema.

Adaptaciones profesionales recomendadas

- Detecta preferencia del sistema con `window.matchMedia('(prefers-color-scheme: dark)')`.
- Agrega transicion suave a fondo, color y bordes.
- Usa nombres de variables semanticas: `--surface`, `--text`, `--border`, `--primary`.

7. Scroll progress

La barra de progreso de scroll muestra cuanto ha avanzado el usuario dentro de la pagina. Es muy util en blogs largos, documentacion, landing pages y guias extensas.

Donde se usa en paginas reales: articulos, documentacion, cursos, tutoriales, landing pages largas y paginas de lectura.

Dependencias internas: solo necesita un div fijo y escuchar el evento scroll.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-.*.	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<div class="scroll-progress" id="scrollProgress"></div>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```
.scroll-progress {  
  position: fixed;  
  top: 0;  
  left: 0;  
  z-index: 2000;  
  width: 0%;  
  height: 4px;  
  background: linear-gradient(90deg, #2563eb, #7c3aed);  
  box-shadow: 0 0 16px rgba(37, 99, 235, 0.55);  
  transition: width 0.08s linear;  
}
```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```
export class ScrollProgress {
  constructor(progressId = "scrollProgress") {
    this.bar = document.getElementById(progressId);
    this.init();
  }

  init() {
    window.addEventListener("scroll", () => this.update(), { passive: true });
    this.update();
  }

  update() {
    const scrollTop = window.scrollY;
    const documentHeight = document.documentElement.scrollHeight - window.innerHeight;
    const progress = documentHeight > 0 ? (scrollTop / documentHeight) * 100 : 0;

    if (this.bar) {
      this.bar.style.width = `${progress}%`;
    }
  }
}
```

script.js - como llamarlo

```
// script.js
import { ScrollProgress } from './funciones.js';

new ScrollProgress();
```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initScrollProgress(progressId = "scrollProgress") {
  const bar = document.getElementById(progressId);

  function update() {
    const scrollTop = window.scrollY;
    const documentHeight = document.documentElement.scrollHeight - window.innerHeight;
    const progress = documentHeight > 0 ? (scrollTop / documentHeight) * 100 : 0;
    if (bar) bar.style.width = `${progress}%`;
  }

  window.addEventListener("scroll", update, { passive: true });
  update();
}

initScrollProgress();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
#scrollProgress	HTML/JS	ID de la barra. Cambialo en ambos sitios si lo renombas.
height	CSS	Grosor de la barra. Puedes usar 3px, 4px o 6px.
background	CSS	Color o degradado de la barra.
z-index	CSS	Debe estar por encima de navbar si quieres que se vea siempre.

Elemento	Donde esta	Como adaptarlo
passive: true	JS	Mejora rendimiento en scroll; dejalo si solo lees scroll.

Paso a paso para implementarlo en otro proyecto

1. Coloca el div #scrollProgress al inicio del body.
2. Copia la regla CSS .scroll-progress.
3. Copia ScrollProgress a funciones.js.
4. Ejecuta new ScrollProgress().
5. Haz una pagina suficientemente larga y prueba el avance al hacer scroll.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
La barra no se mueve	La pagina no tiene altura suficiente	Agrega contenido o prueba en pagina larga.
No se ve	z-index bajo o mismo color que fondo	Sube z-index y cambia background.
Llega a mas de 100%	Calculo de altura incorrecto	Usa scrollHeight - innerHeight como en el ejemplo.

Checklist de prueba

- Al inicio la barra esta en 0%.
- Al final de la pagina llega a 100%.
- No tapa botones importantes.
- Funciona con navbar fija.

Adaptaciones profesionales recomendadas

- Oculta la barra si la pagina no tiene scroll.
- Cambia el color de la barra segun seccion activa.
- Combinala con lectura de articulos largos o documentacion.

8. Back to top

El boton Back to top permite volver rapidamente al inicio de la pagina. Mejora la experiencia en paginas largas y se suele mostrar solo cuando el usuario ha bajado cierta distancia.

Donde se usa en paginas reales: blogs, documentacion, portfolios largos, tiendas, landing pages extensas y guias tecnicas.

Dependencias internas: necesita un boton fijo y detectar el scroll para mostrarlo u ocultarlo.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<button type="button" class="back-to-top" id="backToTop" aria-label="Volver arriba">
  ↑
</button>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

.back-to-top {
  position: fixed;
  right: 22px;
  bottom: 22px;
  z-index: 1300;
  width: 48px;
  height: 48px;
  border: 0;
  border-radius: 50%;
  color: #ffffff;
  background: linear-gradient(135deg, #2563eb, #7c3aed);
  box-shadow: 0 14px 35px rgba(37, 99, 235, 0.35);
  cursor: pointer;
  opacity: 0;
  transform: translateY(12px);
  pointer-events: none;
  transition: opacity 0.2s ease, transform 0.2s ease;
}

.back-to-top.is-visible {
  opacity: 1;
  transform: translateY(0);
  pointer-events: auto;
}

.back-to-top:hover {
  transform: translateY(-3px);
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```

export class BackToTop {
  constructor(config = {}) {
    this.button = document.getElementById(config.buttonId || "backToTop");
    this.visibleAt = config.visibleAt || 360;
    this.init();
  }

  init() {
    window.addEventListener("scroll", () => this.toggle(), { passive: true });
    this.button?.addEventListener("click", () => {
      window.scrollTo({ top: 0, behavior: "smooth" });
    });
    this.toggle();
  }

  toggle() {
    const shouldShow = window.scrollY > this.visibleAt;
    this.button?.classList.toggle("is-visible", shouldShow);
  }
}

```

script.js - como llamarlo

```

// script.js
import { BackToTop } from './funciones.js';

new BackToTop();

```

JavaScript normal sin modulos

script-normal.js - version directa


```
function initBackToTop(buttonId = "backToTop", visibleAt = 360) {
  const button = document.getElementById(buttonId);

  function toggle() {
    const shouldShow = window.scrollY > visibleAt;
    button?.classList.toggle("is-visible", shouldShow);
  }

  window.addEventListener("scroll", toggle, { passive: true });
  button?.addEventListener("click", () => {
    window.scrollTo({ top: 0, behavior: "smooth" });
  });

  toggle();
}

initBackToTop();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
#backToTop	HTML/JS	ID del boton. Cambialo junto con el constructor.
visibleAt	JS	Cantidad de pixeles bajados antes de mostrar el boton.
right/bottom	CSS	Ubicacion del boton en pantalla.
behavior: smooth	JS	Hace que el regreso sea suave. Puedes quitarlo si quieres inmediato.
↑	HTML	Puedes reemplazarlo por un icono SVG.

Paso a paso para implementarlo en otro proyecto

1. Coloca el boton al final del body.
2. Copia el CSS .back-to-top.
3. Copia BackToTop a funciones.js.
4. Importa y ejecuta new BackToTop({ visibleAt: 360 }).
5. Baja la pagina y confirma que el boton aparece.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
El boton nunca aparece	Pagina muy corta o visibleAt muy alto	Baja visibleAt a 100 para probar.
No sube al inicio	El listener no carga o id incorrecto	Revisa #backToTop y que script este al final.
Tapa contenido movil	bottom/right mal ubicados	Ajusta posicion con media queries.

Checklist de prueba

- No se ve al cargar arriba.
- Aparece al hacer scroll hacia abajo.

- Al hacer clic vuelve al inicio suavemente.
- No se cruza con botones de chat o footer.

Adaptaciones profesionales recomendadas

- Ocultalo cuando el usuario este en el footer si tapa contenido.
- Agrega aria-label descriptivo para lectores de pantalla.
- Usa CSS prefers-reduced-motion para usuarios que prefieren menos animaciones.

9. Reveal on scroll

Reveal on scroll muestra elementos con animacion cuando aparecen en pantalla. Es muy usado para que cards, secciones, imagenes y textos entren suavemente mientras el usuario navega.

Donde se usa en paginas reales: landing pages, portafolios, secciones de servicios, tarjetas de productos, galerias y storytelling.

Dependencias internas: usa la API IntersectionObserver y una clase .reveal en los elementos que quieres animar.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<section class="card reveal">
  <h2>Servicio destacado</h2>
  <p>Este bloque aparecera suavemente cuando entre en pantalla.</p>
</section>

<div class="features">
  <article class="feature-card reveal">Card 1</article>
  <article class="feature-card reveal">Card 2</article>
  <article class="feature-card reveal">Card 3</article>
</div>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

.reveal {
  opacity: 0;
  transform: translateY(22px);
  transition: opacity 0.55s ease, transform 0.55s ease;
}

.reveal.is-visible {
  opacity: 1;
  transform: translateY(0);
}

@media (prefers-reduced-motion: reduce) {
  .reveal {
    opacity: 1;
    transform: none;
    transition: none;
  }
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```

export class RevealOnScroll {
  constructor(selector = ".reveal") {
    this.elements = document.querySelectorAll(selector);
    this.init();
  }

  init() {
    if (!('IntersectionObserver' in window)) {
      this.elements.forEach((el) => el.classList.add("is-visible"));
      return;
    }

    const observer = new IntersectionObserver((entries) => {
      entries.forEach((entry) => {
        if (entry.isIntersecting) {
          entry.target.classList.add("is-visible");
          observer.unobserve(entry.target);
        }
      });
    }, { threshold: 0.18 });

    this.elements.forEach((element) => observer.observe(element));
  }
}

```

script.js - como llamarlo

```

// script.js
import { RevealOnScroll } from './funciones.js';

new RevealOnScroll();

```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initRevealOnScroll(selector = ".reveal") {
  const elements = document.querySelectorAll(selector);

  if (!('IntersectionObserver' in window)) {
    elements.forEach((el) => el.classList.add("is-visible"));
    return;
  }

  const observer = new IntersectionObserver((entries) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) {
        entry.target.classList.add("is-visible");
        observer.unobserve(entry.target);
      }
    });
  }, { threshold: 0.18 });

  elements.forEach((element) => observer.observe(element));
}

initRevealOnScroll();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
.reveal	HTML/CSS/JS	Agrega esta clase a cualquier elemento que quieras animar.
.is-visible	CSS/JS	Clase que se agrega cuando el elemento entra en pantalla.
translateY(22px)	CSS	Distancia desde donde entra. Puedes usar 12px o 40px.
threshold	JS	Porcentaje visible necesario para activar la animacion.
unobserve	JS	Evita repetir la animacion. Quitalo si quieres que se repita.

Paso a paso para implementarlo en otro proyecto

1. Agrega class='reveal' a las cards o secciones.
2. Copia CSS de .reveal y .reveal.is-visible.
3. Copia RevealOnScroll en funciones.js.
4. Ejecuta new RevealOnScroll().
5. Haz scroll y verifica que los elementos aparecen al entrar en pantalla.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
Los elementos nunca aparecen	JS no cargo o selector incorrecto	Revisa que tengan clase .reveal y que se ejecute el script.
Aparecen de una vez	Ya tienen .is-visible en HTML	Deja .is-visible solo para elementos que quieras visibles desde el inicio.
Animacion molesta	Transicion exagerada	Reduce translateY y duracion.

Checklist de prueba

- Elementos ocultos aparecen al hacer scroll.
- No quedan textos invisibles permanentemente.
- Funciona en varias secciones.
- Respeta prefers-reduced-motion.

Adaptaciones profesionales recomendadas

- Agrega variantes: .reveal-left, .reveal-right, .reveal-scale.
- Usa delay con style='--delay: 0.2s' para animaciones escalonadas.
- No abuses: anima solo elementos importantes para no saturar la experiencia.

10. Contador animado

El contador animado cuenta desde cero hasta un valor objetivo cuando el elemento aparece en pantalla. Se usa mucho para indicadores, metricas, logros, estadísticas, KPIs y secciones de impacto.

Donde se usa en paginas reales: landing pages, dashboards, secciones de logros, estadísticas de marca, paneles de datos y sitios institucionales.

Dependencias internas: usa atributos data-counter y data-target; puede apoyarse en IntersectionObserver para iniciar solo cuando se vea.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*.	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<section class="stats">
  <article class="stat-card reveal">
    <strong data-counter data-target="1200">0</strong>
    <span>Usuarios registrados</span>
  </article>

  <article class="stat-card reveal">
    <strong data-counter data-target="98">0</strong>
    <span>Porcentaje de satisfaccion</span>
  </article>

  <article class="stat-card reveal">
    <strong data-counter data-target="50">0</strong>
    <span>Componentes reutilizables</span>
  </article>
</section>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```
.stats {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 16px;
}

.stat-card {
  padding: 20px;
  border-radius: 18px;
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid rgba(255, 255, 255, 0.18);
  text-align: center;
}

.stat-card strong {
  display: block;
  font-size: clamp(32px, 6vw, 56px);
  line-height: 1;
  color: #ffffff;
}

.stat-card span {
  display: block;
  margin-top: 8px;
  color: rgba(255, 255, 255, 0.72);
}
```

JavaScript modular con export/import

funciones.js - clase o funcion exportada


```

export class AnimatedCounters {
  constructor(selector = "[data-counter]") {
    this.counters = document.querySelectorAll(selector);
    this.init();
  }

  init() {
    if (!('IntersectionObserver' in window)) {
      this.counters.forEach((counter) => this.animate(counter));
      return;
    }

    const observer = new IntersectionObserver((entries) => {
      entries.forEach((entry) => {
        if (entry.isIntersecting) {
          this.animate(entry.target);
          observer.unobserve(entry.target);
        }
      });
    }, { threshold: 0.35 });

    this.counters.forEach((counter) => observer.observe(counter));
  }

  animate(counter) {
    const target = Number(counter.dataset.target || 0);
    const duration = Number(counter.dataset.duration || 1200);
    const startTime = performance.now();

    const update = (currentTime) => {
      const elapsed = currentTime - startTime;
      const progress = Math.min(elapsed / duration, 1);
      const value = Math.floor(progress * target);
      counter.textContent = value.toLocaleString("es-CO");

      if (progress < 1) {
        requestAnimationFrame(update);
      } else {
        counter.textContent = target.toLocaleString("es-CO");
      }
    };

    requestAnimationFrame(update);
  }
}

```

script.js - como llamarlo

```

// script.js
import { AnimatedCounters } from './funciones.js';

new AnimatedCounters();

```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initAnimatedCounters(selector = "[data-counter]") {
  const counters = document.querySelectorAll(selector);

  function animate(counter) {
    const target = Number(counter.dataset.target || 0);
    const duration = Number(counter.dataset.duration || 1200);
    const startTime = performance.now();

    function update(currentTime) {
      const elapsed = currentTime - startTime;
      const progress = Math.min(elapsed / duration, 1);
      const value = Math.floor(progress * target);
      counter.textContent = value.toLocaleString("es-CO");

      if (progress < 1) {
        requestAnimationFrame(update);
      } else {
        counter.textContent = target.toLocaleString("es-CO");
      }
    }

    requestAnimationFrame(update);
  }

  if (!('IntersectionObserver' in window)) {
    counters.forEach(animate);
    return;
  }

  const observer = new IntersectionObserver((entries) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) {
        animate(entry.target);
        observer.unobserve(entry.target);
      }
    });
  }, { threshold: 0.35 });

  counters.forEach((counter) => observer.observe(counter));
}

initAnimatedCounters();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
data-counter	HTML/JS	Marca los numeros que deben animarse.
data-target	HTML/JS	Numero final al que debe llegar el contador.
data-duration	HTML/JS	Duracion opcional en milisegundos; por defecto 1200.
toLocaleString('es-CO')	JS	Formato de miles. Cambia locale si necesitas otro pais.
threshold	JS	Define que tanto debe verse el contador para iniciar.

Paso a paso para implementarlo en otro proyecto

1. Crea una tarjeta o bloque con un elemento numerico.
2. Agrega data-counter y data-target='numero'.
3. Copia el CSS de .stats y .stat-card o adaptalo a tus cards existentes.
4. Copia AnimatedCounters a funciones.js.

5. Ejecuta `new AnimatedCounters()`.
6. Haz scroll hasta la seccion y verifica que cuenta hasta el valor final.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
Muestra NaN	data-target no es numerico	Usa data-target='1200' sin puntos ni comas.
Cuenta demasiado rapido	duration bajo	Agrega data-duration='2200'.
Se repite muchas veces	No se desobserva el elemento	Mantén <code>observer.unobserve(entry.target)</code> .

Checklist de prueba

- Los numeros empiezan en 0.
- Llegan al valor exacto de data-target.
- La animacion inicia al entrar en pantalla.
- No aparece NaN ni valores negativos no deseados.

Adaptaciones profesionales recomendadas

- Agrega sufijos visuales como %, + o K fuera del data-target para no romper el numero.
- Usa easing para que el contador desacelere al final.
- Conecta los valores a datos reales si luego consumes una API.

Resumen final para reciclar las funcionalidades 1 a 10

Antes de copiar código a un proyecto real, revisa esta lista. Te ayuda a evitar los errores más comunes cuando reutilizas componentes.

Pregunta	Respuesta esperada
¿El HTML tiene los IDs correctos?	Cada id usado por JS existe una sola vez en la página.
¿Las clases CSS coinciden?	Las clases del HTML son iguales a las reglas del CSS.
¿Los data-* coinciden?	data-section debe coincidir con el id de la sección.
¿Uso versión modular o normal?	Solo una versión por página para no duplicar eventos.
¿El componente tiene estado?	Clases como is-open, is-visible o is-active deben ser controladas por JS.
¿Se probó en móvil?	Navbar, sidebar y dropdown no deben salirse de pantalla.
¿Hay accesibilidad básica?	Botones reales, aria-label cuando haga falta y cierre con Escape.

Con estas 10 funcionalidades ya puedes construir la base de una interfaz profesional: navegación fija, menú lateral, rutas internas, tema claro/oscuro, indicadores de scroll y animaciones de entrada.

Actualizacion funcional del bloque 1 a 10

Este anexo actualiza los codigos de copiar y pegar para que queden alineados con la pagina actual y funcionen como referencia implementable.

- Usa estos bloques como fuente vigente para HTML, CSS y JavaScript.
- El contenido anterior del PDF base se conserva, pero este anexo manda para implementacion.

1. Navbar fija glass

Barra superior fija con efecto blur y transparencia para un diseño moderno.

HTML copiar y pegar

```
<header class="glass-navbar">
  <a href="#inicio" class="brand">
    <span class="brand__mark">CW</span>
    <span class="brand__text">Componentes Web</span>
  </a>

  <nav class="navbar-links" aria-label="Navegacion principal">
    <button class="nav-link is-active" type="button">Inicio</button>
    <button class="nav-link" type="button">Componentes</button>
    <button class="nav-link" type="button">Formularios</button>

    <div class="dropdown is-open">
      <button class="nav-link dropdown__button" type="button">Mas</button>
      <div class="dropdown__menu">
        <button type="button">Login demo</button>
        <button type="button">Storage</button>
        <button type="button">Guia</button>
      </div>
    </div>
  </nav>

  <div class="navbar-actions">
    <button class="icon-button icon-button--label" type="button">Dark</button>
    <button class="icon-button icon-button--label" type="button">Menu</button>
  </div>
</header>
```

CSS copiar y pegar

```
:root {
  --nav-primary: #4f46e5;
  --nav-secondary: #0ea5e9;
  --nav-bg: rgba(15, 23, 42, 0.72);
  --nav-border: rgba(255, 255, 255, 0.16);
  --nav-text: #e5eefc;
}

body {
  min-height: 100vh;
  margin: 0;
  padding: 32px;
  font-family: Arial, Helvetica, sans-serif;
  background:
    radial-gradient(circle at top left, rgba(14, 165, 233, 0.22), transparent
30%),
    radial-gradient(circle at bottom right, rgba(79, 70, 229, 0.28), transparent
36%),
    linear-gradient(135deg, #0f172a, #1e293b);
}

.glass-navbar {
  width: min(1120px, 100%);
  margin: 0 auto;
  display: flex;
```

```
    align-items: center;
    justify-content: space-between;
    gap: 16px;
    padding: 14px 18px;
    border: 1px solid var(--nav-border);
    border-radius: 999px;
    background: var(--nav-bg);
    color: var(--nav-text);
    backdrop-filter: blur(20px);
    box-shadow: 0 24px 50px rgba(15, 23, 42, 0.28);
}
```

```
.brand,
.navbar-links,
.navbar-actions {
    display: flex;
    align-items: center;
    gap: 12px;
}
```

```
.brand {
    color: inherit;
    text-decoration: none;
    font-weight: 800;
}
```

```
.brand__mark {
    width: 42px;
    height: 42px;
    display: grid;
    place-items: center;
    border-radius: 50%;
    background: linear-gradient(135deg, var(--nav-secondary), var(--nav-primary));
    color: #ffffff;
}
```

```
.nav-link,
.icon-button,
.dropdown__menu button {
    border: 1px solid rgba(255, 255, 255, 0.12);
    border-radius: 999px;
    background: rgba(255, 255, 255, 0.08);
    color: inherit;
    padding: 10px 14px;
    transition: transform 0.2s ease, background 0.2s ease, border-color 0.2s ease,
    box-shadow 0.2s ease;
    cursor: pointer;
}
```

```
.nav-link:hover,
.icon-button:hover,
.dropdown__menu button:hover {
    transform: translateY(-1px);
    background: rgba(255, 255, 255, 0.14);
}
```

```
.nav-link.is-active {
```

```

border-color: transparent;
background: linear-gradient(135deg, var(--nav-secondary), var(--nav-primary));
box-shadow: 0 12px 24px rgba(79, 70, 229, 0.28);
}

.icon-button--label {
  min-width: 72px;
}

.dropdown {
  position: relative;
}

.dropdown__menu {
  position: absolute;
  top: calc(100% + 10px);
  right: 0;
  min-width: 190px;
  display: grid;
  gap: 8px;
  padding: 12px;
  border-radius: 18px;
  border: 1px solid var(--nav-border);
  background: rgba(15, 23, 42, 0.92);
  box-shadow: 0 18px 34px rgba(15, 23, 42, 0.34);
}

@media (max-width: 860px) {
  .glass-navbar {
    border-radius: 28px;
    flex-wrap: wrap;
    justify-content: center;
  }

  .navbar-links {
    justify-content: center;
    flex-wrap: wrap;
  }

  .navbar-actions {
    width: 100%;
    justify-content: center;
  }
}

```

JS modular copiar y pegar

```

document.addEventListener("DOMContentLoaded", () => {
  const nav = document.querySelector(".glass-navbar");
  if (!nav) return;

  const links = nav.querySelectorAll(".nav-link:not(.dropdown__button)");
  links.forEach((link) => {
    link.addEventListener("click", () => {
      links.forEach((item) => item.classList.remove("is-active"));
      link.classList.add("is-active");
    });
  });
});

```


JS normal copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {  
  const nav = document.querySelector(".glass-navbar");  
  if (!nav) return;  
  
  const links = nav.querySelectorAll(".nav-link:not(.dropdown__button)");  
  links.forEach((link) => {  
    link.addEventListener("click", () => {  
      links.forEach((item) => item.classList.remove("is-active"));  
      link.classList.add("is-active");  
    });  
  });  
});
```

2. Sidebar ocultable

Menú lateral colapsable para pantallas pequeñas controlado por eventos de clic.

HTML copiar y pegar

```
<aside class="sidebar" id="sidebar">
  <div class="sidebar__header">
    <strong>Menu lateral</strong>
    <button id="closeSidebar" type="button">Cerrar</button>
  </div>
  <nav class="sidebar__content">
    <button type="button">Inicio</button>
    <button type="button">Servicios</button>
    <button type="button">Contacto</button>
  </nav>
</aside>
<div class="overlay" id="overlay"></div>
<button id="openSidebar" type="button">Abrir menu</button>
```

CSS copiar y pegar

```
.sidebar {
  position: fixed;
  top: 0;
  left: 0;
  width: 280px;
  height: 100vh;
  padding: 20px;
  background: #0f172a;
  color: #ffffff;
  transform: translateX(-105%);
  transition: transform 0.25s ease;
}
.sidebar.is-open { transform: translateX(0); }
.overlay {
  position: fixed;
  inset: 0;
  background: rgba(15, 23, 42, 0.45);
  opacity: 0;
  pointer-events: none;
  transition: opacity 0.25s ease;
}
.overlay.is-active {
  opacity: 1;
  pointer-events: auto;
}
.sidebar__content {
  display: grid;
  gap: 10px;
  margin-top: 16px;
}
.sidebar__content button,
#openSidebar,
#closeSidebar {
  padding: 10px 14px;
  border: 0;
  border-radius: 10px;
  cursor: pointer;
}
```

JS modular copiar y pegar

```
export class SidebarMenu {
  constructor() {
    this.sidebar = document.getElementById("sidebar");
    this.overlay = document.getElementById("overlay");
    this.openBtn = document.getElementById("openSidebar");
    this.closeBtn = document.getElementById("closeSidebar");
    this.init();
  }
  init() {
    this.openBtn.addEventListener("click", () => {
      this.sidebar.classList.add("is-open");
      this.overlay.classList.add("is-active");
    });
    this.closeBtn.addEventListener("click", () => this.close());
    this.overlay.addEventListener("click", () => this.close());
  }
  close() {
    this.sidebar.classList.remove("is-open");
    this.overlay.classList.remove("is-active");
  }
}
```

JS normal copiar y pegar

```
class SidebarMenu {
  // Mismo código que la versión modular sin la palabra "export"
}
```

3. Submenús laterales

Menús colapsables dentro del sidebar para agrupar enlaces por categorías.

HTML copiar y pegar

```
<div class="side-group">
  <button class="side-group__button">Interfaz &#x2604;/button>
  <div class="side-group__content">
    <button data-section="componentes">Componentes UI</button>
    <button data-section="formularios">Formularios</button>
  </div>
</div>
```

CSS copiar y pegar

```
.side-group__content {
  max-height: 0;
  overflow: hidden;
  transition: max-height var(--transition);
  padding-left: 12px;
}
.side-group.is-open .side-group__content {
  max-height: 220px;
}
```

JS modular copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {
  document.querySelectorAll(".side-group__button").forEach((button) => {
    button.addEventListener("click", () => {
      button.parentElement.classList.toggle("is-open");
    });
  });
});
```

JS normal copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {
  document.querySelectorAll(".side-group__button").forEach((button) => {
    button.addEventListener("click", () => {
      button.parentElement.classList.toggle("is-open");
    });
  });
});
```

4. Router de secciones

Controla el cambio de vistas activas en una SPA sencilla de forma dinámica.

HTML copiar y pegar

```
<nav class="router-nav">
  <button type="button" data-section="inicio" class="is-active">Inicio</button>
  <button type="button" data-section="servicios">Servicios</button>
  <button type="button" data-section="contacto">Contacto</button>
</nav>

<section id="inicio" data-view-section class="view-section is-active">Contenido
inicio</section>
<section id="servicios" data-view-section class="view-section">Contenido
servicios</section>
<section id="contacto" data-view-section class="view-section">Contenido
contacto</section>
```

CSS copiar y pegar

```
.router-nav {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  margin-bottom: 16px;
}

.router-nav button {
  padding: 10px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 999px;
  background: #ffffff;
  cursor: pointer;
}

.router-nav button.is-active {
  background: #2563eb;
  color: #ffffff;
  border-color: #2563eb;
}

.view-section { display: none; }
.view-section.is-active { display: block; }
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
export class RouterSections {
  constructor(toast) {
    this.sections = $$('[data-view-section]');
    this.buttons = $$('[data-section]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () =>
        this.showSection(button.dataset.section));
    });
  }
}
```

```

showSection(sectionId) {
  const target = document.getElementById(sectionId);
  if (!target) return;

  this.sections.forEach((section) => section.classList.remove("is-active"));
  target.classList.add("is-active");

  this.buttons.forEach((button) => {
    button.classList.toggle("is-active", button.dataset.section ===
sectionId);
  });

  window.scrollTo({ top: 0, behavior: "smooth" });
  this.toast?.show(`Seccion activa: ${sectionId}`);
}
}

document.addEventListener("DOMContentLoaded", () => {
  new RouterSections();
});

```

JS normal copiar y pegar

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class RouterSections {
  constructor(toast) {
    this.sections = $$('[data-view-section]');
    this.buttons = $$('[data-section]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () =>
this.showSection(button.dataset.section));
    });
  }

  showSection(sectionId) {
    const target = document.getElementById(sectionId);
    if (!target) return;

    this.sections.forEach((section) => section.classList.remove("is-active"));
    target.classList.add("is-active");

    this.buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.section ===
sectionId);
    });

    window.scrollTo({ top: 0, behavior: "smooth" });
    this.toast?.show(`Seccion activa: ${sectionId}`);
  }
}

```

```
document.addEventListener("DOMContentLoaded", () => {  
  new RouterSections();  
});
```

5. Dropdown superior

Menú desplegable en la barra de navegación para configuraciones o perfiles.

HTML copiar y pegar

```
<div class="dropdown" data-dropdown>
  <button type="button" data-dropdown-button>Mas opciones</button>
  <div class="dropdown__menu">
    <button type="button">Perfil</button>
    <button type="button">Configuracion</button>
    <button type="button">Salir</button>
  </div>
</div>
```

CSS copiar y pegar

```
.dropdown { position: relative; display: inline-block; }
.dropdown__menu {
  position: absolute;
  top: calc(100% + 8px);
  right: 0;
  min-width: 180px;
  display: grid;
  gap: 8px;
  padding: 10px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  background: #ffffff;
  opacity: 0;
  transform: translateY(-6px);
  pointer-events: none;
  transition: all 0.2s ease;
}
.dropdown.is-open .dropdown__menu {
  opacity: 1;
  transform: translateY(0);
  pointer-events: auto;
}
.dropdown button {
  padding: 10px 14px;
  border-radius: 10px;
  border: 0;
  cursor: pointer;
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class Dropdowns {
  constructor() {
    this.dropdowns = $$('.[data-dropdown]');
    this.init();
  }

  init() {
    this.dropdowns.forEach((dropdown) => {
      const button = $('[data-dropdown-button]', dropdown);
      button.addEventListener("click", (event) => {
```



```

        event.stopPropagation();
        dropdown.classList.toggle("is-open");
    });
});
document.addEventListener("click", () => this.dropdowns.forEach((d) =>
d.classList.remove("is-open")));
}
}

document.addEventListener("DOMContentLoaded", () => {
    new Dropdowns();
});

```

JS normal copiar y pegar

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class Dropdowns {
    constructor() {
        this.dropdowns = $$('#[data-dropdown]');
        this.init();
    }

    init() {
        this.dropdowns.forEach((dropdown) => {
            const button = $('[data-dropdown-button]', dropdown);
            button.addEventListener("click", (event) => {
                event.stopPropagation();
                dropdown.classList.toggle("is-open");
            });
        });
        document.addEventListener("click", () => this.dropdowns.forEach((d) =>
d.classList.remove("is-open")));
    }
}

document.addEventListener("DOMContentLoaded", () => {
    new Dropdowns();
});

```

6. Tema claro/oscuro

Alterna entre temas claro y oscuro guardando la preferencia en localStorage.

HTML copiar y pegar

```
<button id="themeToggle" type="button">Tema</button>
```

CSS copiar y pegar

```
:root {
  --bg: #0f172a;
  --text: #ffffff;
}
[data-theme="light"] {
  --bg: #f8fafc;
  --text: #0f172a;
}
body {
  background: var(--bg);
  color: var(--text);
  transition: background 0.2s ease, color 0.2s ease;
}
#themeToggle {
  padding: 10px 14px;
  border: 0;
  border-radius: 999px;
  cursor: pointer;
}
```

JS modular copiar y pegar

```
export class ThemeManager {
  constructor(buttonId = "themeToggle") {
    this.button = document.getElementById(buttonId);
    this.theme = localStorage.getItem("kit-theme") || "dark";
    this.init();
  }

  init() {
    this.applyTheme();
    this.button.addEventListener("click", () => {
      this.theme = this.theme === "dark" ? "light" : "dark";
      localStorage.setItem("kit-theme", this.theme);
      this.applyTheme();
    });
  }

  applyTheme() {
    document.documentElement.dataset.theme = this.theme;
    this.button.textContent = this.theme === "dark" ? "Dark" : "Light";
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new ThemeManager();
});
```

JS normal copiar y pegar

```
class ThemeManager {
  constructor(buttonId = "themeToggle") {
    this.button = document.getElementById(buttonId);
```

```
    this.theme = localStorage.getItem("kit-theme") || "dark";
    this.init();
  }

  init() {
    this.applyTheme();
    this.button.addEventListener("click", () => {
      this.theme = this.theme === "dark" ? "light" : "dark";
      localStorage.setItem("kit-theme", this.theme);
      this.applyTheme();
    });
  }

  applyTheme() {
    document.documentElement.dataset.theme = this.theme;
    this.button.textContent = this.theme === "dark" ? "Dark" : "Light";
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new ThemeManager();
});
```

7. Scroll progress

Línea superior que indica el porcentaje de scroll en la página actual.

HTML copiar y pegar

```
<div class="scroll-progress" id="scrollProgress"></div>
<button class="back-to-top" id="backToTop" type="button">Subir</button>
<div style="min-height: 200vh; padding-top: 24px;">Haz scroll para probar.</div>
```

CSS copiar y pegar

```
.scroll-progress {
  position: fixed;
  top: 0;
  left: 0;
  width: 0;
  height: 4px;
  background: linear-gradient(90deg, #2563eb, #7c3aed);
}
.back-to-top {
  position: fixed;
  right: 20px;
  bottom: 20px;
  opacity: 0;
  pointer-events: none;
  padding: 10px 14px;
  border: 0;
  border-radius: 999px;
  background: #2563eb;
  color: #ffffff;
}
.back-to-top.is-visible {
  opacity: 1;
  pointer-events: auto;
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
export class ScrollTools {
  constructor() {
    this.progress = $("#scrollProgress");
    this.backButton = $("#backToTop");
    this.init();
  }
}
```

```
init() {
  window.addEventListener("scroll", () => this.update());
  this.backButton.addEventListener("click", () => window.scrollTo({ top: 0,
behavior: "smooth" }));
  this.update();
}
```

```
update() {
  const total = document.documentElement.scrollHeight - window.innerHeight;
  const progress = total > 0 ? (window.scrollY / total) * 100 : 0;
  this.progress.style.width = `${progress}%`;
  this.backButton.classList.toggle("is-visible", window.scrollY > 400);
}
```

```

    }
  }

  document.addEventListener("DOMContentLoaded", () => {
    new ScrollTools();
  });

```

JS normal copiar y pegar

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

```

```

class ScrollTools {
  constructor() {
    this.progress = $("#scrollProgress");
    this.backButton = $("#backToTop");
    this.init();
  }

  init() {
    window.addEventListener("scroll", () => this.update());
    this.backButton.addEventListener("click", () => window.scrollTo({ top: 0,
behavior: "smooth" }));
    this.update();
  }

  update() {
    const total = document.documentElement.scrollHeight - window.innerHeight;
    const progress = total > 0 ? (window.scrolly / total) * 100 : 0;
    this.progress.style.width = `${progress}%`;
    this.backButton.classList.toggle("is-visible", window.scrolly > 400);
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new ScrollTools();
});

```

8. Back to top

Botón flotante que aparece al hacer scroll hacia abajo y permite volver al inicio.

HTML copiar y pegar

```
<button class="back-to-top" id="backToTop">⬆️</button>
```

CSS copiar y pegar

```
.back-to-top {  
  position: fixed; right: 20px; bottom: 20px;  
  opacity: 0; pointer-events: none;  
  transition: opacity var(--transition);  
}  
.back-to-top.is-visible {  
  opacity: 1; pointer-events: auto;  
}
```

JS modular copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {  
  const button = document.getElementById("backToTop");  
  if (!button) return;  
  
  const update = () => {  
    button.classList.toggle("is-visible", window.scrollY > 320);  
  };  
  
  window.addEventListener("scroll", update);  
  button.addEventListener("click", () => {  
    window.scrollTo({ top: 0, behavior: "smooth" });  
  });  
  update();  
});
```

JS normal copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {  
  const button = document.getElementById("backToTop");  
  if (!button) return;  
  
  const update = () => {  
    button.classList.toggle("is-visible", window.scrollY > 320);  
  };  
  
  window.addEventListener("scroll", update);  
  button.addEventListener("click", () => {  
    window.scrollTo({ top: 0, behavior: "smooth" });  
  });  
  update();  
});
```

9. Reveal on scroll

Efecto de aparición gradual de elementos utilizando el API IntersectionObserver.

HTML copiar y pegar

```
<div class="reveal">Contenido animado</div>
```

CSS copiar y pegar

```
.reveal {  
  opacity: 0; transform: translateY(18px);  
  transition: opacity 0.6s ease, transform 0.6s ease;  
}  
.reveal.is-visible {  
  opacity: 1; transform: translateY(0);  
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);  
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];  
  
export class RevealOnScroll {  
  constructor(selector = ".reveal") {  
    this.elements = $$(selector);  
    this.observer = new IntersectionObserver((entries) => {  
      entries.forEach((entry) => {  
        if (entry.isIntersecting) entry.target.classList.add("is-visible");  
      });  
    }, { threshold: 0.15 });  
    this.elements.forEach((element) => this.observer.observe(element));  
  }  
}  
  
document.addEventListener("DOMContentLoaded", () => {  
  new RevealOnScroll();  
});
```

JS normal copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);  
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];  
  
class RevealOnScroll {  
  constructor(selector = ".reveal") {  
    this.elements = $$(selector);  
    this.observer = new IntersectionObserver((entries) => {  
      entries.forEach((entry) => {  
        if (entry.isIntersecting) entry.target.classList.add("is-visible");  
      });  
    }, { threshold: 0.15 });  
    this.elements.forEach((element) => this.observer.observe(element));  
  }  
}  
  
document.addEventListener("DOMContentLoaded", () => {  
  new RevealOnScroll();  
});
```

10. Contador animado

Sube el valor numérico de una estadística desde 0 hasta la meta de forma progresiva.

HTML copiar y pegar

```
<span data-counter data-target="150">0</span>
```

CSS copiar y pegar

```
[data-counter] {  
  display: inline-block;  
  font-size: 2rem;  
  font-weight: 700;  
  color: #2563eb;  
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);  
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
export class CounterAnimation {  
  constructor(selector = "[data-counter]") {  
    this.counters = $$(selector);  
    this.init();  
  }  
  
  init() {  
    this.counters.forEach((counter) => {  
      const target = Number(counter.dataset.target || 0);  
      let current = 0;  
      const step = Math.max(1, Math.ceil(target / 40));  
      const interval = setInterval(() => {  
        current += step;  
        if (current >= target) {  
          current = target;  
          clearInterval(interval);  
        }  
        counter.textContent = current;  
      }, 30);  
    });  
  }  
}
```

```
document.addEventListener("DOMContentLoaded", () => {  
  new CounterAnimation();  
});
```

JS normal copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);  
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
class CounterAnimation {  
  constructor(selector = "[data-counter]") {  
    this.counters = $$(selector);  
    this.init();  
  }  
  
  init() {  
    this.counters.forEach((counter) => {  
      const target = Number(counter.dataset.target || 0);
```



```
let current = 0;
const step = Math.max(1, Math.ceil(target / 40));
const interval = setInterval(() => {
  current += step;
  if (current >= target) {
    current = target;
    clearInterval(interval);
  }
  counter.textContent = current;
}, 30);
});
}
}

document.addEventListener("DOMContentLoaded", () => {
  new CounterAnimation();
});
```

Guía de reciclaje de componentes web - Funcionalidades 10 a 25

Extensión del documento 1. Mantiene el mismo formato: explicación, HTML, CSS, JavaScript modular, JavaScript normal, variables que debes cambiar, pasos, errores comunes y checklist.

Este documento sigue el orden de la tabla de contenido del recurso unificado. En esa tabla, la funcionalidad 10 es Contador animado y la 25 es Acordeón. Por eso esta guía cubre exactamente ese bloque para que puedas continuar después del documento 1 sin perder la numeración original.

#	Funcionalidad	Uso principal
10	Contador animado	Cuenta visualmente desde cero hasta un valor objetivo. Es útil para secciones de métricas, dashboards, estadísticas, resultados, landing pages y tarje...
11	Login demo	Crea una pantalla o tarjeta de inicio de sesión para practicar formularios, validación, cambios de estado y almacenamiento de sesión. En un proyecto r...
12	Logout demo	Cierra la sesión simulada y limpia datos temporales del navegador. Se complementa con el login demo y con componentes que muestran contenido solo cuan...
13	Recordar usuario	Guarda el correo, nombre o preferencia del usuario para rellenar campos automáticamente. Es común en formularios de login, preferencias, temas visuale...
14	Password toggle	Permite mostrar u ocultar una contraseña. Es una funcionalidad muy común en formularios de login, registro y cambio de contraseña.
15	Validación de formulario	Valida campos obligatorios, correo, longitud mínima y reglas personalizadas antes de procesar o enviar datos.
16	Mensajes por campo	Muestra errores o ayudas debajo de cada input. Complementa la validación y hace que el usuario sepa exactamente qué corregir.
17	Character counter	Cuenta los caracteres de un textarea o input. Se usa mucho en comentarios, bios, descripciones, formularios de contacto y publicaciones.
18	Range preview	Muestra en tiempo real el valor de un input type="range". Es útil para filtros de precio, volumen, porcentaje, rating, tamaños, cantidades y configura...
19	File info	Muestra el nombre, tamaño y tipo de archivo seleccionado. Es útil en formularios con carga de CV, imágenes, documentos, facturas o evidencias.
20	Autosave draft	Guarda automáticamente un borrador mientras el usuario escribe. Evita pérdida de información si se cierra la página por accidente.
21	Stepper / Wizard	Divide un proceso en pasos. Es común en formularios largos, registros, compras, onboarding y creación de perfiles.
22	Skeleton loader	Muestra una estructura de carga temporal antes de renderizar datos reales. Es muy usado en páginas modernas para que la espera se sienta más profesio...
23	Select personalizado	Reemplaza el select nativo por un selector bonito, controlable y editable. Permite opciones con estilo, flecha personalizada, blur y mejor integración...

#	Funcionalidad	Uso principal
24	Tabs / pestañas	Permite cambiar entre paneles internos sin recargar la página. Se usa en perfiles, productos, dashboards, documentación y configuraciones.
25	Acordeón / FAQ	Abre y cierra bloques de contenido. Es perfecto para preguntas frecuentes, secciones de ayuda, detalles de producto y menús informativos.

Cómo usar este documento

- Primero identifica la funcionalidad que quieres reutilizar según el número y el nombre.
- Copia el HTML mínimo en tu index.html o en la sección donde lo necesites.
- Copia el CSS en styles.css. Si ya tienes variables globales, adapta colores, radios y espacios.
- Elige solo una versión de JavaScript: modular con funciones.js + script.js, o normal con script-normal.js.
- Cambia los ids, clases y data-* exactamente donde se indique en la tabla de variables.
- Prueba la funcionalidad sola antes de mezclarla con otros componentes.

10. Contador animado

¿Qué hace?

Cuenta visualmente desde cero hasta un valor objetivo. Es útil para secciones de métricas, dashboards, estadísticas, resultados, landing pages y tarjetas tipo "clientes atendidos", "proyectos", "ventas" o "porcentaje de avance".

¿Cuándo usarlo?

Úsalo cuando quieras mostrar un número de forma más atractiva que un texto estático. También sirve para practicar manipulación del DOM, intervalos de tiempo y atributos data-.*.

Estructura que debes copiar

HTML mínimo

```
<section class="stats-grid">
  <article class="stat-card reveal">
    <span class="stat-card__number" data-counter data-target="150"
      data-suffix="+">0</span>
    <p>Proyectos realizados</p>
  </article>

  <article class="stat-card reveal">
    <span class="stat-card__number" data-counter data-target="98"
      data-suffix="%">0</span>
    <p>Satisfacción</p>
  </article>
</section>
```

CSS necesario

```
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 1rem;
}

.stat-card {
  padding: 1.4rem;
  border-radius: 1.2rem;
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid rgba(255, 255, 255, 0.22);
  backdrop-filter: blur(14px);
  text-align: center;
}

.stat-card__number {
  display: block;
  font-size: clamp(2rem, 5vw, 3rem);
  font-weight: 800;
  color: var(--primary, #2563eb);
}
```

funciones.js - versión modular con export

```
export class AnimatedCounters {
  constructor(selector = "[data-counter]", duration = 1200) {
    this.counters = document.querySelectorAll(selector);
    this.duration = duration;
    this.start();
  }

  start() {
    this.counters.forEach((counter) => this.animate(counter));
  }

  animate(counter) {
    const target = Number(counter.dataset.target);
    const suffix = counter.dataset.suffix || "";
    const startTime = performance.now();

    const update = (currentTime) => {
      const elapsed = currentTime - startTime;
      const progress = Math.min(elapsed / this.duration, 1);
      const value = Math.floor(progress * target);

      counter.textContent = `${value}${suffix}`;

      if (progress < 1) {
        requestAnimationFrame(update);
      }
    };

    requestAnimationFrame(update);
  }
}
```

script.js - importación e inicialización

```
import { AnimatedCounters } from "./funciones.js";

new AnimatedCounters("[data-counter]", 1400);
```

script-normal.js - versión normal sin módulos

```
const counters = document.querySelectorAll("[data-counter]");

counters.forEach((counter) => {
  const target = Number(counter.dataset.target);
  const suffix = counter.dataset.suffix || "";
  let current = 0;
  const step = Math.ceil(target / 60);

  const interval = setInterval(() => {
    current += step;
    if (current >= target) {
      current = target;
      clearInterval(interval);
    }
    counter.textContent = `${current}${suffix}`;
  }, 20);
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
data-target	Número final del contador.	Cámbialo por el valor que quieras mostrar.
data-suffix	Símbolo que aparece al final.	Puedes usar +, %, K, M, COP o dejarlo vacío.
.stat-card	Tarjeta visual de la métrica.	Puedes cambiar colores, fondo, blur y bordes.
duration	Duración de la animación.	Aumenta el valor para que cuente más lento.

Paso a paso para reciclarlo en otro proyecto

1. Copia el HTML del contador dentro de la sección donde mostrarás las métricas.
2. Ajusta data-target según el número final que necesitas.
3. Agrega o quita data-suffix según quieras mostrar %, + u otro símbolo.
4. Copia el CSS de .stats-grid y .stat-card en tu archivo styles.css.
5. En funciones.js exporta la clase AnimatedCounters.
6. En script.js importa la clase y crea una instancia con new AnimatedCounters().
7. Prueba que los números avancen al cargar la página.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Poner data-target con texto en vez de número.
- Olvidar importar la clase si usas módulos.
- Usar demasiados contadores con intervalos pesados; requestAnimationFrame es más fluido.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Contador animado dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Contador animado dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init10() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Contador animado.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.

4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Si quieres que el contador se active solo cuando aparece en pantalla, combínalo con IntersectionObserver o con la funcionalidad Reveal on scroll del documento anterior.

11. Login demo

¿Qué hace?

Crea una pantalla o tarjeta de inicio de sesión para practicar formularios, validación, cambios de estado y almacenamiento de sesión. En un proyecto real se conecta con backend, API o Firebase.

¿Cuándo usarlo?

Úsalo en portafolios, dashboards, intranets de práctica, paneles administrativos y proyectos donde quieras simular ingreso de usuario.

Estructura que debes copiar

HTML mínimo

```
<section class="auth-card" id="loginSection">
  <h2>Iniciar sesión</h2>

  <form id="loginForm" novalidate>
    <label for="loginEmail">Correo</label>
    <input id="loginEmail" type="email" placeholder="correo@ejemplo.com" required>
    <small class="field-error" data-error="loginEmail"></small>

    <label for="loginPassword">Contraseña</label>
    <input id="loginPassword" type="password" placeholder="Mínimo 6 caracteres"
      required minlength="6">
    <small class="field-error" data-error="loginPassword"></small>

    <button type="submit">Entrar</button>
  </form>

  <p id="loginMessage" class="form-message"></p>
</section>
```

CSS necesario

```
.auth-card {
  max-width: 420px;
  margin: 2rem auto;
  padding: 2rem;
  border-radius: 1.5rem;
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid rgba(255, 255, 255, 0.2);
  backdrop-filter: blur(16px);
}

.auth-card label {
  display: block;
  margin-top: 1rem;
  margin-bottom: .4rem;
  font-weight: 700;
}

.auth-card input {
  width: 100%;
  padding: .85rem 1rem;
  border-radius: .9rem;
  border: 1px solid rgba(255, 255, 255, 0.28);
  background: rgba(255, 255, 255, 0.12);
  color: inherit;
}

.field-error {
  display: block;
  min-height: 1rem;
  margin-top: .3rem;
  color: #fca5a5;
  font-size: .82rem;
}

.form-message.is-success { color: #86efac; }
.form-message.is-error { color: #fca5a5; }
```


funciones.js - versión modular con export

```
export class LoginDemo {
  constructor(formId, messageId) {
    this.form = document.getElementById(formId);
    this.message = document.getElementById(messageId);
    this.init();
  }

  init() {
    this.form.addEventListener("submit", (event) => {
      event.preventDefault();
      this.login();
    });
  }

  login() {
    const email = this.form.querySelector("#loginEmail").value.trim();
    const password = this.form.querySelector("#loginPassword").value.trim();

    if (!email || !password) {
      this.showMessage("Completa todos los campos.", "error");
      return;
    }

    if (password.length < 6) {
      this.showMessage("La contraseña debe tener mínimo 6 caracteres.",
        "error");
      return;
    }

    const user = { email, loggedAt: new Date().toISOString() };
    sessionStorage.setItem("kit-session-user", JSON.stringify(user));
    this.showMessage("Ingreso correcto. Sesión demo guardada.", "success");
  }

  showMessage(text, type) {
    this.message.textContent = text;
    this.message.className = `form-message is-${type}`;
  }
}
```

script.js - importación e inicialización

```
import { LoginDemo } from "./funciones.js";

new LoginDemo("loginForm", "loginMessage");
```

script-normal.js - versión normal sin módulos

```
const loginForm = document.getElementById("loginForm");
const loginMessage = document.getElementById("loginMessage");

loginForm.addEventListener("submit", (event) => {
  event.preventDefault();

  const email = document.getElementById("loginEmail").value.trim();
  const password = document.getElementById("loginPassword").value.trim();

  if (!email || !password) {
    loginMessage.textContent = "Completa todos los campos.";
    loginMessage.className = "form-message is-error";
    return;
  }

  sessionStorage.setItem("kit-session-user", JSON.stringify({ email }));
  loginMessage.textContent = "Ingreso correcto.";
  loginMessage.className = "form-message is-success";
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#loginForm	Formulario que escucha el submit.	Cambia el id si tu formulario tiene otro nombre.
#loginEmail	Campo de correo.	Debe coincidir con el selector usado en JS.
#loginPassword	Campo de contraseña.	Puedes cambiar minlength según tu regla.
kit-session-user	Clave de sessionStorage.	Cámbiala si usas varios proyectos.

Paso a paso para reciclarlo en otro proyecto

1. Copia la tarjeta de login en la página donde quieres simular el acceso.
2. Verifica que el formulario tenga id="loginForm" o ajusta el id en JavaScript.
3. Conecta el CSS de .auth-card, .field-error y .form-message.
4. Si usas módulos, exporta LoginDemo en funciones.js e impórtalo en script.js.
5. Prueba enviar el formulario vacío, con contraseña corta y con datos válidos.
6. Para producción reemplaza la simulación por una llamada fetch a tu backend.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Creer que este login es seguridad real.
- No usar event.preventDefault() y que la página se recargue.
- Repetir ids si hay más de un login.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Login demo dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Login demo dentro de una card reutilizable y controla el estado con clases como is-active o is-open.

- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init11()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Login demo.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

No guardes contraseñas en localStorage ni sessionStorage. Para producción usa backend, tokens seguros, HTTPS y validación del servidor.

12. Logout demo

¿Qué hace?

Cierra la sesión simulada y limpia datos temporales del navegador. Se complementa con el login demo y con componentes que muestran contenido solo cuando existe usuario.

¿Cuándo usarlo?

Úsalo en dashboards de práctica, paneles demo y proyectos donde quieras simular la entrada y salida de un usuario.

Estructura que debes copiar

HTML mínimo

```
<div class="user-panel">
  <span id="userStatus">Sin sesión activa</span>
  <button id="logoutButton" type="button">Cerrar sesión</button>
</div>
```

CSS necesario

```
.user-panel {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  padding: 1rem;
  border-radius: 1rem;
  background: rgba(255, 255, 255, 0.1);
}

#logoutButton {
  border: 0;
  border-radius: .8rem;
  padding: .75rem 1rem;
  cursor: pointer;
  color: #fff;
  background: linear-gradient(135deg, #ef4444, #f97316);
}
```

funciones.js - versión modular con export

```
export class LogoutDemo {
  constructor(buttonId, statusId, storageKey = "kit-session-user") {
    this.button = document.getElementById(buttonId);
    this.status = document.getElementById(statusId);
    this.storageKey = storageKey;
    this.init();
  }

  init() {
    this.renderStatus();
    this.button.addEventListener("click", () => this.logout());
  }

  renderStatus() {
    const user = JSON.parse(sessionStorage.getItem(this.storageKey));
    this.status.textContent = user ? `Sesión: ${user.email}` : "Sin sesión activa";
  }

  logout() {
    sessionStorage.removeItem(this.storageKey);
    this.renderStatus();
  }
}
```

script.js - importación e inicialización

```
import { LogoutDemo } from "../funciones.js";

new LogoutDemo("logoutButton", "userStatus", "kit-session-user");
```

script-normal.js - versión normal sin módulos

```
const logoutButton = document.getElementById("logoutButton");
const userStatus = document.getElementById("userStatus");

function renderUserStatus() {
  const user = JSON.parse(sessionStorage.getItem("kit-session-user"));
  userStatus.textContent = user ? `Sesión: ${user.email}` : "Sin sesión activa";
}

logoutButton.addEventListener("click", () => {
  sessionStorage.removeItem("kit-session-user");
  renderUserStatus();
});

renderUserStatus();
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
storageKey	Clave usada para guardar la sesión.	Debe ser la misma del login.
#logoutButton	Botón que cierra sesión.	Cambia el id si usas otro botón.
#userStatus	Texto donde se muestra el estado.	Puedes ponerlo en navbar o panel lateral.

Paso a paso para reciclarlo en otro proyecto

1. Agrega el panel o solo el botón de cerrar sesión donde lo necesites.
2. Asegúrate de usar la misma clave de sessionStorage que el login.
3. Copia el CSS o adapta el botón a tu diseño.
4. Inicializa la clase LogoutDemo después de LoginDemo.
5. Inicia sesión, recarga y verifica que el usuario aparezca.
6. Presiona cerrar sesión y revisa que el estado se actualice.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Usar localStorage y sessionStorage mezclados sin control.
- Eliminar una clave diferente a la que guarda el login.
- No actualizar visualmente el estado después de cerrar sesión.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.

Paso	Qué hacer	Recomendación
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Logout demo dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Logout demo dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init12() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Logout demo.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Después de cerrar sesión en producción también debes invalidar el token del servidor o borrar cookies seguras si las usas.

13. Recordar usuario

¿Qué hace?

Guarda el correo, nombre o preferencia del usuario para rellenar campos automáticamente. Es común en formularios de login, preferencias, temas visuales y configuraciones.

¿Cuándo usarlo?

Úsalo cuando quieras mejorar experiencia de usuario sin pedir los mismos datos cada vez.

Estructura que debes copiar

HTML mínimo

```
<label class="remember-row">
  <input type="checkbox" id="rememberUser">
  Recordar mi correo
</label>
```

CSS necesario

```
.remember-row {
  display: flex;
  align-items: center;
  gap: .6rem;
  margin-top: .8rem;
  font-size: .9rem;
  color: var(--text-muted, #cbd5e1);
}

.remember-row input {
  width: 18px;
  height: 18px;
  accent-color: var(--primary, #2563eb);
}
```

funciones.js - versión modular con export

```
export class RememberUser {
  constructor(emailInputId, checkboxId, storageKey = "kit-remember-email") {
    this.emailInput = document.getElementById(emailInputId);
    this.checkbox = document.getElementById(checkboxId);
    this.storageKey = storageKey;
    this.init();
  }

  init() {
    const savedEmail = localStorage.getItem(this.storageKey);

    if (savedEmail) {
      this.emailInput.value = savedEmail;
      this.checkbox.checked = true;
    }
  }

  saveIfChecked() {
    if (this.checkbox.checked) {
      localStorage.setItem(this.storageKey, this.emailInput.value.trim());
    } else {
      localStorage.removeItem(this.storageKey);
    }
  }
}
```

script.js - importación e inicialización

```
import { RememberUser } from "../funciones.js";

const rememberUser = new RememberUser("loginEmail", "rememberUser");

// Dentro del submit del login, después de validar:
rememberUser.saveIfChecked();
```

script-normal.js - versión normal sin módulos

```
const rememberCheckbox = document.getElementById("rememberUser");
const emailInput = document.getElementById("loginEmail");
const savedEmail = localStorage.getItem("kit-remember-email");

if (savedEmail) {
  emailInput.value = savedEmail;
  rememberCheckbox.checked = true;
}

function saveRememberedEmail() {
  if (rememberCheckbox.checked) {
    localStorage.setItem("kit-remember-email", emailInput.value.trim());
  } else {
    localStorage.removeItem("kit-remember-email");
  }
}
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#rememberUser	Checkbox de recordar usuario.	Cambia el id si tienes varios formularios.
#loginEmail	Input que se rellena automáticamente.	Debe ser el campo de correo real.
kit-remember-email	Clave de localStorage.	Personalízala por proyecto.

Paso a paso para reciclarlo en otro proyecto

1. Agrega el checkbox debajo del correo en el login.
2. Copia el CSS para que el checkbox se alinee bien.
3. Crea la instancia de RememberUser en script.js.
4. Dentro del submit, llama saveIfChecked() después de validar los datos.
5. Prueba marcar el checkbox, iniciar sesión y recargar la página.
6. Prueba desmarcarlo y verificar que el correo ya no se guarde.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Guardar contraseñas en localStorage: no se debe hacer.
- Llamar saveIfChecked antes de validar el correo.
- No cambiar la clave si mezclas varios proyectos.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Recordar usuario dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Recordar usuario dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init13() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Recordar usuario.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Solo recuerda datos no sensibles. Recordar un correo es aceptable; recordar contraseñas en el navegador con JS no lo es.

14. Password toggle

¿Qué hace?

Permite mostrar u ocultar una contraseña. Es una funcionalidad muy común en formularios de login, registro y cambio de contraseña.

¿Cuándo usarlo?

Úsalo cuando tengas inputs type="password" y quieras mejorar la experiencia del usuario.

Estructura que debes copiar

HTML mínimo

```
<div class="password-field">
  <input id="loginPassword" type="password" placeholder="Contraseña">
  <button type="button" data-toggle-password="loginPassword">Mostrar</button>
</div>
```

CSS necesario

```
.password-field {
  display: flex;
  align-items: center;
  gap: .5rem;
}

.password-field input {
  flex: 1;
}

.password-field button {
  width: auto;
  min-height: auto;
  padding: .75rem .9rem;
  border-radius: .8rem;
  border: 1px solid rgba(255, 255, 255, 0.2);
  background: rgba(255, 255, 255, 0.12);
  color: inherit;
}
```

funciones.js - versión modular con export

```
export class PasswordToggle {
  constructor(selector = "[data-toggle-password]") {
    this.buttons = document.querySelectorAll(selector);
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.toggle(button));
    });
  }

  toggle(button) {
    const inputId = button.dataset.togglePassword;
    const input = document.getElementById(inputId);
    const isHidden = input.type === "password";

    input.type = isHidden ? "text" : "password";
    button.textContent = isHidden ? "Ocultar" : "Mostrar";
  }
}
```

script.js - importación e inicialización

```
import { PasswordToggle } from "./funciones.js";

new PasswordToggle("[data-toggle-password]");
```

script-normal.js - versión normal sin módulos

```
document.querySelectorAll("[data-toggle-password]").forEach((button) => {
  button.addEventListener("click", () => {
    const input = document.getElementById(button.dataset.togglePassword);
    const isHidden = input.type === "password";

    input.type = isHidden ? "text" : "password";
    button.textContent = isHidden ? "Ocultar" : "Mostrar";
  });
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
data-toggle-password	Debe contener el id del input password.	Si el input cambia de id, actualiza este valor.
#loginPassword	Input que cambia entre password y text.	Puedes usarlo en cualquier formulario.
.password-field	Contenedor visual.	Puedes modificar flex, gap y colores.

Paso a paso para reciclarlo en otro proyecto

1. Encierra el input y el botón dentro de .password-field.
2. El botón debe tener data-toggle-password con el id del input.
3. Copia el CSS para alinear el botón con el input.
4. Exporta PasswordToggle o copia la versión normal.
5. Prueba que el botón cambie de Mostrar a Ocultar.
6. Verifica que el valor de la contraseña no se borre al alternar.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Poner data-toggle-password sin valor.
- Usar un id que no existe.
- Crear varios inputs con el mismo id.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.

Paso	Qué hacer	Recomendación
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Password toggle dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca Password toggle dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida `ids`, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init14()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Password toggle.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa `ids` y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas `import/export`, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de `localStorage` o `sessionStorage`, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

En móviles, este patrón mejora mucho la experiencia porque reduce errores al escribir contraseñas.

15. Validación de formulario

¿Qué hace?

Valida campos obligatorios, correo, longitud mínima y reglas personalizadas antes de procesar o enviar datos.

¿Cuándo usarlo?

Úsalo en login, registro, contacto, cotizaciones, compras, formularios de soporte y paneles administrativos.

Estructura que debes copiar

HTML mínimo

```
<form id="contactForm" novalidate>
  <label for="contactName">Nombre</label>
  <input id="contactName" type="text" required minlength="3">
  <small class="field-error" data-error="contactName"></small>

  <label for="contactEmail">Correo</label>
  <input id="contactEmail" type="email" required>
  <small class="field-error" data-error="contactEmail"></small>

  <button type="submit">Enviar</button>
</form>
```

CSS necesario

```
input.is-invalid,
textarea.is-invalid {
  border-color: #ef4444;
  box-shadow: 0 0 4px rgba(239, 68, 68, .16);
}

input.is-valid,
textarea.is-valid {
  border-color: #22c55e;
  box-shadow: 0 0 4px rgba(34, 197, 94, .14);
}

.field-error {
  color: #fca5a5;
  font-size: .82rem;
  min-height: 1rem;
}
```

funciones.js - versión modular con export

```

export class FormValidator {
  constructor(formId) {
    this.form = document.getElementById(formId);
    this.fields = this.form.querySelectorAll("input, textarea, select");
    this.init();
  }

  init() {
    this.form.addEventListener("submit", (event) => {
      event.preventDefault();

      if (this.validateAll()) {
        console.log("Formulario válido");
      }
    });

    this.fields.forEach((field) => {
      field.addEventListener("input", () => this.validateField(field));
    });
  }

  validateAll() {
    return [...this.fields].every((field) => this.validateField(field));
  }

  validateField(field) {
    let message = "";

    if (field.hasAttribute("required") && field.value.trim() === "") {
      message = "Este campo es obligatorio.";
    } else if (field.type === "email" && !field.value.includes("@")) {
      message = "Ingresa un correo válido.";
    } else if (field.minLength > 0 && field.value.length < field.minLength) {
      message = `Mínimo ${field.minLength} caracteres.`;
    }

    this.showError(field, message);
    return message === "";
  }

  showError(field, message) {
    const error = this.form.querySelector(`[data-error="${field.id}"]`);
    field.classList.toggle("is-invalid", Boolean(message));
    field.classList.toggle("is-valid", !message && field.value.trim() !== "");
    if (error) error.textContent = message;
  }
}

```

script.js - importación e inicialización

```

import { FormValidator } from "./funciones.js";

new FormValidator("contactForm");

```

script-normal.js - versión normal sin módulos

```
const contactForm = document.getElementById("contactForm");
const fields = contactForm.querySelectorAll("input, textarea, select");

function validateField(field) {
  let message = "";

  if (field.required && field.value.trim() === "") message = "Este campo es obligatorio.";
  if (field.type === "email" && !field.value.includes("@")) message = "Correo inválido.";

  const error = contactForm.querySelector(`[data-error="${field.id}"]`);
  field.classList.toggle("is-invalid", Boolean(message));
  if (error) error.textContent = message;

  return message === "";
}

contactForm.addEventListener("submit", (event) => {
  event.preventDefault();
  const valid = [...fields].every(validateField);
  if (valid) console.log("Formulario válido");
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
required	Indica que el campo no puede ir vacío.	Agrégallo solo a campos obligatorios.
minlength	Longitud mínima.	Cambia el número según tu necesidad.
data-error	Relaciona el mensaje con el id del input.	Debe coincidir exactamente con el id.
.is-invalid / .is-valid	Clases visuales.	Cambia colores y sombra en CSS.

Paso a paso para reciclarlo en otro proyecto

1. Agrega required, type="email" o minlength donde corresponda.
2. Debajo de cada input agrega un small con data-error igual al id del campo.
3. Copia los estilos de campos válidos e inválidos.
4. Exporta FormValidator en funciones.js.
5. Inicializa new FormValidator("idDelFormulario") en script.js.
6. Prueba enviar vacío, con email incorrecto y con datos correctos.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Olvidar novalidate si quieres controlar mensajes con JS.
- Usar data-error diferente al id del campo.
- Validar solo al submit y no en input; la experiencia se siente peor.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Validación de formulario dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Validación de formulario dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init15() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Validación de formulario.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

La validación del frontend mejora la experiencia, pero no reemplaza la validación del backend.

16. Mensajes por campo

¿Qué hace?

Muestra errores o ayudas debajo de cada input. Complementa la validación y hace que el usuario sepa exactamente qué corregir.

¿Cuándo usarlo?

Úsalo en formularios profesionales donde cada campo debe tener retroalimentación individual.

Estructura que debes copiar

HTML mínimo

```
<div class="field">
  <label for="username">Usuario</label>
  <input id="username" type="text" required>
  <small class="field-help">Usa letras y números.</small>
  <small class="field-error" data-error="username"></small>
</div>
```

CSS necesario

```
.field {
  margin-bottom: 1rem;
}

.field-help {
  display: block;
  margin-top: .3rem;
  color: var(--text-muted, #94a3b8);
  font-size: .78rem;
}

.field-error {
  display: block;
  min-height: 1rem;
  margin-top: .25rem;
  color: #fb7185;
  font-size: .8rem;
}
```

funciones.js - versión modular con export

```
export function setFieldMessage(form, fieldId, message = "") {
  const errorBox = form.querySelector(`[data-error="${fieldId}"]`);
  const field = form.querySelector(`#${fieldId}`);

  if (errorBox) errorBox.textContent = message;
  if (field) field.classList.toggle("is-invalid", Boolean(message));
}

export function clearFieldMessages(form) {
  form.querySelectorAll("[data-error]").forEach((errorBox) => {
    errorBox.textContent = "";
  });

  form.querySelectorAll(".is-invalid").forEach((field) => {
    field.classList.remove("is-invalid");
  });
}
```

script.js - importación e inicialización

```
import { setFieldMessage, clearFieldMessages } from "./funciones.js";

const form = document.getElementById("contactForm");
setFieldMessage(form, "username", "El usuario es obligatorio.");
// clearFieldMessages(form);
```

script-normal.js - versión normal sin módulos

```
function setFieldMessage(form, fieldId, message = "") {
  const errorBox = form.querySelector(`[data-error="${fieldId}"]`);
  const field = form.querySelector(`#${fieldId}`);

  if (errorBox) errorBox.textContent = message;
  if (field) field.classList.toggle("is-invalid", Boolean(message));
}
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
data-error	Conecta el mensaje con un input.	Debe tener el mismo valor que el id del input.
.field-help	Texto de ayuda normal.	Úsalo para explicar reglas antes de que haya error.
.field-error	Texto de error.	Solo se llena cuando algo está mal.

Paso a paso para reciclarlo en otro proyecto

1. Agrupa cada input dentro de un div.field.
2. Agrega field-help si quieres instrucciones permanentes.
3. Agrega field-error con data-error igual al id del input.
4. Usa setFieldMessage(form, id, mensaje) desde tu validación.
5. Usa clearFieldMessages(form) antes de validar todo si quieres limpiar errores previos.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Mostrar todos los errores en un solo párrafo genérico.
- No dejar espacio mínimo y que el diseño salte cuando aparece error.
- Usar mensajes muy largos que rompen la tarjeta.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Mensajes por campo dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca Mensajes por campo dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida `ids`, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init16()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Mensajes por campo.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa `ids` y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas `import/export`, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de `localStorage` o `sessionStorage`, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Los mensajes por campo deben ser cortos y específicos. Evita textos genéricos como "error".

17. Character counter

¿Qué hace?

Cuenta los caracteres de un textarea o input. Se usa mucho en comentarios, bios, descripciones, formularios de contacto y publicaciones.

¿Cuándo usarlo?

Úsalo cuando haya un límite de caracteres y el usuario necesite verlo en tiempo real.

Estructura que debes copiar

HTML mínimo

```
<label for="bio">Descripción</label>
<textarea id="bio" maxlength="160" placeholder="Escribe una descripción
  corta"></textarea>
<small id="bioCounter">0 / 160</small>
```

CSS necesario

```
textarea {
  width: 100%;
  min-height: 120px;
  resize: vertical;
  padding: 1rem;
  border-radius: 1rem;
}

#bioCounter {
  display: block;
  margin-top: .3rem;
  text-align: right;
  color: var(--text-muted, #94a3b8);
}

#bioCounter.is-limit {
  color: #f97316;
  font-weight: 700;
}
```

funciones.js - versión modular con export

```
export class CharacterCounter {
  constructor(inputId, counterId) {
    this.input = document.getElementById(inputId);
    this.counter = document.getElementById(counterId);
    this.max = Number(this.input.getAttribute("maxlength")) || 0;
    this.init();
  }

  init() {
    this.update();
    this.input.addEventListener("input", () => this.update());
  }

  update() {
    const length = this.input.value.length;
    this.counter.textContent = this.max ? `${length} / ${this.max}` : `${length}`;
    this.counter.classList.toggle("is-limit", this.max && length >= this.max *
      0.9);
  }
}
```

script.js - importación e inicialización

```
import { CharacterCounter } from "./funciones.js";

new CharacterCounter("bio", "bioCounter");
```

script-normal.js - versión normal sin módulos

```
const bio = document.getElementById("bio");
const bioCounter = document.getElementById("bioCounter");
const max = Number(bio.getAttribute("maxLength"));

function updateCounter() {
  const length = bio.value.length;
  bioCounter.textContent = `${length} / ${max}`;
  bioCounter.classList.toggle("is-limit", length >= max * 0.9);
}

bio.addEventListener("input", updateCounter);
updateCounter();
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
maxlength	Límite real del textarea.	Cámbialo según el máximo permitido.
#bioCounter	Elemento donde se muestra el conteo.	Cambia el id si tienes varios contadores.
.is-limit	Estado cercano al límite.	Puedes cambiarlo a rojo o naranja.

Paso a paso para reciclarlo en otro proyecto

1. Agrega maxlength al input o textarea.
2. Crea un small donde se verá el contador.
3. Copia el CSS de contador y estado is-limit.
4. Inicializa CharacterCounter con el id del campo y el id del contador.
5. Escribe hasta llegar cerca del límite y revisa que cambie el color.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Olvidar maxlength y no tener límite real.
- Usar innerHTML para un texto que no necesita HTML.
- No llamar update() al cargar y que aparezca vacío.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.

Paso	Qué hacer	Recomendación
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Character counter dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca Character counter dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida `ids`, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init17()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Character counter.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa `ids` y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas `import/export`, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de `localStorage` o `sessionStorage`, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Combina el contador con `maxlength` para que el límite sea real, no solo visual.

18. Range preview

¿Qué hace?

Muestra en tiempo real el valor de un input type="range". Es útil para filtros de precio, volumen, porcentaje, rating, tamaños, cantidades y configuraciones.

¿Cuándo usarlo?

Úsalo cuando un slider controle un valor que el usuario necesita leer claramente.

Estructura que debes copiar

HTML mínimo

```
<label for="priceRange">Precio máximo</label>
<input id="priceRange" type="range" min="0" max="100000" step="5000" value="50000">
<p>Valor seleccionado: <strong id="pricePreview">50000</strong></p>
```

CSS necesario

```
input[type="range"] {
  width: 100%;
  accent-color: var(--primary, #2563eb);
}

#pricePreview {
  color: var(--primary, #2563eb);
  font-size: 1.1rem;
}
```

funciones.js - versión modular con export

```
export class RangePreview {
  constructor(rangeId, previewId, formatter = null) {
    this.range = document.getElementById(rangeId);
    this.preview = document.getElementById(previewId);
    this.formatter = formatter || ((value) => value);
    this.init();
  }

  init() {
    this.update();
    this.range.addEventListener("input", () => this.update());
  }

  update() {
    this.preview.textContent = this.formatter(this.range.value);
  }
}
```

script.js - importación e inicialización

```
import { RangePreview } from "./funciones.js";

new RangePreview("priceRange", "pricePreview", (value) =>
  `$$${Number(value).toLocaleString("es-CO")}`);
```

script-normal.js - versión normal sin módulos

```
const priceRange = document.getElementById("priceRange");
const pricePreview = document.getElementById("pricePreview");

function updatePricePreview() {
  pricePreview.textContent = `$$${Number(priceRange.value).toLocaleString("es-CO")}`;
}

priceRange.addEventListener("input", updatePricePreview);
updatePricePreview();
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
min / max / step	Rango y salto del slider.	Ajusta según precio, porcentaje o cantidad.
value	Valor inicial.	Debe estar entre min y max.
formatter	Función para formatear el resultado.	Úsala para COP, %, kg, L, etc.

Paso a paso para reciclarlo en otro proyecto

1. Copia el input type="range" y el elemento preview.
2. Ajusta min, max, step y value.
3. Inicializa RangePreview con los ids correctos.
4. Agrega formatter si quieres moneda o unidad.
5. Mueve el slider y revisa que el valor cambie sin recargar.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- No actualizar el preview al cargar.
- Dejar step demasiado grande o demasiado pequeño.
- Mostrar número sin formato cuando es precio.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Range preview dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Range preview dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init18() o desde initApp().

- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Range preview.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Siempre formatea el valor según contexto: moneda, porcentaje, litros, kilogramos o estrellas.

19. File info

¿Qué hace?

Muestra el nombre, tamaño y tipo de archivo seleccionado. Es útil en formularios con carga de CV, imágenes, documentos, facturas o evidencias.

¿Cuándo usarlo?

Úsalo siempre que tengas `input type="file"` y quieras que el usuario confirme qué archivo eligió.

Estructura que debes copiar

HTML mínimo

```
<label for="fileInput">Subir archivo</label>
<input id="fileInput" type="file" accept=".pdf,.png,.jpg,.jpeg">
<p id="fileInfo" class="file-info">Ningún archivo seleccionado.</p>
```

CSS necesario

```
.file-info {
  margin-top: .6rem;
  padding: .75rem 1rem;
  border-radius: .9rem;
  background: rgba(255, 255, 255, 0.1);
  color: var(--text-muted, #cbd5e1);
}
```

funciones.js - versión modular con export

```
export class FileInfo {
  constructor(inputId, infoId) {
    this.input = document.getElementById(inputId);
    this.info = document.getElementById(infoId);
    this.init();
  }

  init() {
    this.input.addEventListener("change", () => this.update());
  }

  update() {
    const file = this.input.files[0];

    if (!file) {
      this.info.textContent = "Ningún archivo seleccionado.";
      return;
    }

    const sizeKb = (file.size / 1024).toFixed(1);
    this.info.textContent = `${file.name} - ${sizeKb} KB - ${file.type || "tipo desconocido"}`;
  }
}
```

script.js - importación e inicialización

```
import { FileInfo } from "./funciones.js";

new FileInfo("fileInput", "fileInfo");
```

script-normal.js - versión normal sin módulos

```
const fileInput = document.getElementById("fileInput");
const fileInfo = document.getElementById("fileInfo");

fileInput.addEventListener("change", () => {
  const file = fileInput.files[0];

  if (!file) {
    fileInfo.textContent = "Ningún archivo seleccionado.";
    return;
  }

  fileInfo.textContent = `${file.name} - ${((file.size / 1024).toFixed(1))} KB`;
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
accept	Tipos permitidos.	Cambia extensiones según tu caso.
file.size	Peso en bytes.	Puedes convertir a KB o MB.
file.type	Tipo MIME.	Sirve para validar imágenes o PDF.

Paso a paso para reciclarlo en otro proyecto

1. Agrega input type="file" y un contenedor de texto para la información.
2. Configura accept para limitar tipos de archivo.
3. Copia la clase FileInfo o la versión normal.
4. Prueba elegir archivo y cancelar selección.
5. Agrega validaciones de tamaño si el proyecto lo requiere.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Creer que accept es seguridad total; también valida en backend.
- No manejar el caso cuando el usuario cancela.
- Mostrar bytes sin convertir y que sea difícil de leer.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.

Paso	Qué hacer	Recomendación
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa File info dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca File info dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida `ids`, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init19()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a File info.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa `ids` y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas `import/export`, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de `localStorage` o `sessionStorage`, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Para cargas reales, valida también en backend el tipo y tamaño del archivo.

20. Autosave draft

¿Qué hace?

Guarda automáticamente un borrador mientras el usuario escribe. Evita pérdida de información si se cierra la página por accidente.

¿Cuándo usarlo?

Úsalo en comentarios largos, publicaciones, formularios, notas, mensajes y editores de contenido.

Estructura que debes copiar

HTML mínimo

```
<label for="draftText">Borrador</label>
<textarea id="draftText" placeholder="Escribe aquí..."></textarea>
<small id="draftStatus">Sin guardar todavía</small>
```

CSS necesario

```
#draftStatus {
  display: block;
  margin-top: .4rem;
  color: var(--text-muted, #94a3b8);
  font-size: .82rem;
}

#draftStatus.is-saved {
  color: #86efac;
}
```

funciones.js - versión modular con export

```
export class AutosaveDraft {
  constructor(inputId, statusId, storageKey = "kit-draft") {
    this.input = document.getElementById(inputId);
    this.status = document.getElementById(statusId);
    this.storageKey = storageKey;
    this.timeout = null;
    this.init();
  }

  init() {
    this.input.value = localStorage.getItem(this.storageKey) || "";
    this.input.addEventListener("input", () => this.scheduleSave());
  }

  scheduleSave() {
    this.status.textContent = "Guardando...";
    this.status.classList.remove("is-saved");
    clearTimeout(this.timeout);

    this.timeout = setTimeout(() => {
      localStorage.setItem(this.storageKey, this.input.value);
      this.status.textContent = "Borrador guardado automáticamente.";
      this.status.classList.add("is-saved");
    }, 600);
  }
}
```

script.js - importación e inicialización

```
import { AutosaveDraft } from "./funciones.js";

new AutosaveDraft("draftText", "draftStatus", "contact-draft");
```

script-normal.js - versión normal sin módulos

```
const draftText = document.getElementById("draftText");
const draftStatus = document.getElementById("draftStatus");
let draftTimeout;

draftText.value = localStorage.getItem("contact-draft") || "";

draftText.addEventListener("input", () => {
  draftStatus.textContent = "Guardando...";
  clearTimeout(draftTimeout);

  draftTimeout = setTimeout(() => {
    localStorage.setItem("contact-draft", draftText.value);
    draftStatus.textContent = "Borrador guardado.";
  }, 600);
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
storageKey	Clave donde se guarda el borrador.	Cámbiala por formulario o página.
600	Tiempo de espera antes de guardar.	Aumenta si quieres guardar menos seguido.
#draftText	Textarea que se guarda.	Puedes usar input o textarea.

Paso a paso para reciclarlo en otro proyecto

1. Agrega textarea y un status visual.
2. Define una clave única de localStorage para ese borrador.
3. Copia AutosaveDraft e inicialízalo.
4. Escribe texto, espera y recarga la página.
5. Confirma que el texto se restaura automáticamente.
6. Agrega un botón de limpiar borrador si el flujo lo necesita.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Usar la misma clave para varios formularios.
- Guardar información sensible sin avisar.
- Guardar en cada tecla sin debounce o setTimeout.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.

Paso	Qué hacer	Recomendación
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Autosave draft dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Autosave draft dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init20() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Autosave draft.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Agrega una opción de "descartar borrador" para que el usuario pueda limpiar el contenido guardado.

21. Stepper / Wizard

¿Qué hace?

Divide un proceso en pasos. Es común en formularios largos, registros, compras, onboarding y creación de perfiles.

¿Cuándo usarlo?

Úsalo cuando una tarea sea demasiado larga para una sola pantalla.

Estructura que debes copiar

HTML mínimo

```
<section class="stepper" id="projectStepper">
  <div class="stepper__indicators">
    <span class="is-active">1</span>
    <span>2</span>
    <span>3</span>
  </div>

  <article class="stepper__panel is-active" data-step="0">
    <h3>Paso 1: Datos básicos</h3>
    <input placeholder="Nombre del proyecto">
  </article>

  <article class="stepper__panel" data-step="1">
    <h3>Paso 2: Detalles</h3>
    <textarea placeholder="Descripción"></textarea>
  </article>

  <article class="stepper__panel" data-step="2">
    <h3>Paso 3: Confirmación</h3>
    <p>Revisa la información antes de enviar.</p>
  </article>

  <div class="stepper__actions">
    <button type="button" data-step-prev>Anterior</button>
    <button type="button" data-step-next>Siguiente</button>
  </div>
</section>
```

CSS necesario

```
.stepper__indicators {
  display: flex;
  gap: .5rem;
  margin-bottom: 1rem;
}

.stepper__indicators span {
  width: 32px;
  height: 32px;
  display: grid;
  place-items: center;
  border-radius: 999px;
  background: rgba(255, 255, 255, .12);
}

.stepper__indicators span.is-active {
  background: var(--primary, #2563eb);
  color: #fff;
}

.stepper__panel { display: none; }
.stepper__panel.is-active { display: block; }
.stepper__actions { display: flex; gap: .8rem; margin-top: 1rem; }
```

funciones.js - versión modular con export


```
export class Stepper {
  constructor(rootId) {
    this.root = document.getElementById(rootId);
    this.panels = this.root.querySelectorAll(".stepper__panel");
    this.indicators = this.root.querySelectorAll(".stepper__indicators span");
    this.prev = this.root.querySelector("[data-step-prev]");
    this.next = this.root.querySelector("[data-step-next]");
    this.current = 0;
    this.init();
  }

  init() {
    this.prev.addEventListener("click", () => this.goTo(this.current - 1));
    this.next.addEventListener("click", () => this.goTo(this.current + 1));
    this.render();
  }

  goTo(index) {
    if (index < 0 || index >= this.panels.length) return;
    this.current = index;
    this.render();
  }

  render() {
    this.panels.forEach((panel, index) => {
      panel.classList.toggle("is-active", index === this.current);
    });

    this.indicators.forEach((indicator, index) => {
      indicator.classList.toggle("is-active", index === this.current);
    });

    this.prev.disabled = this.current === 0;
    this.next.textContent = this.current === this.panels.length - 1 ? "Finalizar"
      : "Siguiente";
  }
}
```

script.js - importación e inicialización

```
import { Stepper } from "./funciones.js";

new Stepper("projectStepper");
```

script-normal.js - versión normal sin módulos

```
const stepper = document.getElementById("projectStepper");
const panels = stepper.querySelectorAll(".stepper__panel");
const indicators = stepper.querySelectorAll(".stepper__indicators span");
const prevButton = stepper.querySelector("[data-step-prev]");
const nextButton = stepper.querySelector("[data-step-next]");
let currentStep = 0;

function renderStepper() {
  panels.forEach((panel, index) => panel.classList.toggle("is-active", index ===
    currentStep));
  indicators.forEach((item, index) => item.classList.toggle("is-active", index ===
    currentStep));
}

prevButton.addEventListener("click", () => { currentStep = Math.max(0, currentStep -
  1); renderStepper(); });
nextButton.addEventListener("click", () => { currentStep = Math.min(panels.length - 1,
  currentStep + 1); renderStepper(); });
renderStepper();
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#projectStepper	Contenedor principal.	Cambia el id si tienes varios steppers.
.stepper__panel	Cada paso.	Duplica o elimina panels para más o menos pasos.

Elemento	Para qué sirve	Cómo adaptarlo
data-step-prev / data-step-next	Botones de navegación.	Deben estar dentro del stepper.

Paso a paso para reciclarlo en otro proyecto

1. Copia el contenedor .stepper completo.
2. Cambia títulos y campos dentro de cada .stepper__panel.
3. Agrega un indicador por cada panel.
4. Copia estilos de indicadores, panels y acciones.
5. Inicializa new Stepper("projectStepper").
6. Prueba avanzar, retroceder y llegar al último paso.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Tener más indicadores que panels o al contrario.
- Poner los botones fuera del root y que querySelector no los encuentre.
- No validar cada paso antes de avanzar si el formulario es real.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Stepper / Wizard dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Stepper / Wizard dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init21() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Stepper / Wizard.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

En un wizard real, valida cada paso antes de permitir avanzar al siguiente.

22. Skeleton loader

¿Qué hace?

Muestra una estructura de carga temporal antes de renderizar datos reales. Es muy usado en páginas modernas para que la espera se sienta más profesional.

¿Cuándo usarlo?

Úsalo mientras esperas datos de una API, una búsqueda, cards de productos, listas o tablas.

Estructura que debes copiar

HTML mínimo

```
<section id="contentLoader" class="skeleton-grid">
  <article class="skeleton-card"></article>
  <article class="skeleton-card"></article>
  <article class="skeleton-card"></article>
</section>
```

CSS necesario

```
.skeleton-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 1rem;
}

.skeleton-card {
  height: 140px;
  border-radius: 1rem;
  background: linear-gradient(90deg, #e2e8f0 25%, #f8fafc 37%, #e2e8f0 63%);
  background-size: 400% 100%;
  animation: skeleton-loading 1.2s infinite;
}

@keyframes skeleton-loading {
  0% { background-position: 100% 0; }
  100% { background-position: 0 0; }
}
```

funciones.js - versión modular con export

```
export class SkeletonLoader {
  constructor(containerId) {
    this.container = document.getElementById(containerId);
  }

  show(count = 3) {
    this.container.innerHTML = Array.from({ length: count }, () => {
      return `<article class="skeleton-card"></article>`;
    }).join("");
  }

  hideWithContent(html) {
    this.container.innerHTML = html;
  }
}
```

script.js - importación e inicialización

```
import { SkeletonLoader } from "../funciones.js";

const loader = new SkeletonLoader("contentLoader");
loader.show(3);

setTimeout(() => {
  loader.hideWithContent(`
    <article class="card">Contenido cargado</article>
    <article class="card">Otro contenido</article>
  `);
}, 1500);
```

script-normal.js - versión normal sin módulos

```
const contentLoader = document.getElementById("contentLoader");

function showSkeleton(count = 3) {
  contentLoader.innerHTML = Array.from({ length: count }, () => {
    return `<article class="skeleton-card"></article>`;
  }).join("");
}

showSkeleton(3);
setTimeout(() => {
  contentLoader.innerHTML = `<article class="card">Contenido cargado</article>`;
}, 1500);
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
count	Cantidad de placeholders.	Hazlo igual al número aproximado de cards reales.
.skeleton-card	Diseño del placeholder.	Cambia alto, radio o animación.
hideWithContent	Reemplaza skeleton por contenido real.	Pásale HTML seguro o creado con DOM.

Paso a paso para reciclarlo en otro proyecto

1. Agrega un contenedor donde irá la carga.
2. Copia el CSS del skeleton y keyframes.
3. Muestra skeleton antes de traer datos.
4. Cuando lleguen los datos, reemplaza con contenido real.
5. Prueba con setTimeout y luego cámbialo por fetch real.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Dejar skeleton infinito si falla la API.
- Usar demasiados skeletons y saturar la pantalla.
- Renderizar HTML no confiable sin sanitizar.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Skeleton loader dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Skeleton loader dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init22() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Skeleton loader.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Incluye estado de error si la API falla; no dejes el skeleton cargando para siempre.

23. Select personalizado

¿Qué hace?

Reemplaza el select nativo por un selector bonito, controlable y editable. Permite opciones con estilo, flecha personalizada, blur y mejor integración visual.

¿Cuándo usarlo?

Úsalo cuando el select nativo se vea mal o no combine con el diseño, como en tarjetas glassmorphism.

Estructura que debes copiar

HTML mínimo

```
<div class="custom-select" id="statusSelect">
  <input type="hidden" id="statusValue" value="pendiente">

  <button type="button" class="custom-select__button">
    <span class="custom-select__value">Pendiente</span>
    <span class="custom-select__arrow">▼</span>
  </button>

  <ul class="custom-select__options">
    <li class="custom-select__option is-selected"
      data-value="pendiente">Pendiente</li>
    <li class="custom-select__option" data-value="proceso">En proceso</li>
    <li class="custom-select__option" data-value="finalizado">Finalizado</li>
  </ul>
</div>
```

CSS necesario

```
.custom-select {
  position: relative;
  width: 100%;
  z-index: 20;
}

.custom-select__button {
  width: 100%;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: .9rem 1rem;
  border-radius: 1rem;
  border: 1px solid rgba(255, 255, 255, .25);
  background: rgba(255, 255, 255, .14);
  color: inherit;
  cursor: pointer;
}

.custom-select__options {
  position: absolute;
  top: calc(100% + .5rem);
  left: 0;
  width: 100%;
  padding: .5rem;
  margin: 0;
  list-style: none;
  border-radius: 1rem;
  background: rgba(15, 23, 42, .88);
  backdrop-filter: blur(18px);
  opacity: 0;
  transform: translateY(-8px);
  pointer-events: none;
  transition: .2s ease;
}

.custom-select.is-open .custom-select__options {
  opacity: 1;
  transform: translateY(0);
  pointer-events: auto;
}

.custom-select__option {
  padding: .75rem .9rem;
  border-radius: .8rem;
  cursor: pointer;
}

.custom-select__option:hover,
.custom-select__option.is-selected {
  background: linear-gradient(135deg, #2563eb, #7c3aed);
}
```

funciones.js - versión modular con export


```
export class CustomSelect {
  constructor(rootId) {
    this.root = document.getElementById(rootId);
    this.hiddenInput = this.root.querySelector("input[type='hidden']");
    this.button = this.root.querySelector(".custom-select__button");
    this.valueText = this.root.querySelector(".custom-select__value");
    this.options = this.root.querySelectorAll(".custom-select__option");
    this.init();
  }

  init() {
    this.button.addEventListener("click", () =>
      this.root.classList.toggle("is-open"));

    this.options.forEach((option) => {
      option.addEventListener("click", () => this.select(option));
    });

    document.addEventListener("click", (event) => {
      if (!this.root.contains(event.target))
        this.root.classList.remove("is-open");
    });
  }

  select(option) {
    this.hiddenInput.value = option.dataset.value;
    this.valueText.textContent = option.textContent;
    this.options.forEach((item) => item.classList.remove("is-selected"));
    option.classList.add("is-selected");
    this.root.classList.remove("is-open");
    this.hiddenInput.dispatchEvent(new Event("change"));
  }
}
```

script.js - importación e inicialización

```
import { CustomSelect } from "./funciones.js";

new CustomSelect("statusSelect");

// Para leer el valor:
const status = document.getElementById("statusValue").value;
```

script-normal.js - versión normal sin módulos

```
const customSelect = document.getElementById("statusSelect");
const customButton = customSelect.querySelector(".custom-select__button");
const customValue = customSelect.querySelector(".custom-select__value");
const hiddenInput = customSelect.querySelector("input[type='hidden']");
const customOptions = customSelect.querySelectorAll(".custom-select__option");

customButton.addEventListener("click", () =>
  customSelect.classList.toggle("is-open"));
customOptions.forEach((option) => {
  option.addEventListener("click", () => {
    hiddenInput.value = option.dataset.value;
    customValue.textContent = option.textContent;
    customSelect.classList.remove("is-open");
  });
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#statusSelect	Contenedor del selector.	Cámbialo por unidad, categoría, ciudad, etc.
#statusValue	Input oculto con el valor real.	Tu JS debe leer este value.
data-value	Valor lógico de cada opción.	Cámbialo según lo que tu app necesite.
.custom-select__value	Texto visible actual.	Debe coincidir con la opción inicial.

Paso a paso para reciclarlo en otro proyecto

1. Copia el bloque HTML completo del custom select.
2. Cambia el id del contenedor y del input hidden.
3. Agrega, elimina o edita `li.custom-select__option`.
4. Cada `li` debe tener `data-value` y texto visible.
5. Copia el CSS del selector.
6. Inicializa `new CustomSelect("idDelSelector")`.
7. Lee el valor desde el input hidden con `.value`.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como `is-active`, `is-open`, `is-valid` o `is-invalid` estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con `type="module"`.
- Que si usas la versión normal no dejes `imports` ni `exports` en el archivo.

Errores comunes

- Leer el texto del botón en vez del input oculto.
- Olvidar `data-value` en las opciones.
- No subir `z-index` y que el menú quede detrás de otra tarjeta.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: <code>loginForm</code> por <code>formAcceso</code> .
2	Busca ese mismo id en <code>script.js</code> o <code>script-normal.js</code> .	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay <code>data-*</code> como <code>data-tab</code> , <code>data-step</code> o <code>data-value</code> , revisa qué lo lee en JS.	Los atributos <code>data-*</code> suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Select personalizado dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca Select personalizado dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida ids, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init23()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Select personalizado.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.

4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Mantén un input hidden porque facilita leer el valor desde formularios y desde JavaScript.

24. Tabs / pestañas

¿Qué hace?

Permite cambiar entre paneles internos sin recargar la página. Se usa en perfiles, productos, dashboards, documentación y configuraciones.

¿Cuándo usarlo?

Úsalo cuando varias secciones están relacionadas pero no deben verse todas al mismo tiempo.

Estructura que debes copiar

HTML mínimo

```
<div class="tabs" id="productTabs">
  <div class="tabs__buttons">
    <button class="is-active" data-tab="description">Descripción</button>
    <button data-tab="details">Detalles</button>
    <button data-tab="reviews">Reseñas</button>
  </div>

  <section class="tab-panel is-active" id="description">Contenido de
    descripción</section>
  <section class="tab-panel" id="details">Contenido de detalles</section>
  <section class="tab-panel" id="reviews">Contenido de reseñas</section>
</div>
```

CSS necesario

```
.tabs__buttons {
  display: flex;
  flex-wrap: wrap;
  gap: .5rem;
  margin-bottom: 1rem;
}

.tabs__buttons button {
  width: auto;
  padding: .75rem 1rem;
  border-radius: 999px;
  background: rgba(255, 255, 255, .12);
  color: inherit;
}

.tabs__buttons button.is-active {
  background: linear-gradient(135deg, #2563eb, #7c3aed);
  color: #fff;
}

.tab-panel {
  display: none;
  padding: 1rem;
  border-radius: 1rem;
  background: rgba(255, 255, 255, .08);
}

.tab-panel.is-active { display: block; }
```

funciones.js - versión modular con export

```
export class Tabs {
  constructor(rootId) {
    this.root = document.getElementById(rootId);
    this.buttons = this.root.querySelectorAll("[data-tab]");
    this.panels = this.root.querySelectorAll(".tab-panel");
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.activate(button.dataset.tab));
    });
  }

  activate(tabId) {
    this.buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.tab === tabId);
    });

    this.panels.forEach((panel) => {
      panel.classList.toggle("is-active", panel.id === tabId);
    });
  }
}
```

script.js - importación e inicialización

```
import { Tabs } from "../funciones.js";

new Tabs("productTabs");
```

script-normal.js - versión normal sin módulos

```
const tabs = document.getElementById("productTabs");
const tabButtons = tabs.querySelectorAll("[data-tab]");
const tabPanels = tabs.querySelectorAll(".tab-panel");

tabButtons.forEach((button) => {
  button.addEventListener("click", () => {
    const tabId = button.dataset.tab;
    tabButtons.forEach((btn) => btn.classList.toggle("is-active", btn ===
      button));
    tabPanels.forEach((panel) => panel.classList.toggle("is-active", panel.id ===
      tabId));
  });
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
data-tab	Nombre del panel que se activará.	Debe coincidir con el id del panel.
.is-active	Clase que muestra botón y panel activos.	No cambies el nombre si no ajustas JS/CSS.
#productTabs	Contenedor raíz.	Permite tener varias zonas de tabs independientes.

Paso a paso para reciclarlo en otro proyecto

1. Copia la estructura .tabs completa.
2. Cambia textos de botones y contenido de paneles.
3. Asegúrate de que data-tab="x" coincida con id="x".
4. Copia los estilos de botones y paneles.
5. Inicializa new Tabs("productTabs").
6. Prueba todos los botones y verifica que solo haya un panel activo.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- data-tab no coincide con ningún id.
- No poner is-active inicial y que no se vea nada.
- Usar ids repetidos en varias secciones de tabs.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Tabs / pestañas dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Tabs / pestañas dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init24() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Tabs / pestañas.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Si las tabs cambian contenido importante, considera actualizar la URL con hash para compartir el estado.

25. Acordeón / FAQ

¿Qué hace?

Abre y cierra bloques de contenido. Es perfecto para preguntas frecuentes, secciones de ayuda, detalles de producto y menús informativos.

¿Cuándo usarlo?

Úsalo cuando tienes mucho contenido textual y quieres ahorrar espacio visual.

Estructura que debes copiar

HTML mínimo

```
<section class="accordion" id="faqAccordion">
  <article class="accordion_item">
    <button type="button" class="accordion_button">¿Qué incluye el
      servicio?</button>
    <div class="accordion_content">
      <p>Incluye diseño, desarrollo y soporte inicial.</p>
    </div>
  </article>

  <article class="accordion_item">
    <button type="button" class="accordion_button">¿Cuánto tarda?</button>
    <div class="accordion_content">
      <p>Depende del alcance del proyecto.</p>
    </div>
  </article>
</section>
```

CSS necesario

```
.accordion_item {
  border-bottom: 1px solid rgba(255, 255, 255, .15);
}

.accordion_button {
  width: 100%;
  padding: 1rem;
  border: 0;
  background: transparent;
  color: inherit;
  text-align: left;
  font-weight: 700;
  cursor: pointer;
}

.accordion_content {
  max-height: 0;
  overflow: hidden;
  transition: max-height .25s ease;
}

.accordion_item.is-open .accordion_content {
  max-height: 220px;
}

.accordion_content p {
  padding: 0 1rem 1rem;
  margin: 0;
  color: var(--text-muted, #cbd5e1);
}
```

funciones.js - versión modular con export


```
export class Accordion {
  constructor(rootId, allowMultiple = false) {
    this.root = document.getElementById(rootId);
    this.items = this.root.querySelectorAll(".accordion__item");
    this.allowMultiple = allowMultiple;
    this.init();
  }

  init() {
    this.items.forEach((item) => {
      const button = item.querySelector(".accordion__button");
      button.addEventListener("click", () => this.toggle(item));
    });
  }

  toggle(selectedItem) {
    if (!this.allowMultiple) {
      this.items.forEach((item) => {
        if (item !== selectedItem) item.classList.remove("is-open");
      });
    }

    selectedItem.classList.toggle("is-open");
  }
}
```

script.js - importación e inicialización

```
import { Accordion } from "./funciones.js";

new Accordion("faqAccordion", false);
```

script-normal.js - versión normal sin módulos

```
const faqAccordion = document.getElementById("faqAccordion");
const accordionItems = faqAccordion.querySelectorAll(".accordion__item");

accordionItems.forEach((item) => {
  const button = item.querySelector(".accordion__button");
  button.addEventListener("click", () => {
    accordionItems.forEach((otherItem) => {
      if (otherItem !== item) otherItem.classList.remove("is-open");
    });
    item.classList.toggle("is-open");
  });
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#faqAccordion	Contenedor del acordeón.	Cambia el id si tienes varios acordeones.
.accordion__item	Cada bloque desplegable.	Duplica este article para más preguntas.
allowMultiple	Permite abrir varios bloques a la vez.	true abre varios; false solo uno.
max-height	Altura máxima del contenido abierto.	Auméntala si el contenido es largo.

Paso a paso para reciclarlo en otro proyecto

1. Copia el section.accordion completo.
2. Duplica article.accordion__item para cada pregunta o bloque.
3. Cambia el texto del botón y del contenido.
4. Copia el CSS y ajusta max-height si hace falta.
5. Inicializa new Accordion("faqAccordion", false).
6. Prueba abrir una pregunta y confirmar que las demás se cierran.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Poner contenido muy largo y que se corte por max-height.
- No usar button y perder accesibilidad.
- Olvidar que allowMultiple cambia el comportamiento.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Acordeón / FAQ dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Acordeón / FAQ dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init25() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Acordeón / FAQ.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Para accesibilidad avanzada, puedes agregar aria-expanded al botón y actualizarlo con JavaScript.

Apéndice: mapa rápido de integración

#	Funcionalidad	Archivos	Versión JS	Revisión clave
10	Contador animado	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
11	Login demo	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
12	Logout demo	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
13	Recordar usuario	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
14	Password toggle	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
15	Validación de formulario	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
16	Mensajes por campo	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
17	Character counter	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
18	Range preview	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
19	File info	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
20	Autosave draft	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
21	Stepper / Wizard	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
22	Skeleton loader	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
23	Select personalizado	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
24	Tabs / pestañas	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
25	Acordeón / FAQ	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*

Checklist final antes de copiar a otro proyecto

- Copiar solo la funcionalidad necesaria para no llenar el proyecto de código que no vas a usar.
- Mantener una sola estrategia de JavaScript: modular o normal.
- Si usas módulos, index.html debe tener script type="module".
- Si usas script normal, elimina export, import y clases que no inicialices.
- Revisar en móvil, tablet y escritorio.
- Probar estados vacíos, errores, valores válidos y valores extremos.
- Cambiar nombres de localStorage para evitar choques entre proyectos.
- Usar nombres claros para ids y clases: loginForm, productTabs, faqAccordion, etc.
- Comentar el código si lo vas a entregar como tarea o compartir con el equipo.

Actualizacion funcional del bloque 11 a 25

Este anexo actualiza los codigos de copiar y pegar para que queden alineados con la pagina actual y funcionen como referencia implementable.

- Usa estos bloques como fuente vigente para HTML, CSS y JavaScript.
- El contenido anterior del PDF base se conserva, pero este anexo manda para implementacion.

11. Login demo

Simula un inicio de sesión local para pruebas de desarrollo frontend.

HTML copiar y pegar

```
<form id="loginForm" class="demo-form" novalidate>
  <label for="loginEmail">Correo</label>
  <input id="loginEmail" type="email" placeholder="usuario@correo.com" required>

  <label for="loginPassword">Contraseña</label>
  <input id="loginPassword" type="password" placeholder="Minimo 6 caracteres"
required minlength="6">

  <label class="check-line">
    <input type="checkbox" id="rememberUser">
    Recordar usuario
  </label>

  <button type="submit">Ingresar</button>
  <button id="logoutButton" type="button">Cerrar sesion</button>
</form>
<p id="sessionStatus">Aun no hay sesion iniciada.</p>
<div id="toastContainer"></div>
```

CSS copiar y pegar

```
.demo-form {
  display: grid;
  gap: 10px;
  max-width: 420px;
}
.demo-form input,
.demo-form button {
  padding: 10px 12px;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
}
.check-line {
  display: flex;
  gap: 8px;
  align-items: center;
}
#toastContainer {
  position: fixed;
  right: 20px;
  bottom: 20px;
  display: grid;
  gap: 10px;
}
.toast {
  padding: 12px 14px;
  border-radius: 12px;
  background: #0f172a;
  color: #ffffff;
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

export class LoginDemo {
  constructor(toast) {
    this.form = $("#loginForm");
    this.status = $("#sessionStatus");
    this.logout = $("#logoutButton");
    this.remember = $("#rememberUser");
    this.toast = toast;
    this.init();
  }

  init() {
    this.renderStatus();
    this.form.addEventListener("submit", (event) => this.login(event));
    this.logout.addEventListener("click", () => this.logoutUser());
  }

  login(event) {
    event.preventDefault();
    const email = $("#loginEmail").value.trim();
    const password = $("#loginPassword").value.trim();
    if (!email.includes("@") || password.length < 6) {
      this.toast.show("Revisa correo y contrase a.");
      return;
    }
    const user = { email, date: new Date().toLocaleString() };
    if (this.remember.checked) saveJSON("kit-user", user);
    sessionStorage.setItem("kit-session", JSON.stringify(user));
    this.renderStatus(user);
    this.toast.show("Sesion iniciada correctamente.");
  }
}

```

```

}

logoutUser() {
  sessionStorage.removeItem("kit-session");
  localStorage.removeItem("kit-user");
  this.renderStatus(null);
  this.toast.show("Sesion cerrada.");
}

renderStatus(user = null) {
  const current = user || readJSON("kit-user") ||
JSON.parse(sessionStorage.getItem("kit-session") || "null");
  this.status.textContent = current ? 'Usuario activo: ${current.email}.
Ingreso: ${current.date}' : "Aun no hay sesion iniciada.";
}
}

document.addEventListener("DOMContentLoaded", () => {
  const toast = new ToastManager();
  new LoginDemo(toast);
});

```

JS normal copiar y pegar

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

```

```

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

```

```

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

```

```

class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

```

```

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

```

```

class LoginDemo {
  constructor(toast) {
    this.form = $("#loginForm");
    this.status = $("#sessionStatus");
    this.logout = $("#logoutButton");
  }
}

```



```

    this.remember = $("#rememberUser");
    this.toast = toast;
    this.init();
}

init() {
    this.renderStatus();
    this.form.addEventListener("submit", (event) => this.login(event));
    this.logout.addEventListener("click", () => this.logoutUser());
}

login(event) {
    event.preventDefault();
    const email = $("#loginEmail").value.trim();
    const password = $("#loginPassword").value.trim();
    if (!email.includes("@") || password.length < 6) {
        this.toast.show("Revisa correo y contraseña.");
        return;
    }
    const user = { email, date: new Date().toLocaleString() };
    if (this.remember.checked) saveJSON("kit-user", user);
    sessionStorage.setItem("kit-session", JSON.stringify(user));
    this.renderStatus(user);
    this.toast.show("Sesion iniciada correctamente.");
}

logoutUser() {
    sessionStorage.removeItem("kit-session");
    localStorage.removeItem("kit-user");
    this.renderStatus(null);
    this.toast.show("Sesion cerrada.");
}

renderStatus(user = null) {
    const current = user || readJSON("kit-user") ||
JSON.parse(sessionStorage.getItem("kit-session") || "null");
    this.status.textContent = current ? 'Usuario activo: ${current.email}.
Ingreso: ${current.date}' : "Aun no hay sesion iniciada.";
}
}

document.addEventListener("DOMContentLoaded", () => {
    const toast = new ToastManager();
    new LoginDemo(toast);
});

```

12. Logout demo

Borra el token o indicador de sesi n simulada del almacenamiento local.

HTML copiar y pegar

```
<button id="logoutButton" type="button">Cerrar sesi n</button>
<p id="sessionStatus">Sesi n simulada activa.</p>
```

CSS copiar y pegar

```
#logoutButton {
  padding: 10px 14px;
  border: 0;
  border-radius: 999px;
  background: #ef4444;
  color: #ffffff;
  cursor: pointer;
}
```

JS modular copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {
  const button = document.getElementById("logoutButton");
  const status = document.getElementById("sessionStatus");
  sessionStorage.setItem("kit-session", JSON.stringify({ active: true }));

  button?.addEventListener("click", () => {
    sessionStorage.removeItem("kit-session");
    localStorage.removeItem("kit-user");
    if (status) status.textContent = "Sesi n cerrada correctamente.";
  });
});
```

JS normal copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {
  const button = document.getElementById("logoutButton");
  const status = document.getElementById("sessionStatus");
  sessionStorage.setItem("kit-session", JSON.stringify({ active: true }));

  button?.addEventListener("click", () => {
    sessionStorage.removeItem("kit-session");
    localStorage.removeItem("kit-user");
    if (status) status.textContent = "Sesi n cerrada correctamente.";
  });
});
```

13. Recordar usuario

Guarda el correo de login en localStorage para recordar las credenciales del usuario.

HTML copiar y pegar

```
<form id="rememberForm" class="demo-form">
  <input id="rememberEmail" type="email" placeholder="usuario@correo.com">
  <label class="check-line">
    <input type="checkbox" id="rememberUser">
    Recordar usuario
  </label>
  <button type="submit">Guardar preferencia</button>
</form>
<p id="rememberStatus">Sin datos guardados.</p>
```

CSS copiar y pegar

```
.demo-form {
  display: grid;
  gap: 10px;
  max-width: 420px;
}
.demo-form input,
.demo-form button {
  padding: 10px 12px;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
}
.check-line {
  display: flex;
  gap: 8px;
  align-items: center;
}
```

JS modular copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {
  const form = document.getElementById("rememberForm");
  const email = document.getElementById("rememberEmail");
  const remember = document.getElementById("rememberUser");
  const status = document.getElementById("rememberStatus");
  const saved = localStorage.getItem("remember-email");

  if (saved && email && remember) {
    email.value = saved;
    remember.checked = true;
    status.textContent = 'Usuario recordado: ${saved}';
  }

  form?.addEventListener("submit", (event) => {
    event.preventDefault();
    if (!email || !remember || !status) return;
    if (remember.checked && email.value.trim()) {
      localStorage.setItem("remember-email", email.value.trim());
      status.textContent = 'Usuario recordado: ${email.value.trim()}';
    } else {
      localStorage.removeItem("remember-email");
      status.textContent = "Recordatorio eliminado.";
    }
  });
});
```

```
});
```

JS normal copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {  
  const form = document.getElementById("rememberForm");  
  const email = document.getElementById("rememberEmail");  
  const remember = document.getElementById("rememberUser");  
  const status = document.getElementById("rememberStatus");  
  const saved = localStorage.getItem("remember-email");  
  
  if (saved && email && remember) {  
    email.value = saved;  
    remember.checked = true;  
    status.textContent = 'Usuario recordado: ${saved}';  
  }  
  
  form?.addEventListener("submit", (event) => {  
    event.preventDefault();  
    if (!email || !remember || !status) return;  
    if (remember.checked && email.value.trim()) {  
      localStorage.setItem("remember-email", email.value.trim());  
      status.textContent = 'Usuario recordado: ${email.value.trim()}';  
    } else {  
      localStorage.removeItem("remember-email");  
      status.textContent = "Recordatorio eliminado.";  
    }  
  });  
});
```

14. Password toggle

Botón integrado que alterna el atributo de entrada de tipo 'password' a 'text'.

HTML copiar y pegar

```
<input id="passwordInput" type="password">
<button data-toggle-password="passwordInput">Ver</button>
```

CSS copiar y pegar

```
.input-action { display: flex; gap: 8px; }
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
export class PasswordToggle {
  constructor() {
    this.buttons = $$('[data-toggle-password]');
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => {
        const input =
document.getElementById(button.dataset.togglePassword);
        input.type = input.type === "password" ? "text" : "password";
        button.textContent = input.type === "password" ? "Ver" : "Ocultar";
      });
    });
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new PasswordToggle();
});
```

JS normal copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class PasswordToggle {
  constructor() {
    this.buttons = $$('[data-toggle-password]');
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => {
        const input =
document.getElementById(button.dataset.togglePassword);
        input.type = input.type === "password" ? "text" : "password";
        button.textContent = input.type === "password" ? "Ver" : "Ocultar";
      });
    });
  }
}
```

```
document.addEventListener("DOMContentLoaded", () => {  
  new PasswordToggle();  
});
```

15. Validaci3n formularios

Comprueba campos vac3os, correos correctos y longitudes m3nimas en el cliente.

HTML copiar y pegar

```
<form id="contactForm" novalidate class="demo-form">
  <input id="contactName" type="text" placeholder="Nombre" required minlength="3">
  <small class="field-error" data-error="contactName"></small>

  <input id="contactEmail" type="email" placeholder="Correo" required>
  <small class="field-error" data-error="contactEmail"></small>

  <textarea id="contactMessage" placeholder="Mensaje" required
minlength="10"></textarea>
  <small class="field-error" data-error="contactMessage"></small>

  <button type="submit">Enviar</button>
</form>
<div id="toastContainer"></div>
```

CSS copiar y pegar

```
.demo-form {
  display: grid;
  gap: 10px;
  max-width: 420px;
}
.demo-form input,
.demo-form textarea,
.demo-form button {
  padding: 10px 12px;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
}
.field-error {
  min-height: 18px;
  color: #ef4444;
  font-size: 0.875rem;
}
.toast {
  padding: 12px 14px;
  border-radius: 12px;
  background: #0f172a;
  color: #ffffff;
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
```

```

        this.container.appendChild(toast);
        setTimeout(() => toast.remove(), time);
    }
}

export class FormValidation {
    constructor(formId, toast) {
        this.form = document.getElementById(formId);
        this.toast = toast;
        if (this.form) this.init();
    }

    init() {
        this.form.addEventListener("submit", (event) => {
            event.preventDefault();
            const valid = this.validate();
            this.toast.show(valid ? "Formulario enviado correctamente." : "Completa
los campos requeridos.");
        });
    }

    validate() {
        let isValid = true;
        $$('input[required], textarea[required], select[required]',
this.form).forEach((input) => {
            const error = document.querySelector("[data-error=\"${input.id}\"]");
            let message = "";
            if (!input.value.trim()) message = "Este campo es obligatorio.";
            if (input.type === "email" && input.value && !input.value.includes("@"))
message = "Correo no valido.";
            if (input.minLength > 0 && input.value.length < input.minLength) message
= 'Minimo ${input.minLength} caracteres.';
            if (error) error.textContent = message;
            input.style.borderColor = message ? "#ef4444" : "#22c55e";
            if (message) isValid = false;
        });
        return isValid;
    }
}

document.addEventListener("DOMContentLoaded", () => {
    const toast = new ToastManager();
    new FormValidation("contactForm", toast);
});

```

JS normal copiar y pegar

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class ToastManager {
    constructor(containerId = "toastContainer") {
        this.container = document.getElementById(containerId);
    }

    show(message, time = 2800) {
        const toast = document.createElement("div");
        toast.className = "toast";
        toast.textContent = message;
    }
}

```



```

        this.container.appendChild(toast);
        setTimeout(() => toast.remove(), time);
    }
}

class FormValidation {
    constructor(formId, toast) {
        this.form = document.getElementById(formId);
        this.toast = toast;
        if (this.form) this.init();
    }

    init() {
        this.form.addEventListener("submit", (event) => {
            event.preventDefault();
            const valid = this.validate();
            this.toast.show(valid ? "Formulario enviado correctamente." : "Completa
los campos requeridos.");
        });
    }

    validate() {
        let isValid = true;
        $$('input[required], textarea[required], select[required]',
this.form).forEach((input) => {
            const error = document.querySelector("[data-error=\"${input.id}\"]");
            let message = "";
            if (!input.value.trim()) message = "Este campo es obligatorio.";
            if (input.type === "email" && input.value && !input.value.includes("@"))
message = "Correo no valido.";
            if (input.minLength > 0 && input.value.length < input.minLength) message
= 'Minimo ${input.minLength} caracteres.';
            if (error) error.textContent = message;
            input.style.borderColor = message ? "#ef4444" : "#22c55e";
            if (message) isValid = false;
        });
        return isValid;
    }
}

document.addEventListener("DOMContentLoaded", () => {
    const toast = new ToastManager();
    new FormValidation("contactForm", toast);
});

```

16. Mensajes de error

Mensajes descriptivos por campo que aparecen de forma dinámica si hay error.

HTML copiar y pegar

```
<input id="emailInput" required>  
<small class="field-error" data-error="emailInput"></small>
```

CSS copiar y pegar

```
.field-error { color: #ef4444; min-height: 18px; }
```

JS modular copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {  
  const form = document.getElementById("contactForm");  
  const email = document.getElementById("emailInput");  
  const error = document.querySelector("[data-error="emailInput"]");  
  
  form?.addEventListener("submit", (event) => {  
    event.preventDefault();  
    if (!email || !error) return;  
    error.textContent = email.value.includes("@") ? "" : "Correo no valido.";  
  });  
});
```

JS normal copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {  
  const form = document.getElementById("contactForm");  
  const email = document.getElementById("emailInput");  
  const error = document.querySelector("[data-error="emailInput"]");  
  
  form?.addEventListener("submit", (event) => {  
    event.preventDefault();  
    if (!email || !error) return;  
    error.textContent = email.value.includes("@") ? "" : "Correo no valido.";  
  });  
});
```

17. Character counter

Contador de caracteres dinámico para áreas de texto con límite de caracteres.

HTML copiar y pegar

```
<textarea id="msg" maxlength="160"></textarea>
<span><span id="counter">0</span>/160</span>
```

CSS copiar y pegar

```
#msg {
  width: 100%;
  min-height: 120px;
  padding: 12px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
}
#counter {
  font-weight: 700;
  color: #2563eb;
}
```

JS modular copiar y pegar

```
export class CharacterCounter {
  constructor(inputId, counterId) {
    this.input = document.getElementById(inputId);
    this.counter = document.getElementById(counterId);
    this.init();
  }

  init() {
    this.input.addEventListener("input", () => this.counter.textContent =
this.input.value.length);
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new CharacterCounter("msg", "counter");
});
```

JS normal copiar y pegar

```
class CharacterCounter {
  constructor(inputId, counterId) {
    this.input = document.getElementById(inputId);
    this.counter = document.getElementById(counterId);
    this.init();
  }

  init() {
    this.input.addEventListener("input", () => this.counter.textContent =
this.input.value.length);
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new CharacterCounter("msg", "counter");
});
```

18. Range preview

Visualiza el valor dinámico actual de un selector deslizante de tipo rango.

HTML copiar y pegar

```
<input type="range" id="range" min="0" max="100">
<span id="rangeVal">50</span>
```

CSS copiar y pegar

```
#range {
  width: 100%;
  accent-color: #2563eb;
}
#rangeVal {
  display: inline-block;
  min-width: 40px;
  font-weight: 700;
}
```

JS modular copiar y pegar

```
export class RangePreview {
  constructor(inputId, valueId) {
    this.input = document.getElementById(inputId);
    this.value = document.getElementById(valueId);
    this.init();
  }

  init() {
    this.input.addEventListener("input", () => this.value.textContent =
this.input.value);
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new RangePreview("range", "rangeVal");
});
```

JS normal copiar y pegar

```
class RangePreview {
  constructor(inputId, valueId) {
    this.input = document.getElementById(inputId);
    this.value = document.getElementById(valueId);
    this.init();
  }

  init() {
    this.input.addEventListener("input", () => this.value.textContent =
this.input.value);
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new RangePreview("range", "rangeVal");
});
```

19. File info

Muestra el nombre del archivo subido y su peso en kilobytes para guiar al usuario.

HTML copiar y pegar

```
<input type="file" id="file">
<p id="fileInfo">No hay archivo seleccionado.</p>
```

CSS copiar y pegar

```
#file {
  display: block;
  margin-bottom: 10px;
}
#fileInfo {
  color: #475569;
  font-size: 0.95rem;
}
```

JS modular copiar y pegar

```
export class FileInfo {
  constructor(inputId, infoId) {
    this.input = document.getElementById(inputId);
    this.info = document.getElementById(infoId);
    this.init();
  }

  init() {
    this.input.addEventListener("change", () => {
      const file = this.input.files[0];
      this.info.textContent = file ? `${file.name} - ${(file.size /
1024).toFixed(1)} KB` : "No hay archivo seleccionado.";
    });
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new FileInfo("file", "fileInfo");
});
```

JS normal copiar y pegar

```
class FileInfo {
  constructor(inputId, infoId) {
    this.input = document.getElementById(inputId);
    this.info = document.getElementById(infoId);
    this.init();
  }

  init() {
    this.input.addEventListener("change", () => {
      const file = this.input.files[0];
      this.info.textContent = file ? `${file.name} - ${(file.size /
1024).toFixed(1)} KB` : "No hay archivo seleccionado.";
    });
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new FileInfo("file", "fileInfo");
});
```

20. Autosave draft

Guarda de forma automática en localStorage el borrador de un campo de texto.

HTML copiar y pegar

```
<textarea id="msg" rows="4" placeholder="Escribe tu borrador"></textarea>
<button id="clearDraft" type="button">Limpiar borrador</button>
<div id="toastContainer"></div>
```

CSS copiar y pegar

```
#msg {
  width: 100%;
  min-height: 120px;
  padding: 12px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
}
#clearDraft {
  margin-top: 10px;
  padding: 10px 14px;
  border: 0;
  border-radius: 999px;
  background: #ef4444;
  color: #ffffff;
}
```

JS modular copiar y pegar

```
function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

export class AutoSaveDraft {
  constructor(inputId, clearId, key, toast) {
    this.input = document.getElementById(inputId);
    this.clearButton = document.getElementById(clearId);
    this.key = key;
    this.toast = toast;
    this.init();
  }
}
```

```

init() {
  this.input.value = localStorage.getItem(this.key) || "";
  this.input.dispatchEvent(new Event("input"));
  this.input.addEventListener("input", debounce(() =>
localStorage.setItem(this.key, this.input.value), 250));
  this.clearButton.addEventListener("click", () => {
    localStorage.removeItem(this.key);
    this.input.value = "";
    this.input.dispatchEvent(new Event("input"));
    this.toast.show("Borrador eliminado.");
  });
}
}

document.addEventListener("DOMContentLoaded", () => {
  const toast = new ToastManager();
  new AutoSaveDraft("msg", "clearDraft", "draft-demo", toast);
});

```

JS normal copiar y pegar

```

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

class AutoSaveDraft {
  constructor(inputId, clearId, key, toast) {
    this.input = document.getElementById(inputId);
    this.clearButton = document.getElementById(clearId);
    this.key = key;
    this.toast = toast;
    this.init();
  }

  init() {
    this.input.value = localStorage.getItem(this.key) || "";
    this.input.dispatchEvent(new Event("input"));
    this.input.addEventListener("input", debounce(() =>
localStorage.setItem(this.key, this.input.value), 250));
    this.clearButton.addEventListener("click", () => {
      localStorage.removeItem(this.key);

```

```
        this.input.value = "";
        this.input.dispatchEvent(new Event("input"));
        this.toast.show("Borrador eliminado.");
    });
}
}

document.addEventListener("DOMContentLoaded", () => {
    const toast = new ToastManager();
    new AutoSaveDraft("msg", "clearDraft", "draft-demo", toast);
});
```


21. Stepper

Gu a al usuario en un proceso de varios pasos mostrando el paso activo.

HTML copiar y pegar

```
<div class="stepper" data-stepper>
  <div class="stepper__steps">
    <span class="is-active">1</span>
    <span>2</span>
    <span>3</span>
  </div>
  <p class="stepper__text">Paso 1: Datos personales</p>
  <button data-step-prev type="button">Anterior</button>
  <button data-step-next type="button">Siguiente</button>
</div>
```

CSS copiar y pegar

```
.stepper { display: grid; gap: 12px; }
.stepper__steps {
  display: flex;
  gap: 10px;
}
.stepper__steps span {
  width: 34px;
  height: 34px;
  display: grid;
  place-items: center;
  border-radius: 50%;
  background: #e2e8f0;
}
.stepper__steps span.is-active {
  background: #2563eb;
  color: #ffffff;
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
export class Stepper {
  constructor(selector = "[data-stepper]") {
    this.stepper = $(selector);
    this.index = 0;
    this.texts = ["Paso 1: Datos personales", "Paso 2: Preferencias", "Paso 3:
Confirmacion"];
    this.init();
  }
}
```

```
  init() {
    $('[data-step-next]', this.stepper).addEventListener("click", () =>
this.move(1));
    $('[data-step-prev]', this.stepper).addEventListener("click", () =>
this.move(-1));
    this.render();
  }
```

```
  move(direction) {
    this.index = Math.min(2, Math.max(0, this.index + direction));
```

```

        this.render();
    }

    render() {
        $$(".stepper__steps span", this.stepper).forEach((step, i) =>
step.classList.toggle("is-active", i <= this.index));
        $$(".stepper__text", this.stepper).textContent = this.texts[this.index];
    }
}

document.addEventListener("DOMContentLoaded", () => {
    new Stepper();
});

```

JS normal copiar y pegar

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class Stepper {
    constructor(selector = "[data-stepper]") {
        this.stepper = $(selector);
        this.index = 0;
        this.texts = ["Paso 1: Datos personales", "Paso 2: Preferencias", "Paso 3:
Confirmacion"];
        this.init();
    }

    init() {
        $("[data-step-next]", this.stepper).addEventListener("click", () =>
this.move(1));
        $("[data-step-prev]", this.stepper).addEventListener("click", () =>
this.move(-1));
        this.render();
    }

    move(direction) {
        this.index = Math.min(2, Math.max(0, this.index + direction));
        this.render();
    }

    render() {
        $$(".stepper__steps span", this.stepper).forEach((step, i) =>
step.classList.toggle("is-active", i <= this.index));
        $$(".stepper__text", this.stepper).textContent = this.texts[this.index];
    }
}

document.addEventListener("DOMContentLoaded", () => {
    new Stepper();
});

```

22. Skeleton loader

Muestra bloques grises de carga animados mientras se descargan datos reales.

HTML copiar y pegar

```
<button id="loadSkeleton" type="button">Simular carga</button>
<div id="skeletonArea"></div>
<div id="toastContainer"></div>
```

CSS copiar y pegar

```
.skeleton {
  height: 18px;
  margin-top: 12px;
  border-radius: 999px;
  background: linear-gradient(90deg, #e2e8f0, #f8fafc, #e2e8f0);
  background-size: 200% 100%;
  animation: loading 1.2s infinite;
}
@keyframes loading {
  to { background-position: -200% 0; }
}
```

JS modular copiar y pegar

```
class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }
```

```
  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}
```

```
export class SkeletonLoader {
  constructor(buttonId, areaId, toast) {
    this.button = document.getElementById(buttonId);
    this.area = document.getElementById(areaId);
    this.toast = toast;
    this.init();
  }
```

```
  init() {
    this.button.addEventListener("click", () => {
      this.area.innerHTML = '<div class="skeleton"></div><div
class="skeleton"></div><div class="skeleton"></div>';
      setTimeout(() => {
        this.area.innerHTML = '<p class="muted">Contenido cargado
correctamente.</p>';
        this.toast.show("Carga simulada finalizada.");
      }, 1200);
    });
  }
}
```

```
document.addEventListener("DOMContentLoaded", () => {
  const toast = new ToastManager();
  new SkeletonLoader("loadSkeleton", "skeletonArea", toast);
});
```

JS normal copiar y pegar

```
class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }
```

```
  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}
```

```
class SkeletonLoader {
  constructor(buttonId, areaId, toast) {
    this.button = document.getElementById(buttonId);
    this.area = document.getElementById(areaId);
    this.toast = toast;
    this.init();
  }
```

```
  init() {
    this.button.addEventListener("click", () => {
      this.area.innerHTML = '<div class="skeleton"></div><div
class="skeleton"></div><div class="skeleton"></div>';
      setTimeout(() => {
        this.area.innerHTML = '<p class="muted">Contenido cargado
correctamente.</p>';
        this.toast.show("Carga simulada finalizada.");
      }, 1200);
    });
  }
}
```

```
document.addEventListener("DOMContentLoaded", () => {
  const toast = new ToastManager();
  new SkeletonLoader("loadSkeleton", "skeletonArea", toast);
});
```

23. Select personalizado

Reemplaza el select del navegador con una lista estilizable y adaptada.

HTML copiar y pegar

```
<div class="custom-select" data-custom-select>
  <input type="hidden" id="selectProd" value="apa">
  <button type="button" class="custom-select__button">
    <span class="custom-select__value">APA artesanal</span>
    <span class="custom-select__arrow">&#9633</span>
  </button>
  <ul class="custom-select__options">
    <li class="custom-select__option is-selected" data-value="apa">APA</li>
    <li class="custom-select__option" data-value="stout">Stout</li>
  </ul>
</div>
```

CSS copiar y pegar

```
.custom-select {
  position: relative;
  max-width: 260px;
}
.custom-select__button {
  width: 100%;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px 12px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  background: #ffffff;
  cursor: pointer;
}
.custom-select__options {
  position: absolute;
  top: calc(100% + 8px);
  left: 0;
  width: 100%;
  margin: 0;
  padding: 8px;
  list-style: none;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  background: #ffffff;
  display: none;
}
.custom-select.is-open .custom-select__options {
  display: grid;
  gap: 6px;
}
.custom-select__option {
  padding: 10px 12px;
  border-radius: 10px;
  cursor: pointer;
}
.custom-select__option.is-selected {
  background: #dbeafe;
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export class CustomSelect {
  constructor(selector = "[data-custom-select]", onChange = null) {
    this.selects = $$(selector);
    this.onChange = onChange;
    this.init();
  }

  init() {
    this.selects.forEach((select) => {
      const button = $(".custom-select__button", select);
      const valueText = $(".custom-select__value", select);
      const hiddenInput = $("input[type='hidden']", select);
      const options = $$(".custom-select__option", select);

      button.addEventListener("click", (event) => {
        event.stopPropagation();
        this.closeAll(select);
        select.classList.toggle("is-open");
      });

      options.forEach((option) => {
        option.addEventListener("click", () => {
          hiddenInput.value = option.dataset.value;
          valueText.textContent = option.textContent;
          options.forEach((item) => item.classList.remove("is-selected"));
          option.classList.add("is-selected");
          select.classList.remove("is-open");
          this.onChange?.(hiddenInput.value, select);
          document.dispatchEvent(new CustomEvent("custom-select:change", {
            detail: { value: hiddenInput.value, inputId: hiddenInput.id } }));
        });
      });

      document.addEventListener("click", () => this.closeAll());
    });

    closeAll(except = null) {
      this.selects.forEach((select) => {
        if (select !== except) select.classList.remove("is-open");
      });
    }
  }

  document.addEventListener("DOMContentLoaded", () => {
    new CustomSelect();
  });
}
```

JS normal copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class CustomSelect {
```

```

constructor(selector = "[data-custom-select]", onChange = null) {
  this.selects = $$$(selector);
  this.onChange = onChange;
  this.init();
}

init() {
  this.selects.forEach((select) => {
    const button = $$(".custom-select__button", select);
    const valueText = $$(".custom-select__value", select);
    const hiddenInput = $$(".input[type='hidden']", select);
    const options = $$(".custom-select__option", select);

    button.addEventListener("click", (event) => {
      event.stopPropagation();
      this.closeAll(select);
      select.classList.toggle("is-open");
    });

    options.forEach((option) => {
      option.addEventListener("click", () => {
        hiddenInput.value = option.dataset.value;
        valueText.textContent = option.textContent;
        options.forEach((item) => item.classList.remove("is-selected"));
        option.classList.add("is-selected");
        select.classList.remove("is-open");
        this.onChange?.(hiddenInput.value, select);
        document.dispatchEvent(new CustomEvent("custom-select:change", {
          detail: { value: hiddenInput.value, inputId: hiddenInput.id } }));
      });
    });
  });

  document.addEventListener("click", () => this.closeAll());
}

closeAll(except = null) {
  this.selects.forEach((select) => {
    if (select !== except) select.classList.remove("is-open");
  });
}

document.addEventListener("DOMContentLoaded", () => {
  new CustomSelect();
});

```

24. Tabs

Agrupar informaci3n diferente en la misma 1rea alternando pesta1as activas.

HTML copiar y pegar

```
<div class="tabs" data-tabs>
  <div class="tabs__buttons">
    <button type="button" data-tab="panel1" class="is-active">HTML</button>
    <button type="button" data-tab="panel2">CSS</button>
  </div>
  <div class="tab-panel is-active" id="panel1">Contenido HTML</div>
  <div class="tab-panel" id="panel2">Contenido CSS</div>
</div>
```

CSS copiar y pegar

```
.tabs__buttons {
  display: flex;
  gap: 8px;
  margin-bottom: 12px;
}
.tabs__buttons button {
  padding: 10px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 999px;
  background: #ffffff;
  cursor: pointer;
}
.tabs__buttons button.is-active {
  background: #2563eb;
  color: #ffffff;
  border-color: #2563eb;
}
.tab-panel { display: none; }
.tab-panel.is-active { display: block; }
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
export class Tabs {
  constructor(selector = "[data-tabs]") {
    this.components = $$ (selector);
    this.init();
  }
}
```

```
init() {
  this.components.forEach((tabs) => {
    const buttons = $$("[data-tab]", tabs);
    const panels = $$(".tab-panel", tabs);
    buttons.forEach((button) => {
      button.addEventListener("click", () => {
        buttons.forEach((item) => item.classList.remove("is-active"));
        panels.forEach((panel) => panel.classList.remove("is-active"));
        button.classList.add("is-active");
        document.getElementById(button.dataset.tab)?.classList.add("is-
active");
      });
    });
  });
}
```



```

    });
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new Tabs();
});

```

JS normal copiar y pegar

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

```

```

class Tabs {
  constructor(selector = "[data-tabs]") {
    this.components = $$ (selector);
    this.init();
  }

  init() {
    this.components.forEach((tabs) => {
      const buttons = $$("[data-tab]", tabs);
      const panels = $$(".tab-panel", tabs);
      buttons.forEach((button) => {
        button.addEventListener("click", () => {
          buttons.forEach((item) => item.classList.remove("is-active"));
          panels.forEach((panel) => panel.classList.remove("is-active"));
          button.classList.add("is-active");
          document.getElementById(button.dataset.tab)?.classList.add("is-
active");
        });
      });
    });
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new Tabs();
});

```

25. Acordeón

Sección colapsable ideal para FAQs donde solo se muestra el texto al hacer clic.

HTML copiar y pegar

```
<div class="accordion" data-accordion>
  <button class="accordion__button">¿Para qué sirve?</button>
  <div class="accordion__content">Para reutilizar código.</div>
</div>
```

CSS copiar y pegar

```
.accordion {
  max-width: 520px;
}
.accordion__button {
  width: 100%;
  padding: 12px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  background: #ffffff;
  text-align: left;
  cursor: pointer;
}
.accordion__content {
  max-height: 0;
  overflow: hidden;
  margin-top: 8px;
  border-radius: 12px;
  background: #f8fafc;
  transition: max-height 0.2s ease, padding 0.2s ease;
}
.accordion__content.is-open {
  max-height: 140px;
  padding: 12px;
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
export class Accordion {
  constructor(selector = "[data-accordion]") {
    this.components = $$(selector);
    this.init();
  }

  init() {
    this.components.forEach((accordion) => {
      const buttons = $$(".accordion__button", accordion);
      buttons.forEach((button) => {
        button.addEventListener("click", () =>
          button.nextElementSibling.classList.toggle("is-open"));
      });
    });
  }
}
```

```
document.addEventListener("DOMContentLoaded", () => {
```

```
    new Accordion();  
  });
```

JS normal copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);  
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
class Accordion {  
  constructor(selector = "[data-accordion]") {  
    this.components = $$(selector);  
    this.init();  
  }  
  
  init() {  
    this.components.forEach((accordion) => {  
      const buttons = $$(".accordion__button", accordion);  
      buttons.forEach((button) => {  
        button.addEventListener("click", () =>  
button.nextElementSibling.classList.toggle("is-open"));  
      });  
    });  
  }  
}  
  
document.addEventListener("DOMContentLoaded", () => {  
  new Accordion();  
});
```

Guia de reciclaje de componentes web

Funcionalidades 26 a 40 - continuacion del recurso unificado

HTML + CSS + JavaScript modular y version normal

Este documento mantiene el mismo enfoque de la guia anterior: cada funcionalidad se explica como una pieza reciclable. La idea es que puedas copiar el bloque HTML, el bloque CSS y el bloque JavaScript, cambiar los ids, clases, data-attributes o variables necesarias, y reutilizarlo en cualquier proyecto web.

El orden sigue la tabla de contenido del recurso unificado: desde la funcionalidad 26, Modal, hasta la 40, Exportar JSON.

Rango	Funcionalidades cubiertas	Objetivo de esta guia
26 a 40	Modal, Toast, Carrusel, Tooltip, Badge, Copy clipboard, Cards dinamicas, Busqueda debounce, Filtros chips, Ordenamiento, Paginacion, Tabla ordenable, Filtro de tabla, Notas localStorage y Exportar JSON.	Aprender a reciclar cada componente en otros proyectos con instrucciones claras, variables a cambiar y ejemplos en codigo modular y normal.

Tabla de contenido

- 26. Modal reusable
- 27. Toast / notificaciones flotantes
- 28. Carrusel / slider
- 29. Tooltip / ayuda contextual
- 30. Badge / indicador numerico
- 31. Copy clipboard / copiar al portapapeles
- 32. Cards dinamicas desde un array
- 33. Búsqueda con debounce
- 34. Filtros chips por categoria
- 35. Ordenamiento de resultados
- 36. Paginacion
- 37. Tabla ordenable por columnas
- 38. Filtro de tabla por texto
- 39. Notas con localStorage
- 40. Exportar JSON

Como usar esta guia

- 1 Elige la funcionalidad que necesitas reutilizar.
- 2 Copia primero el HTML minimo del componente.
- 3 Copia el CSS asociado y modifica colores, tamaños o variables visuales.
- 4 Escoge una sola version de JavaScript: modular con export/import o normal sin modulos.
- 5 Cambia ids, clases y data-attributes segun la tabla de variables de cada funcionalidad.
- 6 Prueba la funcionalidad con el checklist antes de integrarla con otros componentes.

Regla importante: no mezcles la version modular y la version normal al mismo tiempo. Si usas import/export, carga el HTML con type="module". Si usas JS normal, usa un script comun sin import/export.

Convenciones que se repiten en las funcionalidades 26 a 40

Convencion	Para que sirve	Como adaptarla
data-*	Guarda informacion en HTML para que JavaScript la lea sin depender del texto visible.	Cambia valores como data-copy, data-category, data-page o data-open-modal segun tu proyecto.
is-active / is-open / is-hidden	Clases de estado agregadas por JavaScript para cambiar la vista.	No las escribas manualmente salvo estado inicial; JS debe agregarlas o quitarlas.
containerId	Id del contenedor donde se renderiza o controla el componente.	Debe coincidir con el id del HTML.
callback / onResults / onPageChange	Funcion que se ejecuta cuando un componente produce resultados.	Usala para conectar filtros, busqueda, paginacion o render de cards.
localStorage key	Nombre con el que guardas datos en el navegador.	Cambia claves como kit-notes si vas a mezclar varios proyectos.

Ejemplo de conexion entre componentes: Cards dinamicas + Busqueda debounce + Filtros chips + Ordenamiento + Paginacion. En proyectos reales, lo mas limpio es tener un estado global con los datos originales, filtros activos, texto de busqueda, orden elegido y pagina actual.

26. Modal reutilizable

Aspecto	Explicacion
Que hace	Ventana emergente para confirmar acciones, mostrar formularios, avisos, detalles o contenido que no debe ocupar toda la pagina.
Donde se usa en industria	Confirmar eliminaciones, login rapido, politicas, vista previa de producto, formulario de contacto, mensajes de exito o errores importantes.
Como se personaliza	Cambia el id del modal, los botones data-open-modal y data-close-modal, el titulo, el contenido y el comportamiento de cierre.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="btn" data-open-modal="modalContacto">Abrir modal</button>

<div class="modal" id="modalContacto" aria-hidden="true">
  <div class="modal__overlay" data-close-modal></div>

  <article class="modal__content" role="dialog" aria-modal="true" aria-labelledby="modalTitulo">
    <button class="modal__close" type="button" data-close-modal aria-label="Cerrar modal">x</button>
    <h2 id="modalTitulo">Contactar asesor</h2>
    <p>Completa tus datos y nos comunicaremos contigo.</p>
    <form class="form">
      <input type="text" placeholder="Nombre" required>
      <input type="email" placeholder="Correo" required>
      <button class="btn" type="submit">Enviar</button>
    </form>
  </article>
</div>
```

CSS del componente

styles.css

```

.modal {
  position: fixed;
  inset: 0;
  display: grid;
  place-items: center;
  padding: 24px;
  opacity: 0;
  visibility: hidden;
  pointer-events: none;
  z-index: 1000;
  transition: opacity .25s ease, visibility .25s ease;
}
.modal.is-open {
  opacity: 1;
  visibility: visible;
  pointer-events: auto;
}
.modal__overlay {
  position: absolute;
  inset: 0;
  background: rgba(15, 23, 42, .65);
  backdrop-filter: blur(10px);
}
.modal__content {
  position: relative;
  width: min(520px, 100%);
  padding: 28px;
  border-radius: 22px;
  background: rgba(255,255,255,.92);
  color: #0f172a;
  box-shadow: 0 30px 80px rgba(0,0,0,.35);
  transform: translateY(12px) scale(.98);
  transition: transform .25s ease;
}
.modal.is-open .modal__content {
  transform: translateY(0) scale(1);
}
.modal__close {
  position: absolute;
  top: 12px;
  right: 12px;
  border: 0;
  border-radius: 999px;
  padding: 8px 11px;
  cursor: pointer;
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js


```

export class ModalManager {
  constructor() {
    this.openButtons = document.querySelectorAll('[data-open-modal]');
    this.closeButtons = document.querySelectorAll('[data-close-modal]');
    this.activeModal = null;
    this.init();
  }

  init() {
    this.openButtons.forEach((button) => {
      button.addEventListener('click', () => this.open(button.dataset.openModal));
    });

    this.closeButtons.forEach((button) => {
      button.addEventListener('click', () => this.closeActive());
    });

    document.addEventListener('keydown', (event) => {
      if (event.key === 'Escape') this.closeActive();
    });
  }

  open(modalId) {
    const modal = document.getElementById(modalId);
    if (!modal) return;
    modal.classList.add('is-open');
    modal.setAttribute('aria-hidden', 'false');
    document.body.classList.add('no-scroll');
    this.activeModal = modal;
  }

  closeActive() {
    if (!this.activeModal) return;
    this.activeModal.classList.remove('is-open');
    this.activeModal.setAttribute('aria-hidden', 'true');
    document.body.classList.remove('no-scroll');
    this.activeModal = null;
  }
}

// script.js
import { ModalManager } from './funciones.js';
new ModalManager();

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class ModalManager {
  constructor() {
    this.openButtons = document.querySelectorAll('[data-open-modal]');
    this.closeButtons = document.querySelectorAll('[data-close-modal]');
    this.activeModal = null;
    this.init();
  }
  init() {
    this.openButtons.forEach((button) => {
      button.addEventListener('click', () => this.open(button.dataset.openModal));
    });
    this.closeButtons.forEach((button) => {
      button.addEventListener('click', () => this.closeActive());
    });
    document.addEventListener('keydown', (event) => {
      if (event.key === 'Escape') this.closeActive();
    });
  }
  open(modalId) {
    const modal = document.getElementById(modalId);
    if (!modal) return;
    modal.classList.add('is-open');
    modal.setAttribute('aria-hidden', 'false');
    document.body.classList.add('no-scroll');
    this.activeModal = modal;
  }
  closeActive() {
    if (!this.activeModal) return;
    this.activeModal.classList.remove('is-open');
    this.activeModal.setAttribute('aria-hidden', 'true');
    document.body.classList.remove('no-scroll');
    this.activeModal = null;
  }
}
new ModalManager();

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
id="modalContacto"	Identificador unico del modal.	Debe coincidir con data-open-modal="modalContacto".
data-open-modal	Indica que boton abre un modal.	Cambia el valor por el id de tu modal.
data-close-modal	Marca elementos que cierran el modal.	Ponlo en la X y en el overlay si quieres cerrar al hacer clic afuera.
.modal.is-open	Clase de estado abierto.	No la escribes en HTML; JS la agrega y la quita.

Implementacion paso a paso

- 1 Copia el bloque HTML del modal al final del body para que quede por encima del resto del contenido.
- 2 Copia el CSS del modal en styles.css. Si ya tienes un z-index alto en navbar, sube el z-index del modal.
- 3 Copia la clase ModalManager en funciones.js si usas modulos, o en script-normal.js si no usas modulos.
- 4 Crea un boton con data-open-modal y asegúrate de que el valor coincida exactamente con el id del modal.
- 5 Prueba abrir, cerrar con la X, cerrar con el fondo y cerrar con la tecla Escape.

Errores comunes y como solucionarlos

- El modal no abre: revisa que el id del modal y data-open-modal sean iguales.
- El modal queda detras de otros elementos: aumenta z-index en .modal.
- La pagina se sigue moviendo al abrir: agrega .no-scroll { overflow: hidden; } al CSS.

- El boton dentro del modal recarga la pagina: si es un formulario, controla el submit con `preventDefault` cuando sea demo.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con `ToastManager` para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en `localStorage` si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con `aria-label`, `aria-hidden`, `focus-visible` y navegacion por teclado cuando aplique.

27. Toast / notificaciones flotantes

Aspecto	Explicacion
Que hace	Muestra mensajes temporales sin interrumpir al usuario. Es ideal para confirmaciones rapidas.
Donde se usa en industria	Guardar cambios, copiar texto, iniciar sesion, mostrar error de API, confirmar envio de formulario o aviso de carrito.
Como se personaliza	Cambia posicion, duracion, colores por tipo y mensaje.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="btn" data-toast="Datos guardados correctamente" data-toast-type="success">
  Mostrar toast
</button>

<div class="toast-container" id="toastContainer" aria-live="polite"></div>
```

CSS del componente

styles.css

```
.toast-container {
  position: fixed;
  right: 20px;
  bottom: 20px;
  display: grid;
  gap: 12px;
  z-index: 1200;
}
.toast {
  min-width: 260px;
  max-width: 380px;
  padding: 14px 16px;
  border-radius: 16px;
  color: #fff;
  background: rgba(15, 23, 42, .9);
  border: 1px solid rgba(255,255,255,.18);
  box-shadow: 0 16px 45px rgba(0,0,0,.28);
  backdrop-filter: blur(12px);
  animation: toastIn .25s ease both;
}
.toast--success { background: rgba(22, 163, 74, .92); }
.toast--error { background: rgba(220, 38, 38, .92); }
.toast--info { background: rgba(37, 99, 235, .92); }
.toast.is-leaving { animation: toastOut .25s ease both; }
@keyframes toastIn { from { transform: translateY(12px); opacity: 0; } to { transform: translateY(0); opacity: 1; } }
@keyframes toastOut { to { transform: translateY(12px); opacity: 0; } }
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class ToastManager {
  constructor(containerId = 'toastContainer') {
    this.container = document.getElementById(containerId);
  }

  show(message, type = 'info', duration = 3000) {
    if (!this.container) return;

    const toast = document.createElement('div');
    toast.className = `toast toast--${type}`;
    toast.textContent = message;
    this.container.appendChild(toast);

    setTimeout(() => {
      toast.classList.add('is-leaving');
      toast.addEventListener('animationend', () => toast.remove(), { once: true });
    }, duration);
  }

  bindButtons() {
    document.querySelectorAll('[data-toast]').forEach((button) => {
      button.addEventListener('click', () => {
        this.show(button.dataset.toast, button.dataset.toastType || 'info');
      });
    });
  }
}

// script.js
import { ToastManager } from './funciones.js';
const toast = new ToastManager();
toast.bindButtons();
// toast.show('Producto agregado', 'success');
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class ToastManager {
  constructor(containerId = 'toastContainer') {
    this.container = document.getElementById(containerId);
  }
  show(message, type = 'info', duration = 3000) {
    if (!this.container) return;
    const toast = document.createElement('div');
    toast.className = `toast toast--${type}`;
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => {
      toast.classList.add('is-leaving');
      toast.addEventListener('animationend', () => toast.remove(), { once: true });
    }, duration);
  }
  bindButtons() {
    document.querySelectorAll('[data-toast]').forEach((button) => {
      button.addEventListener('click', () => {
        this.show(button.dataset.toast, button.dataset.toastType || 'info');
      });
    });
  }
}

const toast = new ToastManager();
toast.bindButtons();
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
toastContainer	Contenedor donde se insertan los mensajes.	Cambia el id si quieres varios contenedores.
data-toast	Texto del mensaje.	Personaliza el texto visible.
data-toast-type	Tipo visual: success, error, info.	Crea mas clases CSS si necesitas warning u otros.
duration	Tiempo antes de desaparecer.	Aumenta a 5000 si el mensaje es largo.

Implementacion paso a paso

- 1 Agrega el contenedor toastContainer al final del body.
- 2 Copia los estilos del toast y ajusta right/bottom si lo quieres arriba o a la izquierda.
- 3 Importa ToastManager o pega la version normal.
- 4 Usa botones con data-toast o llama toast.show() desde otros eventos.
- 5 Prueba varios toasts seguidos para confirmar que se apilan bien.

Errores comunes y como solucionarlos

- No aparece nada: falta el div con id toastContainer.
- Sale sin color: revisa que el tipo coincida con una clase .toast--tipo.
- No se elimina: revisa que el navegador ejecute la animacion o reemplaza animationend por toast.remove en setTimeout.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

28. Carrusel / slider

Aspecto	Explicacion
Que hace	Permite mostrar varios bloques de contenido, imagenes o promociones en un mismo espacio con botones anterior/siguiente.
Donde se usa en industria	Hero de home, testimonios, productos destacados, galeria de imagenes, banners comerciales.
Como se personaliza	Duplica slides, cambia imagenes, textos, velocidad, autoplay y controles.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="carousel" id="mainCarousel">
  <div class="carousel__track">
    <article class="carousel__slide is-active">Slide 1 - Promocion principal</article>
    <article class="carousel__slide">Slide 2 - Producto destacado</article>
    <article class="carousel__slide">Slide 3 - Testimonio</article>
  </div>

  <button class="carousel__btn carousel__btn--prev" type="button" data-carousel-prev>Anterior</button>
  <button class="carousel__btn carousel__btn--next" type="button" data-carousel-next>Siguiente</button>

  <div class="carousel__dots" data-carousel-dots></div>
</section>
```

CSS del componente

styles.css

```

.carousel {
  position: relative;
  overflow: hidden;
  border-radius: 24px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
}
.carousel__track { position: relative; min-height: 240px; }
.carousel__slide {
  position: absolute;
  inset: 0;
  display: grid;
  place-items: center;
  padding: 32px;
  opacity: 0;
  transform: translateX(20px);
  transition: opacity .35s ease, transform .35s ease;
}
.carousel__slide.is-active {
  opacity: 1;
  transform: translateX(0);
  position: relative;
}
.carousel__btn {
  position: absolute;
  top: 50%;
  transform: translateY(-50%);
  border: 0;
  border-radius: 999px;
  padding: 10px 14px;
  cursor: pointer;
}
.carousel__btn--prev { left: 12px; }
.carousel__btn--next { right: 12px; }
.carousel__dots {
  display: flex;
  justify-content: center;
  gap: 8px;
  padding: 12px;
}
.carousel__dot {
  width: 10px;
  height: 10px;
  border-radius: 999px;
  border: 0;
  background: rgba(255,255,255,.35);
  cursor: pointer;
}
.carousel__dot.is-active { background: #2563eb; }

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js


```

export class Carousel {
  constructor(carouselId, { autoplay = false, interval = 4000 } = {}) {
    this.carousel = document.getElementById(carouselId);
    if (!this.carousel) return;
    this.slides = [...this.carousel.querySelectorAll('.carousel__slide')];
    this.prev = this.carousel.querySelector('[data-carousel-prev]');
    this.next = this.carousel.querySelector('[data-carousel-next]');
    this.dotsContainer = this.carousel.querySelector('[data-carousel-dots]');
    this.index = 0;
    this.autoplay = autoplay;
    this.interval = interval;
    this.timer = null;
    this.init();
  }

  init() {
    this.renderDots();
    this.prev?.addEventListener('click', () => this.goTo(this.index - 1));
    this.next?.addEventListener('click', () => this.goTo(this.index + 1));
    if (this.autoplay) this.start();
  }

  renderDots() {
    this.dotsContainer.innerHTML = this.slides.map((_, i) =>
      `<button class="carousel__dot" data-dot="${i}" aria-label="Ir al slide ${i + 1}"></button>`
    ).join('');
    this.dots = [...this.dotsContainer.querySelectorAll('.carousel__dot')];
    this.dots.forEach((dot) => dot.addEventListener('click', () => this.goTo(Number(dot.dataset.
dot))));
    this.update();
  }

  goTo(newIndex) {
    this.index = (newIndex + this.slides.length) % this.slides.length;
    this.update();
  }

  update() {
    this.slides.forEach((slide, i) => slide.classList.toggle('is-active', i === this.index));
    this.dots?.forEach((dot, i) => dot.classList.toggle('is-active', i === this.index));
  }

  start() { this.timer = setInterval(() => this.goTo(this.index + 1), this.interval); }
  stop() { clearInterval(this.timer); }
}

// script.js
import { Carousel } from './funciones.js';
new Carousel('mainCarousel', { autoplay: true, interval: 5000 });

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class Carousel {
  constructor(carouselId, options = {}) {
    this.carousel = document.getElementById(carouselId);
    if (!this.carousel) return;
    this.slides = [...this.carousel.querySelectorAll('.carousel__slide')];
    this.prev = this.carousel.querySelector('[data-carousel-prev]');
    this.next = this.carousel.querySelector('[data-carousel-next]');
    this.dotsContainer = this.carousel.querySelector('[data-carousel-dots]');
    this.index = 0;
    this.autoplay = options.autoplay || false;
    this.interval = options.interval || 4000;
    this.init();
  }
  init() {
    this.renderDots();
    this.prev?.addEventListener('click', () => this.goTo(this.index - 1));
    this.next?.addEventListener('click', () => this.goTo(this.index + 1));
    if (this.autoplay) setInterval(() => this.goTo(this.index + 1), this.interval);
  }
  renderDots() {
    this.dotsContainer.innerHTML = this.slides.map((_, i) => `<button class="carousel__dot" data-
dot="${i}"></button>`).join('');
    this.dots = [...this.dotsContainer.querySelectorAll('.carousel__dot')];
    this.dots.forEach((dot) => dot.addEventListener('click', () => this.goTo(Number(dot.dataset.
dot))));
    this.update();
  }
  goTo(newIndex) {
    this.index = (newIndex + this.slides.length) % this.slides.length;
    this.update();
  }
  update() {
    this.slides.forEach((slide, i) => slide.classList.toggle('is-active', i === this.index));
    this.dots.forEach((dot, i) => dot.classList.toggle('is-active', i === this.index));
  }
}
new Carousel('mainCarousel', { autoplay: true });

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
mainCarousel	Id del carrusel.	Cambia el id si tienes mas de un carrusel.
.carousel__slide	Cada elemento visible del carrusel.	Duplica esta etiqueta para agregar mas slides.
autoplay	Activa cambio automatico.	Pon false para que solo cambie con botones.
interval	Tiempo entre slides.	Valor en milisegundos: 5000 = 5 segundos.

Implementacion paso a paso

- 1 Copia el HTML del carrusel donde quieras mostrarlo.
- 2 Duplica o elimina elementos .carousel__slide segun el numero de contenidos.
- 3 Copia los estilos y ajusta min-height si tus imagenes son mas grandes.
- 4 Inicializa la clase con el id correspondiente.
- 5 Prueba botones, dots y autoplay.

Errores comunes y como solucionarlos

- Solo se ve un slide siempre: revisa que la clase is-active cambie en DevTools.
- Los botones no aparecen: puede estar faltando position: relative en .carousel.
- Los dots no se generan: falta el contenedor data-carousel-dots.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

29. Tooltip / ayuda contextual

Aspecto	Explicacion
Que hace	Muestra una ayuda corta al pasar el mouse o enfocar un elemento.
Donde se usa en industria	Iconos de ayuda, botones con acciones, formularios, tablas con estados, paneles administrativos.
Como se personaliza	Cambia el texto de data-tooltip, la posicion y el estilo visual.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="icon-button" data-tooltip="Editar producto" aria-label="Editar producto">
  Editar
</button>

<span class="info" data-tooltip="Este dato se guarda automaticamente">?</span>
```

CSS del componente

styles.css

```

[data-tooltip] {
  position: relative;
}
[data-tooltip]::before,
[data-tooltip]::after {
  position: absolute;
  left: 50%;
  opacity: 0;
  visibility: hidden;
  pointer-events: none;
  transition: opacity .2s ease, transform .2s ease;
  z-index: 900;
}
[data-tooltip]::before {
  content: attr(data-tooltip);
  bottom: calc(100% + 10px);
  transform: translateX(-50%) translateY(6px);
  width: max-content;
  max-width: 220px;
  padding: 8px 10px;
  border-radius: 10px;
  background: #0f172a;
  color: #fff;
  font-size: 13px;
  box-shadow: 0 10px 25px rgba(0,0,0,.22);
}
[data-tooltip]::after {
  content: "";
  bottom: calc(100% + 4px);
  transform: translateX(-50%) translateY(6px);
  border: 6px solid transparent;
  border-top-color: #0f172a;
}
[data-tooltip]:hover::before,
[data-tooltip]:hover::after,
[data-tooltip]:focus-visible::before,
[data-tooltip]:focus-visible::after {
  opacity: 1;
  visibility: visible;
  transform: translateX(-50%) translateY(0);
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

// Version modular opcional para actualizar tooltips dinamicamente.
export function setTooltip(selector, text) {
  const element = document.querySelector(selector);
  if (!element) return;
  element.setAttribute('data-tooltip', text);
}

export function removeTooltip(selector) {
  const element = document.querySelector(selector);
  if (!element) return;
  element.removeAttribute('data-tooltip');
}

// script.js
import { setTooltip } from './funciones.js';
setTooltip('#botonGuardar', 'Guarda los cambios del formulario');

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
function setTooltip(selector, text) {
  const element = document.querySelector(selector);
  if (!element) return;
  element.setAttribute('data-tooltip', text);
}

function removeTooltip(selector) {
  const element = document.querySelector(selector);
  if (!element) return;
  element.removeAttribute('data-tooltip');
}

setTooltip('#botonGuardar', 'Guarda los cambios del formulario');
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-tooltip	Texto del tooltip.	Cambia el texto directamente en HTML.
bottom	Ubicacion vertical del tooltip.	Cambia a top/left/right si quieres otra posicion.
max-width	Ancho maximo del tooltip.	Sube a 280px si el texto es largo.
z-index	Prioridad visual.	Aumenta si el tooltip queda detras de una card.

Implementacion paso a paso

1. Agrega data-tooltip al elemento que necesita ayuda contextual.
2. Asegurate de que el elemento pueda recibir foco si es interactivo.
3. Copia el CSS y prueba hover y focus-visible.
4. Si el texto es largo, acortalo o aumenta max-width.
5. Usa setTooltip solo si necesitas cambiar el texto desde JavaScript.

Errores comunes y como solucionarlos

- No se ve el tooltip: puede estar recortado por un contenedor con overflow: hidden.
- Se ve muy ancho: usa max-width o divide el texto.
- No funciona en celular: para mobile conviene mostrar ayuda con clic o modal.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.

- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

30. Badge / indicador numerico

Aspecto	Explicacion
Que hace	Muestra un numero o estado pequeño sobre un boton, icono o texto.
Donde se usa en industria	Notificaciones, carrito de compras, mensajes pendientes, tareas abiertas, alertas de panel administrativo.
Como se personaliza	Cambia el id del badge, el numero, los colores y la condicion para ocultarlo.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="notification-button" type="button">
  Notificaciones
  <span class="badge" id="notificationBadge">3</span>
</button>

<button class="btn" id="addNotification">Agregar</button>
<button class="btn" id="clearNotifications">Limpiar</button>
```

CSS del componente

styles.css

```
.notification-button {
  position: relative;
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 10px 16px;
  border-radius: 999px;
}
.badge {
  min-width: 22px;
  height: 22px;
  display: inline-grid;
  place-items: center;
  padding: 0 6px;
  border-radius: 999px;
  background: #ef4444;
  color: #fff;
  font-size: 12px;
  font-weight: 700;
  box-shadow: 0 6px 16px rgba(239,68,68,.35);
}
.badge.is-hidden {
  display: none;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class BadgeCounter {
  constructor(badgeId, initialValue = 0) {
    this.badge = document.getElementById(badgeId);
    this.value = initialValue;
    this.render();
  }

  increment(amount = 1) {
    this.value += amount;
    this.render();
  }

  decrement(amount = 1) {
    this.value = Math.max(0, this.value - amount);
    this.render();
  }

  reset() {
    this.value = 0;
    this.render();
  }

  render() {
    if (!this.badge) return;
    this.badge.textContent = this.value > 99 ? '99+' : this.value;
    this.badge.classList.toggle('is-hidden', this.value === 0);
  }
}

// script.js
import { BadgeCounter } from './funciones.js';
const badge = new BadgeCounter('notificationBadge', 3);
document.getElementById('addNotification').addEventListener('click', () => badge.increment());
document.getElementById('clearNotifications').addEventListener('click', () => badge.reset());
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class BadgeCounter {
  constructor(badgeId, initialValue = 0) {
    this.badge = document.getElementById(badgeId);
    this.value = initialValue;
    this.render();
  }
  increment(amount = 1) { this.value += amount; this.render(); }
  decrement(amount = 1) { this.value = Math.max(0, this.value - amount); this.render(); }
  reset() { this.value = 0; this.render(); }
  render() {
    if (!this.badge) return;
    this.badge.textContent = this.value > 99 ? '99+' : this.value;
    this.badge.classList.toggle('is-hidden', this.value === 0);
  }
}

const badge = new BadgeCounter('notificationBadge', 3);
document.getElementById('addNotification').addEventListener('click', () => badge.increment());
document.getElementById('clearNotifications').addEventListener('click', () => badge.reset());
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
notificationBadge	Id del span que muestra el numero.	Cambia si tienes varios badges.
initialValue	Valor inicial.	Usa 0 si debe iniciar oculto.
99+	Limite visual.	Cambia a 9+ o 999+ segun tu interfaz.

Elemento	Para que sirve	Que debes cambiar
.badge.is-hidden	Clase que oculta el indicador.	Cambia display none por opacity si quieres animacion.

Implementacion paso a paso

- 1 Crea un span con clase badge dentro del boton o icono.
- 2 Copia los estilos del badge.
- 3 Inicializa BadgeCounter con el id correcto.
- 4 Llama increment, decrement o reset segun la accion del usuario.
- 5 Prueba que al llegar a 0 se oculte.

Errores comunes y como solucionarlos

- El badge aparece fuera de lugar: pon position: relative en el contenedor.
- No se actualiza: revisa que el id sea correcto.
- Muestra valores negativos: usa Math.max como en el ejemplo.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

31. Copy clipboard / copiar al portapapeles

Aspecto	Explicacion
Que hace	Permite copiar texto con un clic desde un bloque, input, codigo o cupon.
Donde se usa en industria	Cupones, enlaces de invitacion, codigos, tokens publicos, datos de contacto, comandos de instalacion.
Como se personaliza	Cambia el selector data-copy, el mensaje de confirmacion y el fallback.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<p id="codigoCupon">SIERRA-2026</p>
<button class="btn" data-copy="#codigoCupon">Copiar cupon</button>

<input id="correoEmpresa" value="contacto@empresa.com" readonly>
<button class="btn" data-copy="#correoEmpresa">Copiar correo</button>
```

CSS del componente

styles.css

```
[data-copy] {
  cursor: pointer;
}
.copy-feedback {
  display: inline-block;
  margin-left: 8px;
  color: #16a34a;
  font-weight: 700;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class ClipboardManager {
  constructor({ onSuccess = null, onError = null } = {}) {
    this.onSuccess = onSuccess;
    this.onError = onError;
    this.init();
  }

  init() {
    document.querySelectorAll('[data-copy]').forEach((button) => {
      button.addEventListener('click', () => this.copyFromSelector(button.dataset.copy, button));
    });
  }

  async copyFromSelector(selector, trigger) {
    const target = document.querySelector(selector);
    if (!target) return;
    const text = target.value ?? target.textContent;

    try {
      await navigator.clipboard.writeText(text.trim());
      trigger.textContent = 'Copiado';
      setTimeout(() => trigger.textContent = 'Copiar', 1500);
      this.onSuccess?.(text);
    } catch (error) {
      this.fallbackCopy(text);
      this.onError?.(error);
    }
  }

  fallbackCopy(text) {
    const area = document.createElement('textarea');
    area.value = text;
    document.body.appendChild(area);
    area.select();
    document.execCommand('copy');
    area.remove();
  }
}

// script.js
import { ClipboardManager } from './funciones.js';
new ClipboardManager();

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class ClipboardManager {
  constructor() { this.init(); }
  init() {
    document.querySelectorAll('[data-copy]').forEach((button) => {
      button.addEventListener('click', () => this.copyFromSelector(button.dataset.copy, button));
    });
  }
  async copyFromSelector(selector, trigger) {
    const target = document.querySelector(selector);
    if (!target) return;
    const text = target.value || target.textContent;
    try {
      await navigator.clipboard.writeText(text.trim());
      trigger.textContent = 'Copiado';
      setTimeout(() => trigger.textContent = 'Copiar', 1500);
    } catch (error) {
      const area = document.createElement('textarea');
      area.value = text;
      document.body.appendChild(area);
      area.select();
      document.execCommand('copy');
      area.remove();
    }
  }
}

new ClipboardManager();

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-copy	Selector del elemento que contiene el texto.	Debe apuntar a un id, clase o selector valido.
navigator.clipboard	API moderna de copia.	Funciona mejor en HTTPS o localhost.
trigger.textContent	Texto del boton despues de copiar.	Personaliza Copiado/Copiar segun tu UI.
fallbackCopy	Plan B para navegadores antiguos.	Mantenlo si quieres compatibilidad.

Implementacion paso a paso

- 1 Pon el texto a copiar en un elemento con id.
- 2 Crea un boton con data-copy apuntando a ese selector.
- 3 Copia la clase ClipboardManager y ejecútala.
- 4 Prueba en localhost o servidor seguro.
- 5 Conecta con ToastManager si quieres mostrar una notificacion bonita.

Errores comunes y como solucionarlos

- No copia: el selector data-copy no encuentra el elemento.
- Funciona local pero no en servidor: revisa permisos HTTPS.
- Cambia el texto del boton a Copiado pero no vuelve: revisa setTimeout.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

32. Cards dinamicas desde un array

Aspecto	Explicacion
Que hace	Renderiza tarjetas automaticamente a partir de datos JavaScript.
Donde se usa en industria	Catalogos, portafolios, productos, cursos, blogs, equipos, servicios y dashboards.
Como se personaliza	Edita el array de datos, el template HTML y el contenedor donde se pintan las cards.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="cards-grid" id="cardsGrid"></section>
```

CSS del componente

styles.css

```
.cards-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));  
  gap: 18px;  
}  
.card {  
  padding: 20px;  
  border-radius: 20px;  
  background: rgba(255,255,255,.12);  
  border: 1px solid rgba(255,255,255,.18);  
  box-shadow: 0 14px 35px rgba(0,0,0,.18);  
}  
.card__title { margin: 0 0 8px; font-size: 20px; }  
.card__meta { color: #64748b; font-size: 14px; }  
.card__price { font-weight: 800; color: #2563eb; }
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export class CatalogCards {
  constructor(containerId, items = []) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.render(this.items);
  }

  setItems(items) {
    this.items = items;
    this.render(this.items);
  }

  render(items) {
    if (!this.container) return;
    if (items.length === 0) {
      this.container.innerHTML = '<p>No hay resultados disponibles.</p>';
      return;
    }

    this.container.innerHTML = items.map((item) => `
      <article class="card" data-category="${item.category}">
        <h3 class="card__title">${item.name}</h3>
        <p>${item.description}</p>
        <p class="card__meta">Categoria: ${item.category}</p>
        <p class="card__price">${item.price}</p>
      </article>
    `).join('');
  }
}

// script.js
import { CatalogCards } from './funciones.js';
const products = [
  { name: 'Producto A', category: 'web', price: 12000, description: 'Descripcion corta.' },
  { name: 'Producto B', category: 'data', price: 18000, description: 'Otra descripcion.' }
];
const catalog = new CatalogCards('cardsGrid', products);

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class CatalogCards {
  constructor(containerId, items = []) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.render(this.items);
  }
  setItems(items) {
    this.items = items;
    this.render(this.items);
  }
  render(items) {
    if (!this.container) return;
    if (items.length === 0) {
      this.container.innerHTML = '<p>No hay resultados disponibles.</p>';
      return;
    }
    this.container.innerHTML = items.map((item) => `
      <article class="card" data-category="${item.category}">
        <h3 class="card__title">${item.name}</h3>
        <p>${item.description}</p>
        <p class="card__meta">Categoria: ${item.category}</p>
        <p class="card__price">${item.price}</p>
      </article>
    `).join('');
  }
}
const products = [
  { name: 'Producto A', category: 'web', price: 12000, description: 'Descripcion corta.' },
  { name: 'Producto B', category: 'data', price: 18000, description: 'Otra descripcion.' }
];
const catalog = new CatalogCards('cardsGrid', products);

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
cardsGrid	Contenedor donde se pintan las cards.	Cambia el id si tu seccion tiene otro nombre.
products/items	Array de datos.	Agrega campos como image, rating, stock o url.
render()	Metodo que convierte datos en HTML.	Edita el template para cambiar el diseño.
data-category	Dato util para filtrar.	Debe coincidir con tus filtros chips.

Implementacion paso a paso

- 1 Crea un contenedor vacio para las cards.
- 2 Define un array con objetos consistentes.
- 3 Copia la clase CatalogCards.
- 4 Instancia la clase con el id y el array.
- 5 Edita el template dentro de render para adaptarlo a tu proyecto.

Errores comunes y como solucionarlos

- Se imprime undefined: falta una propiedad en algun objeto del array.
- No aparecen cards: revisa el id del contenedor.
- Hay riesgo de HTML no confiable si los datos vienen de usuarios. Sanitiza si viene de fuentes externas.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

33. Búsqueda con debounce

Aspecto	Explicacion
Que hace	Evita ejecutar una búsqueda en cada tecla inmediatamente. Espera unos milisegundos despues de que el usuario deja de escribir.
Donde se usa en industria	Filtros de catalogo, buscadores, tablas, llamadas a APIs, dashboards y autocompletados.
Como se personaliza	Cambia el delay, el input, la lista de datos y la funcion que filtra.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<input type="search" id="searchInput" placeholder="Buscar productos...">
<section id="cardsGrid" class="cards-grid"></section>
```

CSS del componente

styles.css

```
input[type="search"] {
  width: 100%;
  padding: 12px 14px;
  border-radius: 14px;
  border: 1px solid #cbd5e1;
  margin-bottom: 18px;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export function debounce(callback, delay = 300) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => callback(...args), delay);
  };
}

export class SearchController {
  constructor(inputId, items, onResults, delay = 300) {
    this.input = document.getElementById(inputId);
    this.items = items;
    this.onResults = onResults;
    this.delay = delay;
    this.init();
  }

  init() {
    if (!this.input) return;
    this.input.addEventListener('input', debounce(() => this.search(), this.delay));
  }

  search() {
    const term = this.input.value.trim().toLowerCase();
    const results = this.items.filter((item) => {
      return item.name.toLowerCase().includes(term)
        || item.category.toLowerCase().includes(term)
        || item.description.toLowerCase().includes(term);
    });
    this.onResults(results);
  }
}

// script.js
import { SearchController, CatalogCards } from './funciones.js';
const catalog = new CatalogCards('cardsGrid', products);
new SearchController('searchInput', products, (results) => catalog.render(results), 350);

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

function debounce(callback, delay = 300) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => callback(...args), delay);
  };
}

class SearchController {
  constructor(inputId, items, onResults, delay = 300) {
    this.input = document.getElementById(inputId);
    this.items = items;
    this.onResults = onResults;
    this.delay = delay;
    this.init();
  }

  init() {
    if (!this.input) return;
    this.input.addEventListener('input', debounce(() => this.search(), this.delay));
  }

  search() {
    const term = this.input.value.trim().toLowerCase();
    const results = this.items.filter((item) =>
      item.name.toLowerCase().includes(term) ||
      item.category.toLowerCase().includes(term) ||
      item.description.toLowerCase().includes(term)
    );
    this.onResults(results);
  }
}

new SearchController('searchInput', products, (results) => catalog.render(results), 350);

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
delay	Tiempo de espera para buscar.	300 a 500 ms suele sentirse bien.
items	Datos originales.	Debe ser el array completo, no el ya filtrado.
onResults	Funcion que recibe resultados.	Puedes renderizar cards, tabla o lista.
term	Texto escrito por el usuario.	Puedes normalizar acentos si lo necesitas.

Implementacion paso a paso

- 1 Crea un input tipo search.
- 2 Ten un array original sin modificar.
- 3 Copia debounce y SearchController.
- 4 Pasa una funcion callback que actualice la UI.
- 5 Prueba con mayusculas, minusculas y texto vacio.

Errores comunes y como solucionarlos

- Filtra sobre resultados ya filtrados y pierde datos: siempre filtra desde el array original.
- Busca muy lento: baja el delay.
- Busca demasiado: sube el delay si hace llamadas a una API.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

34. Filtros chips por categoria

Aspecto	Explicacion
Que hace	Permite filtrar datos con botones pequeños tipo etiqueta o categoria.
Donde se usa en industria	Catalogos, portafolios, blogs, dashboards, filtros de productos, categorias de servicios.
Como se personaliza	Cambia data-category y asegúrate de que coincida con la categoria del dato.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<div class="chips" id="categoryChips">
  <button class="chip is-active" data-category="all">Todos</button>
  <button class="chip" data-category="web">Web</button>
  <button class="chip" data-category="data">Datos</button>
  <button class="chip" data-category="design">Diseno</button>
</div>

<section id="cardsGrid" class="cards-grid"></section>
```

CSS del componente

styles.css

```
.chips {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin-bottom: 18px;
}
.chip {
  border: 1px solid #cbd5e1;
  border-radius: 999px;
  padding: 9px 14px;
  background: #fff;
  cursor: pointer;
  transition: background .2s ease, transform .2s ease;
}
.chip:hover { transform: translateY(-1px); }
.chip.is-active {
  background: linear-gradient(135deg, #2563eb, #7c3aed);
  color: #fff;
  border-color: transparent;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class ChipFilters {
  constructor(containerId, items, onResults) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.onResults = onResults;
    this.activeCategory = 'all';
    this.init();
  }

  init() {
    if (!this.container) return;
    this.container.addEventListener('click', (event) => {
      const chip = event.target.closest('[data-category]');
      if (!chip) return;
      this.activeCategory = chip.dataset.category;
      this.container.querySelectorAll('.chip').forEach((item) => item.classList.remove('is-
active'));
      chip.classList.add('is-active');
      this.apply();
    });
  }

  apply() {
    const results = this.activeCategory === 'all'
      ? this.items
      : this.items.filter((item) => item.category === this.activeCategory);
    this.onResults(results);
  }
}

// script.js
import { ChipFilters } from './funciones.js';
new ChipFilters('categoryChips', products, (results) => catalog.render(results));

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class ChipFilters {
  constructor(containerId, items, onResults) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.onResults = onResults;
    this.activeCategory = 'all';
    this.init();
  }

  init() {
    if (!this.container) return;
    this.container.addEventListener('click', (event) => {
      const chip = event.target.closest('[data-category]');
      if (!chip) return;
      this.activeCategory = chip.dataset.category;
      this.container.querySelectorAll('.chip').forEach((item) => item.classList.remove('is-
active'));
      chip.classList.add('is-active');
      this.apply();
    });
  }

  apply() {
    const results = this.activeCategory === 'all' ? this.items : this.items.filter((item) => item.
category === this.activeCategory);
    this.onResults(results);
  }
}

new ChipFilters('categoryChips', products, (results) => catalog.render(results));

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-category	Categoría que representa cada chip.	Debe coincidir con item.category.
all	Categoría especial para mostrar todos.	Puedes cambiarla por todos, pero actualiza JS.

Elemento	Para que sirve	Que debes cambiar
is-active	Clase visual del filtro activo.	Cambia colores en CSS.
onResults	Funcion que actualiza la vista.	Renderiza cards, tabla o lista.

Implementacion paso a paso

- 1 Crea un contenedor de chips.
- 2 Agrega un boton por categoria.
- 3 Asegúrate de que las categorias existan en los datos.
- 4 Inicializa ChipFilters con el contenedor, datos y callback.
- 5 Prueba cada chip y revisa que el activo cambie visualmente.

Errores comunes y como solucionarlos

- Un chip no filtra: data-category no coincide exactamente con item.category.
- No se ve el activo: falta la clase CSS .chip.is-active.
- Al mezclar con busqueda, necesitas un estado global que combine filtros y texto.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

35. Ordenamiento de resultados

Aspecto	Explicacion
Que hace	Ordena listas por precio, nombre, calificacion, fecha u otro criterio.
Donde se usa en industria	E-commerce, catalogos, dashboards, tablas, listados de usuarios y reportes.
Como se personaliza	Cambia las opciones del select y la funcion de comparacion.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<label for="sortSelect">Ordenar por:</label>
<select id="sortSelect">
  <option value="name-asc">Nombre A-Z</option>
  <option value="name-desc">Nombre Z-A</option>
  <option value="price-asc">Menor precio</option>
  <option value="price-desc">Mayor precio</option>
</select>
```

CSS del componente

styles.css

```
#sortSelect {
  width: 100%;
  max-width: 280px;
  padding: 10px 12px;
  border-radius: 12px;
  border: 1px solid #cbd5e1;
  background: #fff;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class SortController {
  constructor(selectId, items, onResults) {
    this.select = document.getElementById(selectId);
    this.items = items;
    this.onResults = onResults;
    this.init();
  }

  init() {
    if (!this.select) return;
    this.select.addEventListener('change', () => this.sort());
  }

  sort() {
    const value = this.select.value;
    const sorted = [...this.items];

    const strategies = {
      'name-asc': (a, b) => a.name.localeCompare(b.name),
      'name-desc': (a, b) => b.name.localeCompare(a.name),
      'price-asc': (a, b) => a.price - b.price,
      'price-desc': (a, b) => b.price - a.price,
    };

    sorted.sort(strategies[value] || strategies['name-asc']);
    this.onResults(sorted);
  }
}

// script.js
import { SortController } from './funciones.js';
new SortController('sortSelect', products, (results) => catalog.render(results));

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class SortController {
  constructor(selectId, items, onResults) {
    this.select = document.getElementById(selectId);
    this.items = items;
    this.onResults = onResults;
    this.init();
  }
  init() {
    if (!this.select) return;
    this.select.addEventListener('change', () => this.sort());
  }
  sort() {
    const value = this.select.value;
    const sorted = [...this.items];
    const strategies = {
      'name-asc': (a, b) => a.name.localeCompare(b.name),
      'name-desc': (a, b) => b.name.localeCompare(a.name),
      'price-asc': (a, b) => a.price - b.price,
      'price-desc': (a, b) => b.price - a.price,
    };
    sorted.sort(strategies[value] || strategies['name-asc']);
    this.onResults(sorted);
  }
}
new SortController('sortSelect', products, (results) => catalog.render(results));

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
sortSelect	Id del select de ordenamiento.	Cambia si tienes otro id.
strategies	Objeto con funciones de orden.	Agrega mas criterios: rating, date, stock.
[...this.items]	Copia del array original.	Evita modificar directamente los datos base.
localeCompare	Orden alfabético.	Muy util para nombres y textos.

Implementacion paso a paso

- 1 Crea un select con opciones de orden.
- 2 Cada value debe existir como clave dentro de strategies.
- 3 Copia SortController.
- 4 Pasa el array y un callback de render.
- 5 Prueba orden ascendente y descendente.

Errores comunes y como solucionarlos

- El orden no cambia: el value del option no coincide con strategies.
- Se dañan los datos originales: no uses sort directamente sobre items si luego necesitas el orden original.
- Ordena numeros como texto: usa resta `a.price - b.price` para valores numericos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con `aria-label`, `aria-hidden`, `focus-visible` y navegacion por teclado cuando aplique.

36. Paginacion

Aspecto	Explicacion
Que hace	Divide muchos resultados en varias paginas para evitar interfaces pesadas.
Donde se usa en industria	Catalogos, tablas administrativas, blogs, busquedas, resultados de API y listados grandes.
Como se personaliza	Cambia perPage, contenedor de botones y render de resultados.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section id="cardsGrid" class="cards-grid"></section>
<nav class="pagination" id="pagination" aria-label="Paginacion"></nav>
```

CSS del componente

styles.css

```
.pagination {
  display: flex;
  justify-content: center;
  flex-wrap: wrap;
  gap: 8px;
  margin-top: 20px;
}
.pagination__button {
  border: 1px solid #cbd5e1;
  background: #fff;
  border-radius: 10px;
  padding: 8px 12px;
  cursor: pointer;
}
.pagination__button.is-active {
  color: #fff;
  background: #2563eb;
  border-color: #2563eb;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class Pagination {
  constructor(containerId, items, perPage, onPageChange) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.perPage = perPage;
    this.onPageChange = onPageChange;
    this.currentPage = 1;
    this.render();
    this.emit();
  }

  setItems(items) {
    this.items = items;
    this.currentPage = 1;
    this.render();
    this.emit();
  }

  get totalPages() {
    return Math.ceil(this.items.length / this.perPage) || 1;
  }

  getPageItems() {
    const start = (this.currentPage - 1) * this.perPage;
    return this.items.slice(start, start + this.perPage);
  }

  render() {
    if (!this.container) return;
    this.container.innerHTML = Array.from({ length: this.totalPages }, (_, i) => `
      <button class="pagination__button ${i + 1 === this.currentPage ? 'is-active' : ''}" data-
page="${i + 1}">
        ${i + 1}
      </button>
    `).join('');
    this.container.querySelectorAll('[data-page]').forEach((button) => {
      button.addEventListener('click', () => {
        this.currentPage = Number(button.dataset.page);
        this.render();
        this.emit();
      });
    });
  }

  emit() {
    this.onPageChange(this.getPageItems(), this.currentPage, this.totalPages);
  }
}

// script.js
import { Pagination } from './funciones.js';
const pager = new Pagination('pagination', products, 6, (pageItems) => catalog.render(pageItems));

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class Pagination {
  constructor(containerId, items, perPage, onPageChange) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.perPage = perPage;
    this.onPageChange = onPageChange;
    this.currentPage = 1;
    this.render();
    this.emit();
  }
  setItems(items) { this.items = items; this.currentPage = 1; this.render(); this.emit(); }
  get totalPages() { return Math.ceil(this.items.length / this.perPage) || 1; }
  getPageItems() {
    const start = (this.currentPage - 1) * this.perPage;
    return this.items.slice(start, start + this.perPage);
  }
  render() {
    if (!this.container) return;
    this.container.innerHTML = Array.from({ length: this.totalPages }, (_, i) => `<button
class="pagination__button ${i + 1} === this.currentPage ? 'is-active' : ''" data-page="${i + 1}">${i +
1}</button>`).join('');
    this.container.querySelectorAll('[data-page]').forEach((button) => {
      button.addEventListener('click', () => {
        this.currentPage = Number(button.dataset.page);
        this.render();
        this.emit();
      });
    });
  }
  emit() { this.onPageChange(this.getPageItems(), this.currentPage, this.totalPages); }
}
const pager = new Pagination('pagination', products, 6, (pageItems) => catalog.render(pageItems));

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
pagination	Contenedor de botones de pagina.	Cambia id segun tu HTML.
perPage	Cantidad de elementos por pagina.	6, 9 o 12 son comunes para cards.
setItems()	Actualiza resultados paginados.	Usalo despues de buscar o filtrar.
slice	Extrae solo los elementos visibles.	No modifica el array original.

Implementacion paso a paso

- 1 Crea el contenedor de resultados y el nav de paginacion.
- 2 Copia la clase Pagination.
- 3 Pasa el array completo, cantidad por pagina y callback.
- 4 Cuando filtres o busques, llama pager.setItems(resultados).
- 5 Prueba ultima pagina y filtros con pocos resultados.

Errores comunes y como solucionarlos

- Se muestran resultados de otra pagina tras filtrar: debes reiniciar currentPage a 1.
- No aparecen botones: totalPages es 0 si el array no llega correctamente.
- Demasiados botones: para listas enormes conviene paginacion con puntos suspensivos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.

- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

37. Tabla ordenable por columnas

Aspecto	Explicacion
Que hace	Permite ordenar filas al hacer clic en los encabezados de una tabla.
Donde se usa en industria	Paneles administrativos, reportes, inventarios, usuarios, facturas, indicadores y listados tecnicos.
Como se personaliza	Agrega data-sort-table a los th y define si la columna es texto o numero.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<table class="data-table" id="ordersTable">
  <thead>
    <tr>
      <th data-sort-table="text">Cliente</th>
      <th data-sort-table="number">Total</th>
      <th data-sort-table="text">Estado</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>Carlos</td><td>120000</td><td>Pagado</td></tr>
    <tr><td>Laura</td><td>80000</td><td>Pendiente</td></tr>
  </tbody>
</table>
```

CSS del componente

styles.css

```
.data-table {
  width: 100%;
  border-collapse: collapse;
  overflow: hidden;
  border-radius: 16px;
}
.data-table th,
.data-table td {
  padding: 12px;
  border-bottom: 1px solid #e2e8f0;
  text-align: left;
}
.data-table th {
  background: #0f172a;
  color: #fff;
  cursor: pointer;
  user-select: none;
}
.data-table tr:hover td { background: #f8fafc; }
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```
export class SortableTable {
  constructor(tableId) {
    this.table = document.getElementById(tableId);
    this.direction = 1;
    this.lastColumn = null;
    this.init();
  }

  init() {
    if (!this.table) return;
    this.table.querySelectorAll('[data-sort-table]').forEach((th, index) => {
      th.addEventListener('click', () => this.sortByColumn(index, th.dataset.sortTable));
    });
  }

  sortByColumn(index, type) {
    const tbody = this.table.querySelector('tbody');
    const rows = [...tbody.querySelectorAll('tr')];

    if (this.lastColumn === index) this.direction *= -1;
    else this.direction = 1;
    this.lastColumn = index;

    rows.sort((a, b) => {
      const valueA = a.children[index].textContent.trim();
      const valueB = b.children[index].textContent.trim();
      if (type === 'number') return (Number(valueA) - Number(valueB)) * this.direction;
      return valueA.localeCompare(valueB) * this.direction;
    });

    tbody.append(...rows);
  }
}

// script.js
import { SortableTable } from './funciones.js';
new SortableTable('ordersTable');
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como `script-normal.js` y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class SortableTable {
  constructor(tableId) {
    this.table = document.getElementById(tableId);
    this.direction = 1;
    this.lastColumn = null;
    this.init();
  }

  init() {
    if (!this.table) return;
    this.table.querySelectorAll('[data-sort-table]').forEach((th, index) => {
      th.addEventListener('click', () => this.sortByColumn(index, th.dataset.sortTable));
    });
  }

  sortByColumn(index, type) {
    const tbody = this.table.querySelector('tbody');
    const rows = [...tbody.querySelectorAll('tr')];
    if (this.lastColumn === index) this.direction *= -1;
    else this.direction = 1;
    this.lastColumn = index;
    rows.sort((a, b) => {
      const valueA = a.children[index].textContent.trim();
      const valueB = b.children[index].textContent.trim();
      if (type === 'number') return (Number(valueA) - Number(valueB)) * this.direction;
      return valueA.localeCompare(valueB) * this.direction;
    });
    tbody.append(...rows);
  }
}

new SortableTable('ordersTable');
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
ordersTable	Id de la tabla.	Cambia por el id real de tu tabla.
data-sort-table	Tipo de ordenamiento.	Usa text o number.
direction	Direccion asc/desc.	Se invierte al hacer clic dos veces en la misma columna.
children[index]	Celda correspondiente a la columna.	No debe haber celdas combinadas complicadas.

Implementacion paso a paso

- 1 Crea una tabla con thead y tbody.
- 2 Agrega data-sort-table a los th que quieras ordenar.
- 3 Copia la clase SortableTable.
- 4 Instancia con el id de la tabla.
- 5 Prueba orden en columnas de texto y numero.

Errores comunes y como solucionarlos

- Ordena mal numeros con simbolos: limpia valores antes de Number.
- No funciona con celdas colspan: evita estructuras complejas para tablas ordenables simples.
- No se nota que es clickeable: agrega cursor pointer y una flecha visual.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

38. Filtro de tabla por texto

Aspecto	Explicacion
Que hace	Filtra filas de una tabla segun lo que el usuario escribe.
Donde se usa en industria	Buscar usuarios, productos, facturas, pedidos, inventarios, registros o historiales.
Como se personaliza	Cambia el id del input, el id de la tabla y el criterio de busqueda.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<input type="search" id="tableFilter" placeholder="Filtrar tabla...">

<table class="data-table" id="ordersTable">
  <tbody>
    <tr><td>Carlos</td><td>120000</td><td>Pagado</td></tr>
    <tr><td>Laura</td><td>80000</td><td>Pendiente</td></tr>
  </tbody>
</table>
```

CSS del componente

styles.css

```
#tableFilter {
  width: 100%;
  padding: 12px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  margin-bottom: 16px;
}
tr.is-hidden {
  display: none;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class TableFilter {
  constructor(inputId, tableId) {
    this.input = document.getElementById(inputId);
    this.table = document.getElementById(tableId);
    this.init();
  }

  init() {
    if (!this.input || !this.table) return;
    this.input.addEventListener('input', () => this.filter());
  }

  filter() {
    const term = this.input.value.trim().toLowerCase();
    const rows = this.table.querySelectorAll('tbody tr');

    rows.forEach((row) => {
      const text = row.textContent.toLowerCase();
      row.classList.toggle('is-hidden', !text.includes(term));
    });
  }
}

// script.js
import { TableFilter } from './funciones.js';
new TableFilter('tableFilter', 'ordersTable');
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class TableFilter {
  constructor(inputId, tableId) {
    this.input = document.getElementById(inputId);
    this.table = document.getElementById(tableId);
    this.init();
  }

  init() {
    if (!this.input || !this.table) return;
    this.input.addEventListener('input', () => this.filter());
  }

  filter() {
    const term = this.input.value.trim().toLowerCase();
    const rows = this.table.querySelectorAll('tbody tr');
    rows.forEach((row) => {
      const text = row.textContent.toLowerCase();
      row.classList.toggle('is-hidden', !text.includes(term));
    });
  }
}

new TableFilter('tableFilter', 'ordersTable');
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
tableFilter	Id del input de busqueda.	Cambia segun tu HTML.
ordersTable	Id de la tabla.	Debe existir en el documento.
row.textContent	Texto completo de cada fila.	Puedes limitar a columnas especificas si quieres.
.is-hidden	Clase para ocultar filas.	Puedes usar visibility si quieres conservar espacio, pero no es comun.

Implementacion paso a paso

- 1 Crea un input search arriba de la tabla.
- 2 Asegura que la tabla tenga tbody.

- 3 Copia TableFilter.
- 4 Instancia con id del input y tabla.
- 5 Prueba con texto de distintas columnas.

Errores comunes y como solucionarlos

- No filtra: la tabla no tiene tbody o el id no coincide.
- Filtra muy lento en tablas enormes: combina con debounce.
- No encuentra acentos: normaliza texto con normalize si lo necesitas.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

39. Notas con localStorage

Aspecto	Explicacion
Que hace	Guarda notas simples en el navegador sin base de datos.
Donde se usa en industria	Borradores, notas rapidas, preferencias, tareas simples, formularios temporales y prototipos.
Como se personaliza	Cambia la clave localStorage, campos de nota y forma de renderizar.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<form id="noteForm" class="form">
  <input id="noteTitle" placeholder="Titulo de la nota" required>
  <textarea id="noteText" placeholder="Escribe una nota" required></textarea>
  <button class="btn" type="submit">Guardar nota</button>
</form>

<section id="notesList" class="notes-list"></section>
```

CSS del componente

styles.css

```
.notes-list {
  display: grid;
  gap: 12px;
  margin-top: 18px;
}
.note {
  padding: 16px;
  border-radius: 16px;
  background: #fff;
  border: 1px solid #e2e8f0;
}
.note_actions {
  display: flex;
  justify-content: flex-end;
  gap: 8px;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export function readJSON(key, fallback = []) {
  try {
    return JSON.parse(localStorage.getItem(key)) ?? fallback;
  } catch {
    return fallback;
  }
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export class NotesApp {
  constructor({ formId, titleId, textId, listId, storageKey = 'kit-notes' }) {
    this.form = document.getElementById(formId);
    this.title = document.getElementById(titleId);
    this.text = document.getElementById(textId);
    this.list = document.getElementById(listId);
    this.storageKey = storageKey;
    this.notes = readJSON(this.storageKey, []);
    this.init();
  }

  init() {
    this.render();
    this.form?.addEventListener('submit', (event) => {
      event.preventDefault();
      this.addNote();
    });
    this.list?.addEventListener('click', (event) => {
      const deleteButton = event.target.closest('[data-delete-note]');
      if (deleteButton) this.deleteNote(deleteButton.dataset.deleteNote);
    });
  }

  addNote() {
    const note = {
      id: crypto.randomUUID(),
      title: this.title.value.trim(),
      text: this.text.value.trim(),
      createdAt: new Date().toISOString()
    };
    this.notes.unshift(note);
    saveJSON(this.storageKey, this.notes);
    this.form.reset();
    this.render();
  }

  deleteNote(id) {
    this.notes = this.notes.filter((note) => note.id !== id);
    saveJSON(this.storageKey, this.notes);
    this.render();
  }

  render() {
    if (!this.list) return;
    this.list.innerHTML = this.notes.map((note) => `
      <article class="note">
        <h3>${note.title}</h3>
        <p>${note.text}</p>
        <div class="note__actions">
          <button type="button" data-delete-note="${note.id}">Eliminar</button>
        </div>
      </article>
    `).join('');
  }
}

// script.js
import { NotesApp } from './funciones.js';
new NotesApp({ formId: 'noteForm', titleId: 'noteTitle', textId: 'noteText', listId: 'notesList' });

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

function readJSON(key, fallback = []) {
  try { return JSON.parse(localStorage.getItem(key)) || fallback; }
  catch { return fallback; }
}
function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}
class NotesApp {
  constructor({ formId, titleId, textId, listId, storageKey = 'kit-notes' }) {
    this.form = document.getElementById(formId);
    this.title = document.getElementById(titleId);
    this.text = document.getElementById(textId);
    this.list = document.getElementById(listId);
    this.storageKey = storageKey;
    this.notes = readJSON(this.storageKey, []);
    this.init();
  }
  init() {
    this.render();
    this.form.addEventListener('submit', (event) => { event.preventDefault(); this.addNote(); });
    this.list.addEventListener('click', (event) => {
      const deleteButton = event.target.closest('[data-delete-note]');
      if (deleteButton) this.deleteNote(deleteButton.dataset.deleteNote);
    });
  }
  addNote() {
    const note = { id: Date.now().toString(), title: this.title.value.trim(), text: this.text.value.trim(), createdAt: new Date().toISOString() };
    this.notes.unshift(note);
    saveJSON(this.storageKey, this.notes);
    this.form.reset();
    this.render();
  }
  deleteNote(id) {
    this.notes = this.notes.filter((note) => note.id !== id);
    saveJSON(this.storageKey, this.notes);
    this.render();
  }
  render() {
    this.list.innerHTML = this.notes.map((note) => `<article class="note"><h3>${note.title}</h3><p>${note.text}</p><button data-delete-note="${note.id}">Eliminar</button></article>`).join('');
  }
}
new NotesApp({ formId: 'noteForm', titleId: 'noteTitle', textId: 'noteText', listId: 'notesList' });

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
kit-notes	Clave de localStorage.	Cambiala para no mezclar proyectos.
noteTitle/noteText	Campos del formulario.	Puedes agregar categoria, prioridad o fecha.
crypto.randomUUID()	Genera id unico.	Usa Date.now si quieres compatibilidad simple.
readJSON/saveJSON	Helpers reutilizables.	Usalos para cualquier dato guardado.

Implementacion paso a paso

- 1 Crea formulario y contenedor de notas.
- 2 Copia helpers readJSON y saveJSON.
- 3 Copia NotesApp.
- 4 Inicializa con los ids correctos.
- 5 Crea, recarga la pagina y confirma que la nota sigue guardada.

Errores comunes y como solucionarlos

- No guarda: revisa que localStorage no este bloqueado.
- Se duplican notas: no inicialices NotesApp dos veces.

- Se borra todo al cambiar de proyecto: usa storageKey diferente por app.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

40. Exportar JSON

Aspecto	Explicacion
Que hace	Descarga datos del navegador o de la app como archivo .json.
Donde se usa en industria	Exportar reportes, backups, notas, configuraciones, carritos, filtros o datos de prototipo.
Como se personaliza	Cambia el origen de datos, nombre del archivo y estructura del JSON.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="btn" id="exportNotes">Exportar notas JSON</button>
```

CSS del componente

styles.css

```
.btn-export {  
  background: linear-gradient(135deg, #0f766e, #2563eb);  
  color: #fff;  
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js


```
export class JSONExporter {
  static download(data, filename = 'datos.json') {
    const json = JSON.stringify(data, null, 2);
    const blob = new Blob([json], { type: 'application/json' });
    const url = URL.createObjectURL(blob);

    const link = document.createElement('a');
    link.href = url;
    link.download = filename;
    document.body.appendChild(link);
    link.click();
    link.remove();

    URL.revokeObjectURL(url);
  }
}

// script.js
import { JSONExporter, readJSON } from './funciones.js';
document.getElementById('exportNotes').addEventListener('click', () => {
  const notes = readJSON('kit-notes', []);
  JSONExporter.download(notes, 'mis-notas.json');
});
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class JSONExporter {
  static download(data, filename = 'datos.json') {
    const json = JSON.stringify(data, null, 2);
    const blob = new Blob([json], { type: 'application/json' });
    const url = URL.createObjectURL(blob);
    const link = document.createElement('a');
    link.href = url;
    link.download = filename;
    document.body.appendChild(link);
    link.click();
    link.remove();
    URL.revokeObjectURL(url);
  }
}

document.getElementById('exportNotes').addEventListener('click', () => {
  const notes = JSON.parse(localStorage.getItem('kit-notes')) || [];
  JSONExporter.download(notes, 'mis-notas.json');
});
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data	Informacion que quieres exportar.	Puede venir de localStorage, un array o una respuesta de API.
filename	Nombre del archivo descargado.	Usa extension .json.
Blob	Objeto que representa archivo en memoria.	Cambia type si exportas otro formato.
URL.revokeObjectURL	Libera memoria despues de descargar.	Dejalo para buenas practicas.

Implementacion paso a paso

- 1 Crea un boton de exportacion.
- 2 Define de donde saldrán los datos: array, localStorage o API.
- 3 Copia JSONExporter.

- 4 Llama `JSONExporter.download(datos, nombreArchivo)`.
- 5 Abre el archivo descargado y revisa que tenga estructura correcta.

Errores comunes y como solucionarlos

- El archivo sale vacio: revisa que los datos existan antes de exportar.
- El JSON sale en una sola linea: usa `JSON.stringify(data, null, 2)`.
- El navegador bloquea la descarga: asegúrate de llamar download desde un clic del usuario.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con `ToastManager` para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en `localStorage` si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con `aria-label`, `aria-hidden`, `focus-visible` y navegacion por teclado cuando aplique.

Cierre: como combinar las funcionalidades 26 a 40

Estas funcionalidades se pueden usar de manera individual, pero en proyectos reales normalmente se combinan. Por ejemplo, un catalogo profesional puede usar Cards dinamicas para mostrar productos, Busqueda debounce para filtrar por texto, Filtros chips para categorias, Ordenamiento para precio o nombre, Paginacion para dividir resultados, Modal para ver detalle del producto, Toast para confirmar acciones y Copy clipboard para copiar un codigo de descuento.

Patron recomendado para combinar busqueda + filtros + orden + paginacion

Ejemplo de estado global

```
const state = {
  originalItems: products,
  searchTerm: '',
  category: 'all',
  sort: 'name-asc',
  page: 1,
  perPage: 6,
};

function applyState() {
  let results = [...state.originalItems];

  if (state.searchTerm !== '') {
    results = results.filter((item) => item.name.toLowerCase().includes(state.searchTerm));
  }

  if (state.category !== 'all') {
    results = results.filter((item) => item.category === state.category);
  }

  results.sort((a, b) => a.name.localeCompare(b.name));

  const start = (state.page - 1) * state.perPage;
  const pageItems = results.slice(start, start + state.perPage);

  catalog.render(pageItems);
}
```

Checklist final general

- Si usas modulos, todas las clases reutilizables deben exportarse desde funciones.js.
- Si usas version normal, no uses import/export.
- Mantén nombres claros para ids y data-attributes.
- Usa localStorage solo para prototipos, preferencias o datos no sensibles.
- No guardes contraseñas, tokens privados o informacion delicada en localStorage.
- Documenta que variables cambiaste para poder reutilizar el componente despues.

Actualizacion funcional del bloque 26 a 40

Este anexo actualiza los codigos de copiar y pegar para que queden alineados con la pagina actual y funcionen como referencia implementable.

- Usa estos bloques como fuente vigente para HTML, CSS y JavaScript.
- El contenido anterior del PDF base se conserva, pero este anexo manda para implementacion.

26. Modal

Ventana emergente que superpone contenido importante usando el tag di logo nativo.

HTML copiar y pegar

```
<button type="button" data-open-modal="demoModal">Abrir modal</button>
<dialog id="demoModal">
  <p>Este modal ya es funcional.</p>
  <button type="button" data-close-modal>Cerrar</button>
</dialog>
```

CSS copiar y pegar

```
dialog {
  padding: 24px;
  border: 0;
  border-radius: 16px;
  max-width: 420px;
  box-shadow: 0 24px 50px rgba(15, 23, 42, 0.24);
}
.modal__content {
  display: grid;
  gap: 12px;
}
dialog::backdrop {
  background: rgba(15, 23, 42, 0.45);
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
export class ModalManager {
  constructor() {
    this.init();
  }

  init() {
    $$('[data-open-modal]').forEach((button) => {
      button.addEventListener("click", () =>
document.getElementById(button.dataset.openModal)?.showModal());
    });
    $$('[data-close-modal]').forEach((button) => {
      button.addEventListener("click", () =>
button.closest("dialog")?.close());
    });
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new ModalManager();
});
```

JS normal copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class ModalManager {
  constructor() {
    this.init();
  }
}
```

```
}

init() {
  $$('[data-open-modal]').forEach((button) => {
    button.addEventListener("click", () =>
document.getElementById(button.dataset.openModal)?.showModal());
  });
  $$('[data-close-modal]').forEach((button) => {
    button.addEventListener("click", () =>
button.closest("dialog")?.close());
  });
}
}

document.addEventListener("DOMContentLoaded", () => {
  new ModalManager();
});
```

27. Toast

Mensaje flotante que aparece abajo a la derecha y se descarta solo en segundos.

HTML copiar y pegar

```
<button id="showToast" type="button">Mostrar toast</button>
<div id="toastContainer"></div>
```

CSS copiar y pegar

```
#toastContainer {
  position: fixed;
  right: 20px;
  bottom: 20px;
  display: grid;
  gap: 10px;
}
.toast {
  padding: 12px 14px;
  border-radius: 12px;
  background: #0f172a;
  color: #ffffff;
  box-shadow: 0 18px 40px rgba(15, 23, 42, 0.25);
}
```

JS modular copiar y pegar

```
class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

document.addEventListener("DOMContentLoaded", () => {
  const toast = new ToastManager();
  document.getElementById("showToast")?.addEventListener("click", () => {
    toast.show("Toast mostrado correctamente.");
  });
});
```

JS normal copiar y pegar

```
class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}
```

```
}  
  
document.addEventListener("DOMContentLoaded", () => {  
  const toast = new ToastManager();  
  document.getElementById("showToast")?.addEventListener("click", () => {  
    toast.show("Toast mostrado correctamente.");  
  });  
});
```


28. Carrusel

Visor de imágenes o testimonios interactivo con controles de avanzar y retroceder.

HTML copiar y pegar

```
<div class="carousel" data-carousel>
  <div class="carousel__slide is-active">Slide 1</div>
  <div class="carousel__slide">Slide 2</div>
  <div class="carousel__slide">Slide 3</div>
  <div class="carousel__actions">
    <button type="button" data-carousel-prev>Anterior</button>
    <button type="button" data-carousel-next>Siguiente</button>
  </div>
</div>
```

CSS copiar y pegar

```
.carousel {
  display: grid;
  gap: 12px;
  max-width: 420px;
}

.carousel__slide {
  display: none;
  padding: 24px;
  border: 1px solid #cbd5e1;
  border-radius: 16px;
  background: #f8fafc;
}

.carousel__slide.is-active {
  display: block;
}

.carousel__actions {
  display: flex;
  gap: 8px;
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
export class Carousel {
  constructor(selector = "[data-carousel]") {
    this.carousel = $(selector);
    this.init();
  }
}
```

```
init() {
  this.carousel.forEach((carousel) => {
    const slides = $$(".carousel__slide", carousel);
    const next = $("[data-carousel-next]", carousel);
    const prev = $("[data-carousel-prev]", carousel);
    let index = 0;
    const render = () => slides.forEach((slide, i) =>
      slide.classList.toggle("is-active", i === index));
    next.addEventListener("click", () => { index = (index + 1) %
      slides.length; render(); });
    prev.addEventListener("click", () => { index = (index - 1 +
      slides.length) % slides.length; render(); });
  });
}
```

```

    });
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new Carousel();
});

```

JS normal copiar y pegar

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

```

```

class Carousel {
  constructor(selector = "[data-carousel]") {
    this.carousel = $(selector);
    this.init();
  }

  init() {
    this.carousel.forEach((carousel) => {
      const slides = $$(".carousel__slide", carousel);
      const next = $(".data-carousel-next", carousel);
      const prev = $(".data-carousel-prev", carousel);
      let index = 0;
      const render = () => slides.forEach((slide, i) =>
        slide.classList.toggle("is-active", i === index));
      next.addEventListener("click", () => { index = (index + 1) %
        slides.length; render(); });
      prev.addEventListener("click", () => { index = (index - 1 +
        slides.length) % slides.length; render(); });
    });
  }
}

document.addEventListener("DOMContentLoaded", () => {
  new Carousel();
});

```

29. Tooltip

Globo de ayuda que aparece por encima de un botón al pasar el mouse por encima.

HTML copiar y pegar

```
<button class="tooltip" data-tooltip="Información extra">?</button>
```

CSS copiar y pegar

```
.tooltip {  
  position: relative;  
  padding: 10px 14px;  
  border: 0;  
  border-radius: 999px;  
  background: #2563eb;  
  color: #ffffff;  
  cursor: help;  
}  
.tooltip::after {  
  content: attr(data-tooltip);  
  position: absolute;  
  left: 50%;  
  bottom: calc(100% + 8px);  
  transform: translateX(-50%);  
  white-space: nowrap;  
  padding: 8px 10px;  
  border-radius: 10px;  
  background: #0f172a;  
  color: #ffffff;  
  opacity: 0;  
  pointer-events: none;  
  transition: opacity 0.2s ease;  
}  
.tooltip:hover::after {  
  opacity: 1;  
}
```

JS modular copiar y pegar

```
// No requiere JS. Se maneja al 100% con selectores y pseudo-elementos de CSS.
```

JS normal copiar y pegar

```
// No requiere JS.
```

30. Badge

Etiqueta circular o numérica pequeña para indicar notificaciones o etiquetas.

HTML copiar y pegar

```
<span class="notification-badge">3</span>
```

CSS copiar y pegar

```
.notification-badge {  
  display: inline-flex; align-items: center; justify-content: center;  
  width: 32px; height: 32px; border-radius: 50%;  
  background: rgba(37, 99, 235, 0.9); color: white;  
}
```

JS modular copiar y pegar

```
// No requiere JS para renderizar, solo para cambiar el número si se quiere.
```

JS normal copiar y pegar

```
// Solo HTML y CSS
```

31. Copy clipboard

Permite copiar un bloque de texto al portapapeles del sistema con un clic.

HTML copiar y pegar

```
<code id="code">WEB-2026</code>
<button data-copy="#code">Copiar</button>
```

CSS copiar y pegar

```
button[data-copy] {
  padding: 10px 14px;
  border: 0;
  border-radius: 999px;
  background: #2563eb;
  color: #ffffff;
  cursor: pointer;
}
code {
  display: inline-block;
  margin-bottom: 10px;
  padding: 8px 10px;
  border-radius: 10px;
  background: #e2e8f0;
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
async function copyText(value) {
  if (navigator.clipboard?.writeText && window.isSecureContext) {
    await navigator.clipboard.writeText(value);
    return true;
  }
}
```

```
const helper = document.createElement("textarea");
helper.value = value;
helper.setAttribute("readonly", "");
helper.style.position = "fixed";
helper.style.opacity = "0";
document.body.appendChild(helper);
helper.select();
```

```
try {
  document.execCommand("copy");
  return true;
} finally {
  helper.remove();
}
}
```

```
class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }
}
```

```
show(message, time = 2800) {
  const toast = document.createElement("div");
```

```

        toast.className = "toast";
        toast.textContent = message;
        this.container.appendChild(toast);
        setTimeout(() => toast.remove(), time);
    }
}

export class CopyToClipboard {
    constructor(toast) {
        this.buttons = $$('[data-copy]');
        this.toast = toast;
        this.init();
    }

    init() {
        this.buttons.forEach((button) => {
            button.addEventListener("click", async () => {
                const target = document.querySelector(button.dataset.copy);
                if (!target) return;
                try {
                    await copyText(target.textContent);
                    this.toast.show("Texto copiado.");
                } catch {
                    this.toast.show("No se pudo copiar el texto.");
                }
            });
        });
    }
}

document.addEventListener("DOMContentLoaded", () => {
    const toast = new ToastManager();
    new CopyToClipboard(toast);
});

```

JS normal copiar y pegar

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

async function copyText(value) {
    if (navigator.clipboard?.writeText && window.isSecureContext) {
        await navigator.clipboard.writeText(value);
        return true;
    }

    const helper = document.createElement("textarea");
    helper.value = value;
    helper.setAttribute("readonly", "");
    helper.style.position = "fixed";
    helper.style.opacity = "0";
    document.body.appendChild(helper);
    helper.select();

    try {
        document.execCommand("copy");
        return true;
    } finally {
        helper.remove();
    }
}

```

```

    }
}

class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

class CopyToClipboard {
  constructor(toast) {
    this.buttons = $$('[data-copy]');
    this.toast = toast;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", async () => {
        const target = document.querySelector(button.dataset.copy);
        if (!target) return;
        try {
          await copyText(target.textContent);
          this.toast.show("Texto copiado.");
        } catch {
          this.toast.show("No se pudo copiar el texto.");
        }
      });
    });
  }
}

document.addEventListener("DOMContentLoaded", () => {
  const toast = new ToastManager();
  new CopyToClipboard(toast);
});

```

32. Cards dinámicas

Renderiza tarjetas visuales en el DOM a partir de una lista de objetos JSON.

HTML copiar y pegar

```
<div class="card-grid" id="catalogGrid"></div>
```

CSS copiar y pegar

```
.card-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));  
  gap: 16px;  
}  
.product-card {  
  padding: 16px;  
  border: 1px solid #cbd5e1;  
  border-radius: 16px;  
  background: #ffffff;  
}
```

JS modular copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {  
  const data = [  
    { name: "APA", price: 12000 },  
    { name: "Stout", price: 14000 },  
    { name: "Cheesecake", price: 11000 }  
  ];  
  const grid = document.getElementById("catalogGrid");  
  if (!grid) return;  
  
  grid.innerHTML = data.map((item) => `  
    <article class="product-card">  
      <h3>${item.name}</h3>  
      <p>${item.price.toLocaleString("es-CO")}</p>  
    </article>  
  `).join("");  
});
```

JS normal copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {  
  const data = [  
    { name: "APA", price: 12000 },  
    { name: "Stout", price: 14000 },  
    { name: "Cheesecake", price: 11000 }  
  ];  
  const grid = document.getElementById("catalogGrid");  
  if (!grid) return;  
  
  grid.innerHTML = data.map((item) => `  
    <article class="product-card">  
      <h3>${item.name}</h3>  
      <p>${item.price.toLocaleString("es-CO")}</p>  
    </article>  
  `).join("");  
});
```


33. Búsqueda

Filtra tarjetas del catálogo buscando coincidencias con el texto de entrada.

HTML copiar y pegar

```
<input id="catalogSearch" type="search" placeholder="Buscar producto">
<div class="card-grid" id="catalogGrid"></div>
```

CSS copiar y pegar

```
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 16px;
  margin-top: 16px;
}
.product-card {
  padding: 16px;
  border: 1px solid #cbd5e1;
  border-radius: 16px;
}
#catalogSearch {
  width: 100%;
  max-width: 320px;
  padding: 10px 12px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
}
```

JS modular copiar y pegar

```
function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}
```

```
document.addEventListener("DOMContentLoaded", () => {
  const data = ["APA", "Stout", "Sour", "Cheesecake"];
  const input = document.getElementById("catalogSearch");
  const grid = document.getElementById("catalogGrid");
  if (!input || !grid) return;

  const render = (term = "") => {
    grid.innerHTML = data
      .filter((item) => item.toLowerCase().includes(term.toLowerCase()))
      .map((item) => `<article class="product-card"><h3>${item}</h3></article>`)
      .join("") || "<p>No hay resultados.</p>";
  };

  input.addEventListener("input", debounce((event) => {
    render(event.target.value);
  }, 250));

  render();
});
```

JS normal copiar y pegar

```

function debounce(callback, delay = 300) {
  let timeoutId;
  return (...args) => {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => callback(...args), delay);
  };
}

document.addEventListener("DOMContentLoaded", () => {
  const data = ["APA", "Stout", "Sour", "Cheesecake"];
  const input = document.getElementById("catalogSearch");
  const grid = document.getElementById("catalogGrid");
  if (!input || !grid) return;

  const render = (term = "") => {
    grid.innerHTML = data
      .filter((item) => item.toLowerCase().includes(term.toLowerCase()))
      .map((item) => `<article class="product-
card"><h3>${item}</h3></article>`)
      .join("") || "<p>No hay resultados.</p>";
  };

  input.addEventListener("input", debounce((event) => {
    render(event.target.value);
  }, 250));

  render();
});

```

34. Filtros chips

Filtrado rápido de elementos mediante botones estilo pastilla de categorías.

HTML copiar y pegar

```
<div id="catalogChips">
  <button class="chip is-active" type="button" data-category="all">Todos</button>
  <button class="chip" type="button" data-category="cerveza">Cerveza</button>
  <button class="chip" type="button" data-category="postre">Postre</button>
</div>
<div class="card-grid" id="catalogGrid"></div>
```

CSS copiar y pegar

```
.chip {
  padding: 10px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 999px;
  background: #ffffff;
  cursor: pointer;
}
.chip.is-active {
  background: #2563eb;
  color: #ffffff;
  border-color: #2563eb;
}
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 16px;
  margin-top: 16px;
}
```

JS modular copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {
  const data = [
    { name: "APA", category: "cerveza" },
    { name: "Stout", category: "cerveza" },
    { name: "Cheesecake", category: "postre" }
  ];
  const chips = document.querySelectorAll("#catalogChips .chip");
  const grid = document.getElementById("catalogGrid");
  if (!grid || !chips.length) return;

  const render = (category = "all") => {
    grid.innerHTML = data
      .filter((item) => category === "all" || item.category === category)
      .map((item) => `<article class="product-card"><h3>${item.name}</h3><p>${item.category}</p></article>`)
      .join("");
  };

  chips.forEach((chip) => {
    chip.addEventListener("click", () => {
      chips.forEach((item) => item.classList.remove("is-active"));
      chip.classList.add("is-active");
      render(chip.dataset.category);
    });
  });
});
```

```
render();
});
```

JS normal copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {
  const data = [
    { name: "APA", category: "cerveza" },
    { name: "Stout", category: "cerveza" },
    { name: "Cheesecake", category: "postre" }
  ];
  const chips = document.querySelectorAll("#catalogChips .chip");
  const grid = document.getElementById("catalogGrid");
  if (!grid || !chips.length) return;

  const render = (category = "all") => {
    grid.innerHTML = data
      .filter((item) => category === "all" || item.category === category)
      .map((item) => `<article class="product-card"><h3>${item.name}</h3><p>${item.category}</p></article>`)
      .join("");
  };

  chips.forEach((chip) => {
    chip.addEventListener("click", () => {
      chips.forEach((item) => item.classList.remove("is-active"));
      chip.classList.add("is-active");
      render(chip.dataset.category);
    });
  });

  render();
});
```

35. Ordenamiento

Ordena el catálogo dinámico por orden alfabético, precio o valoraciones.

HTML copiar y pegar

```
<select id="catalogSort">
  <option value="name">Nombre</option>
  <option value="price">Precio</option>
</select>
<div class="card-grid" id="catalogGrid"></div>
```

CSS copiar y pegar

```
#catalogSort {
  padding: 10px 12px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
}
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 16px;
  margin-top: 16px;
}
```

JS modular copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {
  const data = [
    { name: "APA", price: 12000 },
    { name: "Stout", price: 14000 },
    { name: "Cheesecake", price: 11000 }
  ];
  const select = document.getElementById("catalogSort");
  const grid = document.getElementById("catalogGrid");
  if (!select || !grid) return;

  const render = () => {
    const items = [...data].sort((a, b) => {
      if (select.value === "price") return a.price - b.price;
      return a.name.localeCompare(b.name);
    });
    grid.innerHTML = items.map((item) => `<article class="product-card"><h3>${item.name}</h3><p>${item.price}</p></article>`).join("");
  };

  select.addEventListener("change", render);
  render();
});
```

JS normal copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {
  const data = [
    { name: "APA", price: 12000 },
    { name: "Stout", price: 14000 },
    { name: "Cheesecake", price: 11000 }
  ];
  const select = document.getElementById("catalogSort");
  const grid = document.getElementById("catalogGrid");
  if (!select || !grid) return;
```

```
const render = () => {
  const items = [...data].sort((a, b) => {
    if (select.value === "price") return a.price - b.price;
    return a.name.localeCompare(b.name);
  });
  grid.innerHTML = items.map((item) => `<article class="product-
card"><h3>${item.name}</h3><p>${item.price}</p></article>`).join("");
};

select.addEventListener("change", render);
render();
});
```

36. Paginaci3n

Segmenta el cat3logo en p3ginas para mejorar el rendimiento y legibilidad.

HTML copiar y pegar

```
<div class="card-grid" id="catalogGrid"></div>
<div class="pagination" id="catalogPagination"></div>
```

CSS copiar y pegar

```
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 16px;
}
.pagination {
  display: flex;
  gap: 8px;
  margin-top: 16px;
}
.pagination button {
  padding: 8px 12px;
  border: 1px solid #cbd5e1;
  border-radius: 999px;
  background: #ffffff;
}
.pagination button.is-active {
  background: #2563eb;
  color: #ffffff;
}
```

JS modular copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {
  const data = Array.from({ length: 10 }, (_, index) => ({ name: 'Producto ${index + 1}' }));
  const grid = document.getElementById("catalogGrid");
  const pagination = document.getElementById("catalogPagination");
  let page = 1;
  const perPage = 4;
  if (!grid || !pagination) return;
```

```
  const render = () => {
    const totalPages = Math.ceil(data.length / perPage);
    const items = data.slice((page - 1) * perPage, page * perPage);
    grid.innerHTML = items.map((item) => '<article class="product-card"><h3>${item.name}</h3></article>').join("");
    pagination.innerHTML = Array.from({ length: totalPages }, (_, index) => `
      <button type="button" data-page="${index + 1}" class="${index + 1 ===
page ? "is-active" : ""}">${index + 1}</button>
    `).join("");
    pagination.querySelectorAll("button").forEach((button) => {
      button.addEventListener("click", () => {
        page = Number(button.dataset.page);
        render();
      });
    });
  };
};
```

```
render();
```

```
});
```

JS normal copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {
  const data = Array.from({ length: 10 }, (_, index) => ({ name: 'Producto ${index
+ 1}' }));
  const grid = document.getElementById("catalogGrid");
  const pagination = document.getElementById("catalogPagination");
  let page = 1;
  const perPage = 4;
  if (!grid || !pagination) return;

  const render = () => {
    const totalPages = Math.ceil(data.length / perPage);
    const items = data.slice((page - 1) * perPage, page * perPage);
    grid.innerHTML = items.map((item) => '<article class="product-
card"><h3>${item.name}</h3></article>').join("");
    pagination.innerHTML = Array.from({ length: totalPages }, (_, index) => '
    <button type="button" data-page="${index + 1}" class="${index + 1} ===
page ? "is-active" : """>${index + 1}</button>
    ').join("");
    pagination.querySelectorAll("button").forEach((button) => {
      button.addEventListener("click", () => {
        page = Number(button.dataset.page);
        render();
      });
    });
  };

  render();
});
```


37. Tabla ordenable

Permite ordenar las filas de una tabla al hacer clic en los encabezados.

HTML copiar y pegar

```
<input id="tableSearch" type="search" placeholder="Filtrar filas">
<table id="demoTable" class="data-table">
  <thead>
    <tr>
      <th data-sort-table="name">Nombre</th>
      <th data-sort-table="price">Precio</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>APA</td><td>12000</td></tr>
    <tr><td>Stout</td><td>14000</td></tr>
    <tr><td>Sour</td><td>13000</td></tr>
  </tbody>
</table>
```

CSS copiar y pegar

```
.data-table {
  width: 100%;
  border-collapse: collapse;
}
.data-table th,
.data-table td {
  padding: 10px 12px;
  border-bottom: 1px solid #cbd5e1;
  text-align: left;
}
.data-table th {
  cursor: pointer;
  color: #2563eb;
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
export class SortableTable {
  constructor(tableId, searchId) {
    this.table = document.getElementById(tableId);
    this.search = document.getElementById(searchId);
    this.direction = 1;
    this.init();
  }

  init() {
    $$('[data-sort-table]', this.table).forEach((header, index) => {
      header.addEventListener("click", () => this.sort(index));
    });
    this.search.addEventListener("input", () => this.filter());
  }

  sort(index) {
    const rows = $$("tbody tr", this.table);
    rows.sort((a, b) =>
```

```

a.children[index].textContent.localeCompare(b.children[index].textContent,
undefined, { numeric: true }) * this.direction);
    this.direction *= -1;
    rows.forEach((row) => this.table.tBodies[0].appendChild(row));
}

filter() {
    const term = this.search.value.toLowerCase();
    $$("tbody tr", this.table).forEach((row) => row.style.display =
row.textContent.toLowerCase().includes(term) ? "" : "none");
}
}

document.addEventListener("DOMContentLoaded", () => {
    new SortableTable("demoTable", "tableSearch");
});

```

JS normal copiar y pegar

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

class SortableTable {
    constructor(tableId, searchId) {
        this.table = document.getElementById(tableId);
        this.search = document.getElementById(searchId);
        this.direction = 1;
        this.init();
    }

    init() {
        $$("[data-sort-table]", this.table).forEach((header, index) => {
            header.addEventListener("click", () => this.sort(index));
        });
        this.search.addEventListener("input", () => this.filter());
    }

    sort(index) {
        const rows = $$("tbody tr", this.table);
        rows.sort((a, b) =>
a.children[index].textContent.localeCompare(b.children[index].textContent,
undefined, { numeric: true }) * this.direction);
        this.direction *= -1;
        rows.forEach((row) => this.table.tBodies[0].appendChild(row));
    }

    filter() {
        const term = this.search.value.toLowerCase();
        $$("tbody tr", this.table).forEach((row) => row.style.display =
row.textContent.toLowerCase().includes(term) ? "" : "none");
    }
}

document.addEventListener("DOMContentLoaded", () => {
    new SortableTable("demoTable", "tableSearch");
});

```

38. Filtro de tabla

Filtra las filas de una tabla según las coincidencias con el texto ingresado.

HTML copiar y pegar

```
<input id="tableSearch" type="search" placeholder="Filtrar...">
```

CSS copiar y pegar

```
/* Estilos del buscador de tabla */
```

JS modular copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {  
  const input = document.getElementById("tableSearch");  
  const rows = document.querySelectorAll("#demoTable tbody tr");  
  input?.addEventListener("input", () => {  
    const term = input.value.toLowerCase();  
    rows.forEach((row) => {  
      row.style.display = row.textContent.toLowerCase().includes(term) ? "" :  
"none";  
    });  
  });  
});
```

JS normal copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {  
  const input = document.getElementById("tableSearch");  
  const rows = document.querySelectorAll("#demoTable tbody tr");  
  input?.addEventListener("input", () => {  
    const term = input.value.toLowerCase();  
    rows.forEach((row) => {  
      row.style.display = row.textContent.toLowerCase().includes(term) ? "" :  
"none";  
    });  
  });  
});
```

39. LocalStorage notas

Guarda y recupera un listado de notas del almacenamiento local del navegador.

HTML copiar y pegar

```
<textarea id="noteInput" rows="4" placeholder="Escribe una nota"></textarea>
<button id="saveNote" type="button">Guardar</button>
<button id="exportNotes" type="button">Exportar JSON</button>
<ul id="noteList"></ul>
<div id="toastContainer"></div>
```

CSS copiar y pegar

```
#noteInput {
  width: 100%;
  min-height: 120px;
  padding: 12px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
}
#noteList {
  display: grid;
  gap: 10px;
  padding: 0;
  list-style: none;
}
#noteList li {
  display: flex;
  justify-content: space-between;
  gap: 12px;
  padding: 12px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
}
```

JS modular copiar y pegar

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}
```

```
function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}
```

```
class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }
}
```

```
show(message, time = 2800) {
  const toast = document.createElement("div");
```

```

    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

```

```

export class NotesManager {
  constructor(toast) {
    this.input = $("#noteInput");
    this.saveButton = $("#saveNote");
    this.list = $("#noteList");
    this.exportButton = $("#exportNotes");
    this.toast = toast;
    this.notes = readJSON("kit-notes", []);
    this.init();
  }
}

```

```

init() {
  this.saveButton.addEventListener("click", () => this.add());
  this.exportButton.addEventListener("click", () => this.export());
  this.render();
}

```

```

add() {
  const text = this.input.value.trim();
  if (!text) return this.toast.show("Escribe una nota primero.");
  this.notes.push({ id: Date.now(), text });
  saveJSON("kit-notes", this.notes);
  this.input.value = "";
  this.render();
  this.toast.show("Nota guardada.");
}

```

```

remove(id) {
  this.notes = this.notes.filter((note) => note.id !== id);
  saveJSON("kit-notes", this.notes);
  this.render();
}

```

```

render() {
  this.list.innerHTML = this.notes.map((note) =>
    '<li><span>${note.text}</span><button class="danger-button" data-
    id="${note.id}">Borrar</button></li>').join("");
  $$('button', this.list).forEach((button) => button.addEventListener("click",
    () => this.remove(Number(button.dataset.id))));
}

```

```

export() {
  const blob = new Blob([JSON.stringify(this.notes, null, 2)], { type:
"application/json" });
  const link = document.createElement("a");
  link.href = URL.createObjectURL(blob);
  link.download = "notas.json";
  link.click();
  URL.revokeObjectURL(link.href);
}

```

```

}

document.addEventListener("DOMContentLoaded", () => {
  const toast = new ToastManager();
  new NotesManager(toast);
});

```

JS normal copiar y pegar

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

```

```

function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

```

```

function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch {
    return fallback;
  }
}

```

```

class ToastManager {
  constructor(containerId = "toastContainer") {
    this.container = document.getElementById(containerId);
  }

```

```

  show(message, time = 2800) {
    const toast = document.createElement("div");
    toast.className = "toast";
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => toast.remove(), time);
  }
}

```

```

class NotesManager {
  constructor(toast) {
    this.input = $("#noteInput");
    this.saveButton = $("#saveNote");
    this.list = $("#noteList");
    this.exportButton = $("#exportNotes");
    this.toast = toast;
    this.notes = readJSON("kit-notes", []);
    this.init();
  }

```

```

  init() {
    this.saveButton.addEventListener("click", () => this.add());
    this.exportButton.addEventListener("click", () => this.export());
    this.render();
  }

```

```

  add() {
    const text = this.input.value.trim();
    if (!text) return this.toast.show("Escribe una nota primero.");
  }

```

```

    this.notes.push({ id: Date.now(), text });
    saveJSON("kit-notes", this.notes);
    this.input.value = "";
    this.render();
    this.toast.show("Nota guardada.");
  }

  remove(id) {
    this.notes = this.notes.filter((note) => note.id !== id);
    saveJSON("kit-notes", this.notes);
    this.render();
  }

  render() {
    this.list.innerHTML = this.notes.map((note) =>
    '<li><span>${note.text}</span><button class="danger-button" data-
id="${note.id}">Borrar</button></li>').join("");
    $('button', this.list).forEach((button) => button.addEventListener("click",
    () => this.remove(Number(button.dataset.id))));
  }

  export() {
    const blob = new Blob([JSON.stringify(this.notes, null, 2)], { type:
"application/json" });
    const link = document.createElement("a");
    link.href = URL.createObjectURL(blob);
    link.download = "notas.json";
    link.click();
    URL.revokeObjectURL(link.href);
  }
}

document.addEventListener("DOMContentLoaded", () => {
  const toast = new ToastManager();
  new NotesManager(toast);
});

```

40. Export JSON

Genera y descarga un archivo .json con los datos guardados por el usuario.

HTML copiar y pegar

```
<button id="exportNotes" type="button">Descargar notas.json</button>
```

CSS copiar y pegar

```
#exportNotes {  
  padding: 10px 14px;  
  border: 0;  
  border-radius: 999px;  
  background: #2563eb;  
  color: #ffffff;  
  cursor: pointer;  
}
```

JS modular copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {  
  const notes = [  
    { id: 1, text: "Primera nota" },  
    { id: 2, text: "Segunda nota" }  
  ];  
  document.getElementById("exportNotes")?.addEventListener("click", () => {  
    const blob = new Blob([JSON.stringify(notes, null, 2)], { type:  
"application/json" });  
    const link = document.createElement("a");  
    link.href = URL.createObjectURL(blob);  
    link.download = "notas.json";  
    link.click();  
    URL.revokeObjectURL(link.href);  
  });  
});
```

JS normal copiar y pegar

```
document.addEventListener("DOMContentLoaded", () => {  
  const notes = [  
    { id: 1, text: "Primera nota" },  
    { id: 2, text: "Segunda nota" }  
  ];  
  document.getElementById("exportNotes")?.addEventListener("click", () => {  
    const blob = new Blob([JSON.stringify(notes, null, 2)], { type:  
"application/json" });  
    const link = document.createElement("a");  
    link.href = URL.createObjectURL(blob);  
    link.download = "notas.json";  
    link.click();  
    URL.revokeObjectURL(link.href);  
  });  
});
```